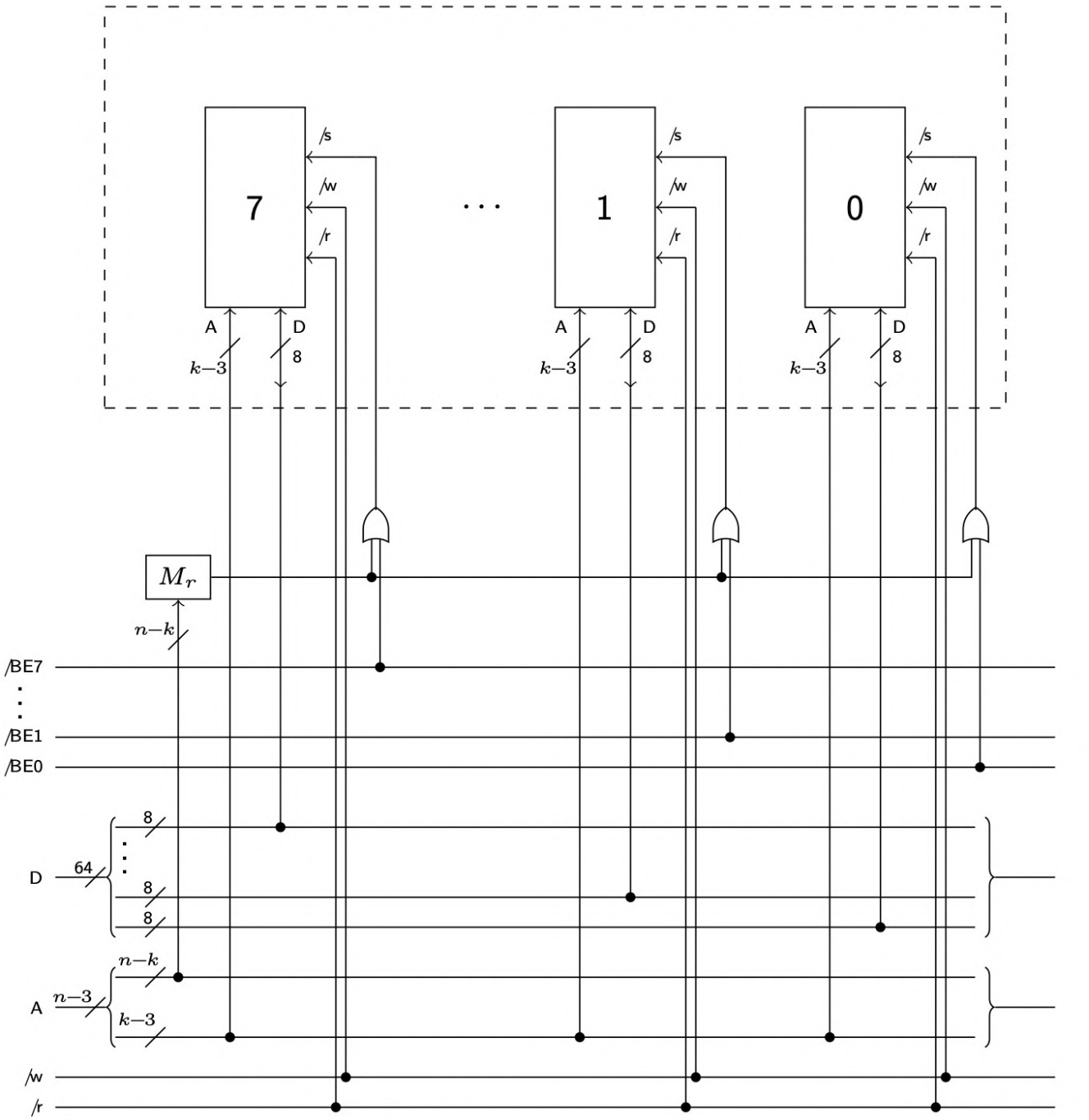


CALCOLATORI ELETTRONICI

Architettura del calcolatore moderno

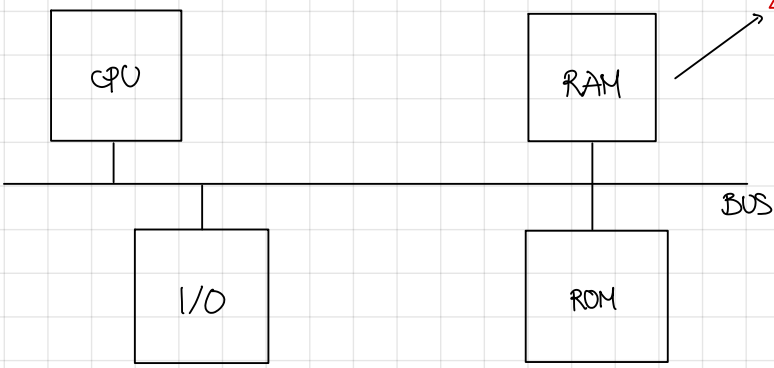
- INTRODUZIONE → Chi fa cosa?

Il computer può eseguire ogni calcolo richiesto da qualunque programma. Il computer è quindi comandato dal software → la CPU preleva istruzione dalla RAM e la esegue. Essa vede solo il contenuto dei registri ma NON ricorda ciò che ha eseguito precedentemente e NON sa cosa eseguirà in futuro. La RAM è solo un CONTENITORE del software in esecuzione. Il sistema operativo comanda ad un livello più alto e CEDE il controllo al software in esecuzione → Viene eseguito UN SOFTWARE per VOLTA. Una volta terminata l'esecuzione, il controllo ritorna al software precedente.

↓

IN GENERALE: tutta la terminologia si basa sulle istruzioni aritmetiche in cui abbiamo 2 operandi ed un risultato

- CALCOLATORI MODERNI



⚠ ATT. non c'è scritto cos'è una particolare istruzione né vi sono scritti gli indirizzi

↓

In tutte le strutture dati è sufficiente conoscere l'indirizzo base e per accedere agli altri elementi il calcolo del loro indirizzo è veloce

memoria

Abbiamo la convergenza di 2 indirizzi: permettono di accedere sia all'intera parola (32 bit) oppure ai byte singoli.

↓

Queste modalità di lettura e scrittura sono poi state estese anche allo spazio di I/O

↓

spazio di I/O

Il software ha il controllo dell'I/O e quindi per interagire con i computer moderni è sempre necessario un software → All'avvio viene caricato un programma presente in ROM: bootstrap

↓

Tutto l'I/O viene ridotto a lettura/scrittura: ogni periferica ha un'interfaccia con una serie di registri → Leggere o scrivere in una periferica ha degli effetti collaterali tipicamente irreversibili. Anche la lettura è distruttiva (registro FI).

↓

Spazio di indirizzamento: serie di indirizzi che possono essere indistintamente di memoria e I/O → Viene fatta una distinzione netta tra indirizzi e memoria

↓

Oggi è presente RAM, periferiche e memoria video. Assumiamo che, in caso noi leggessimo un indirizzo non allocato, venga restituito un valore casuale.

↓

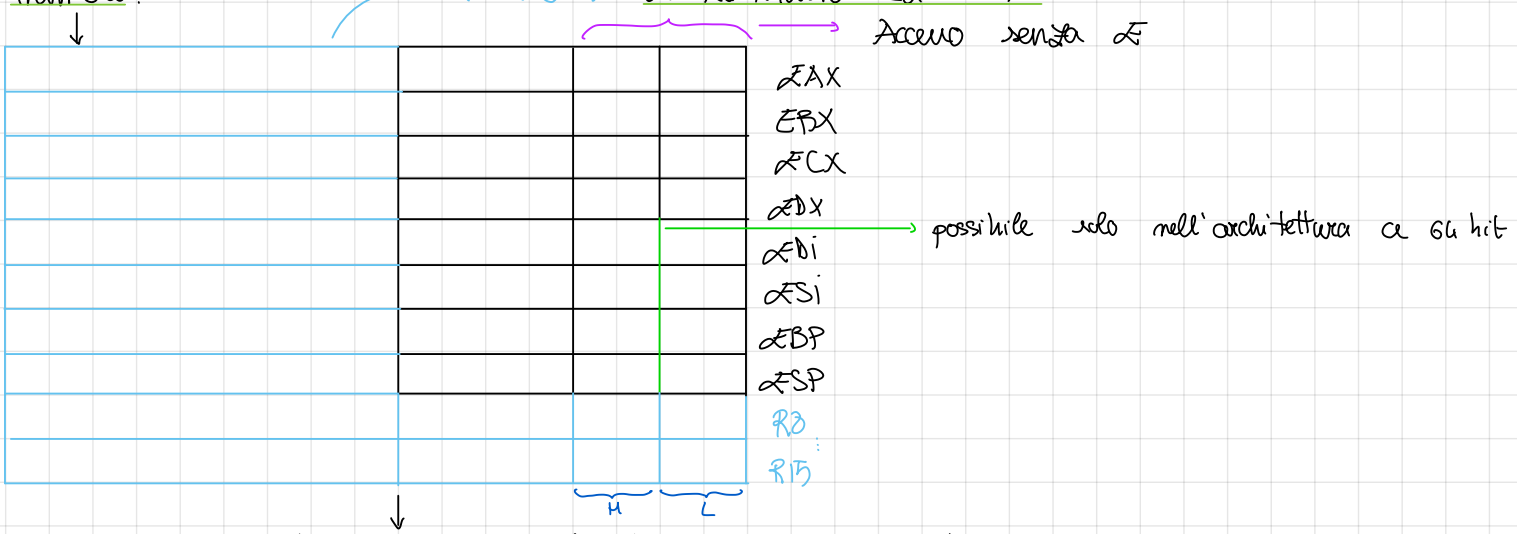
Lo spazio di I/O viene utilizzato dalla CPU solo in caso di IN/OUT, altrimenti viene utilizzata la memoria

Per utilizzare una periferica connessa al bus è necessario che questa abbia un indirizzo non sovrapposto ad altre periferiche → accesso tramite indirizzo

CPU

Il ciclo fondamentale prevede il registro IP che contiene la prossima istruzione da eseguire. Essa preleva l'istruzione dalla memoria, la decodifica e la esegue ed incrementa IP.

Analizziamo l'architettura AMD a 64 bit: estende l'architettura a 32 bit semplificandone alcuni aspetti e permette di eseguire anche programmi a 32 bit. Vengono estesi i registri a 64 bit e vengono raddoppiati in numero.



È possibile adesso avere entrambi gli operandi in memoria e rimangono le stesse 3 tipologie di operandi: immediato, registro, memoria con la stessa sintassi già vista precedentemente in Assembler. Abbiamo poi una nuova tipologia data da offset (%RIP)

Specifica quanto dista l'istruzione da RIP che ⚠ già contiene la prossima istruzione

Per quasi tutte le istruzioni gli immediati e gli offset rimangono a 32 bit per non allungare tutte le istruzioni a 9 byte. L'unica istruzione a supportare questa modalità è MOVABS

Dobbiamo capire PRIMA dove è posizionato il programma in quanto è richiesta una sequenza differente di istruzioni a seconda che questi siano vicini o lontani
↳ L'assemblatore definisce il contenuto della memoria a partire da un programma

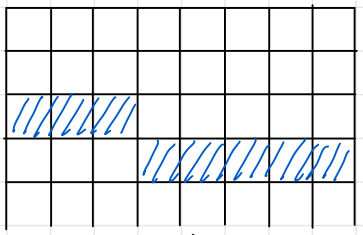
Indirizzando un dato tramite offset e RIP, la sezione DATA e TEXT non possono distare più di 2GB. Questo vale anche per le istruzioni di CALL

SPAZIO di INDIRIZZAMENTO a 64 bit

In teoria è a 64 bit, in pratica non tutti gli indirizzi sono accessibili perché i processori più comuni implementano 48 bit e quindi possono accedere al massimo 256 TB
↳ Divisi in 2 tronconi con "buco" nel mezzo. Gli indirizzi devono essere **normalizzati**: i 16 bit mancanti devono essere uguali al bit 47 quindi o tutti 0 o tutti 1 → 16 bit non importanti per la normalizzazione

Oggetti allineati: Indirizzo di partenza multiplo di una certa potenza di 2 di un certo indirizzo dato

Immaginiamo la memoria a 64 bit come composta da **tutte righe di 8 byte**



→ il processore in un'unica operazione in memoria può leggere una riga di 8 byte o sue sotto-parti. Sono invece necessarie più operazioni di memoria per leggere più righe

Se una variabile si trova nel modo indicato in blu sono richieste due operazioni in memoria ed in più il riordino dei pezzi che si trovano nella parte ingigliata del bus. Alcune architetture (ARM) non accettano proprio le variabili poste in questo modo, mentre Intel supporta questa cosa

Se l'oggetto è **più grande di un byte** (0x1122). Possono esserci 2 possibilità

- **Big Endian**: 0x 11 22 (1 2) → utilizzata da internet
- **Little Endian**: 0x 11 22 (2 1) → le macchine Intel sono Little Endian

Utilizziamo questa memoria:

7	6	5	4	3	2	1	0	
						11	22	0
								8
								16

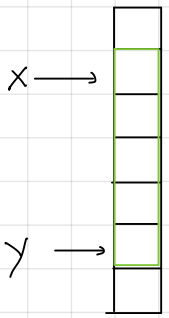
→ Byte numerati da dx a sx

NOMENCLATURA degli INDIRIZZI → Ogni byte è rappresentato da 2 cifre esadecimali

↳ In generale, è importante ricordare: $2^a = 1\underbrace{000\ldots000}_{a}$

$2^a - 1 = 1\underbrace{111\ldots111}_{a}$

offset: distanza tra 2 indirizzi:



→ Δ: quanti indirizzi dobbiamo sollevare

↓

Tra x ed x è $\emptyset$

intervalli: $[x, y)$ per convenzione e perché la dimensione dell'intervallo è $y-x$

↓

Possiamo **comporre** gli intervalli: $[x, y) + [y, z)$: $z - x$ con y contata una sola volta

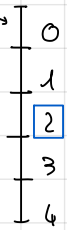
Circolarità:

Gli indirizzi sono **circolari** perché realizziamo l'aritmetica modulare: $\mod 2^n$. Questo accade in quanto abbiamo soltanto n fili. Sommando infatti 1 al massimo rappresentabile, si ottiene un numero composto da n zeri ed 1 all'MSB che viene scartato per la presenza dei soli n fili.

Se l'offset è realizzato su n bit non importa il fatto che sia positivo o negativo. Se l'offset è realizzato su meno di n bit ci interessa invece sapere se è positivo o negativo → Vengono estesi tenendo conto del segno. Consideriamo inoltre sempre intervalli che **non fanno overflow** dall'altra parte

confini / boundaries: Lo spazio di indirizzamento viene diviso in parti di una determinata dimensione (potenze di 2). Gli indirizzi che si trovano all'**inizio** di ciascuna di queste parti vengono chiamati **boundaries**. Se

guardo un indirizzo al confine di una porzione di 2^b b.t noto che avrà b bit a 0 (quelli meno significativi). Una **regione naturale di 2^b** è l'insieme degli indirizzi che sta tra i confini di 2^b.

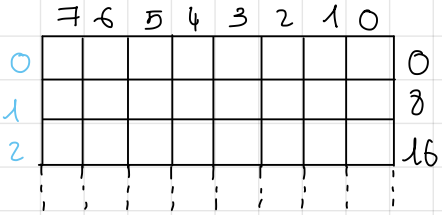


→ Posiamo creare una **geografia** assegnando a ciascuna regione un numero che identifica tutti gli indirizzi che cadono all'interno di ciascuna regione naturale

↓

Il 2 compare in tutti gli indirizzi. In particolare, i b bit meno significativi cambiano, mentre gli n-b bit più significativi sono costanti e uguali a 2. I meno significativi vanno da 0 a 2^b-1 e rappresentano l'offset all'interno della regione

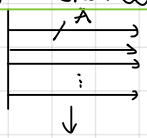
Applicazione:



→ L'indirizzo di ciascuna riga di 8 byte può essere scomposto nel suo numero di regione naturale e nell'offset rispetto ad essa

↓

Quando la CPU emette un indirizzo, essa fornisce solo il numero di riga ed 8 byte enable di modo da dire quali byte della riga le interessano:

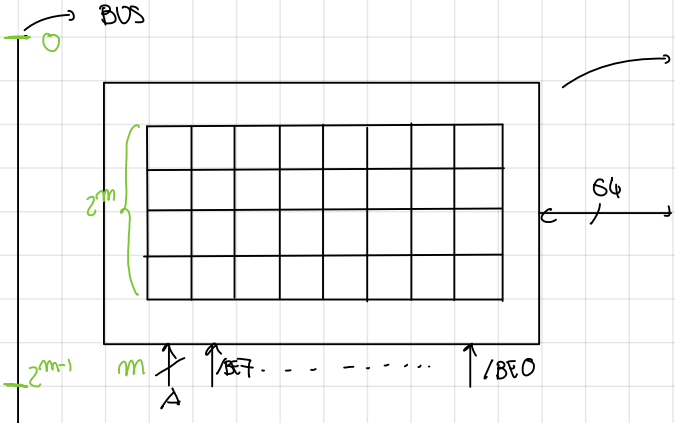


↓

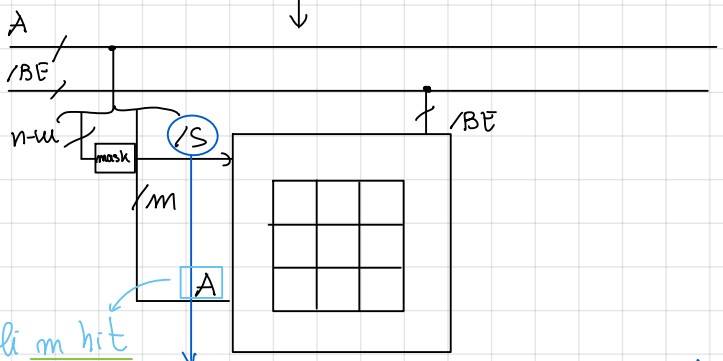
È più flessibile perché potrebbe leggere byte sparpagliati ma è limitante perché può leggere una riga per volta → **MOV 7, % EAX** : 3 operazioni:

a. A=0 e /BE7=0
b. A=1 e /BE0 = /BE1 = /BE2=0
c. Shift

ORGANIZZAZIONE MEMORIA



Come fa a capire che l'indirizzo è rivolto a lui?



Gli m bit meno significativi specificano l'offset

Gli n-m bit più significativi identificano il numero della regione

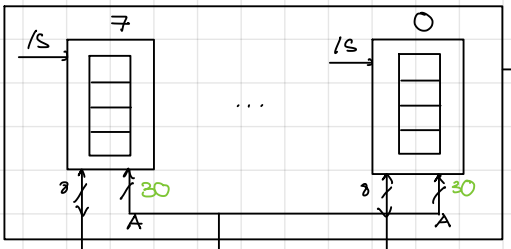
↓

Ogni cosa che occupa indirizzi è un **oggetto**. Si parla di indirizzo dell'oggetto riferendosi al **primo byte** che occupa. L'allineamento riguarda le proprietà di tale indirizzo

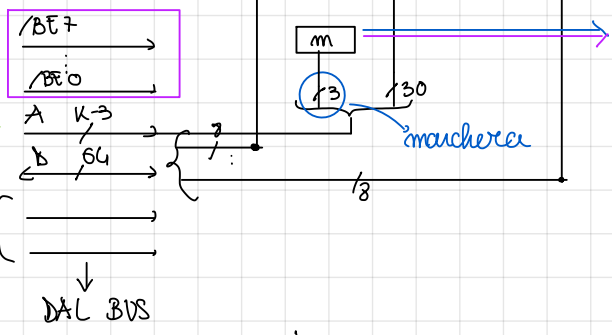
Si dice che un oggetto è **allineato a 2^b** se il suo indirizzo è un confine di 2^b. Si dice che un oggetto è **allineato ad un altro oggetto** quando l'altro

Ha una dimensione che è potenza di 2 e che il primo è allineato a quella potenza di 2 → avere allineato ad un long significa averel allineati ad 8 byte. Si parla di **allineamento naturale** quando la dimensione di un oggetto è una potenza di 2 ed è allineato a se stesso.

per rispondere a richieste



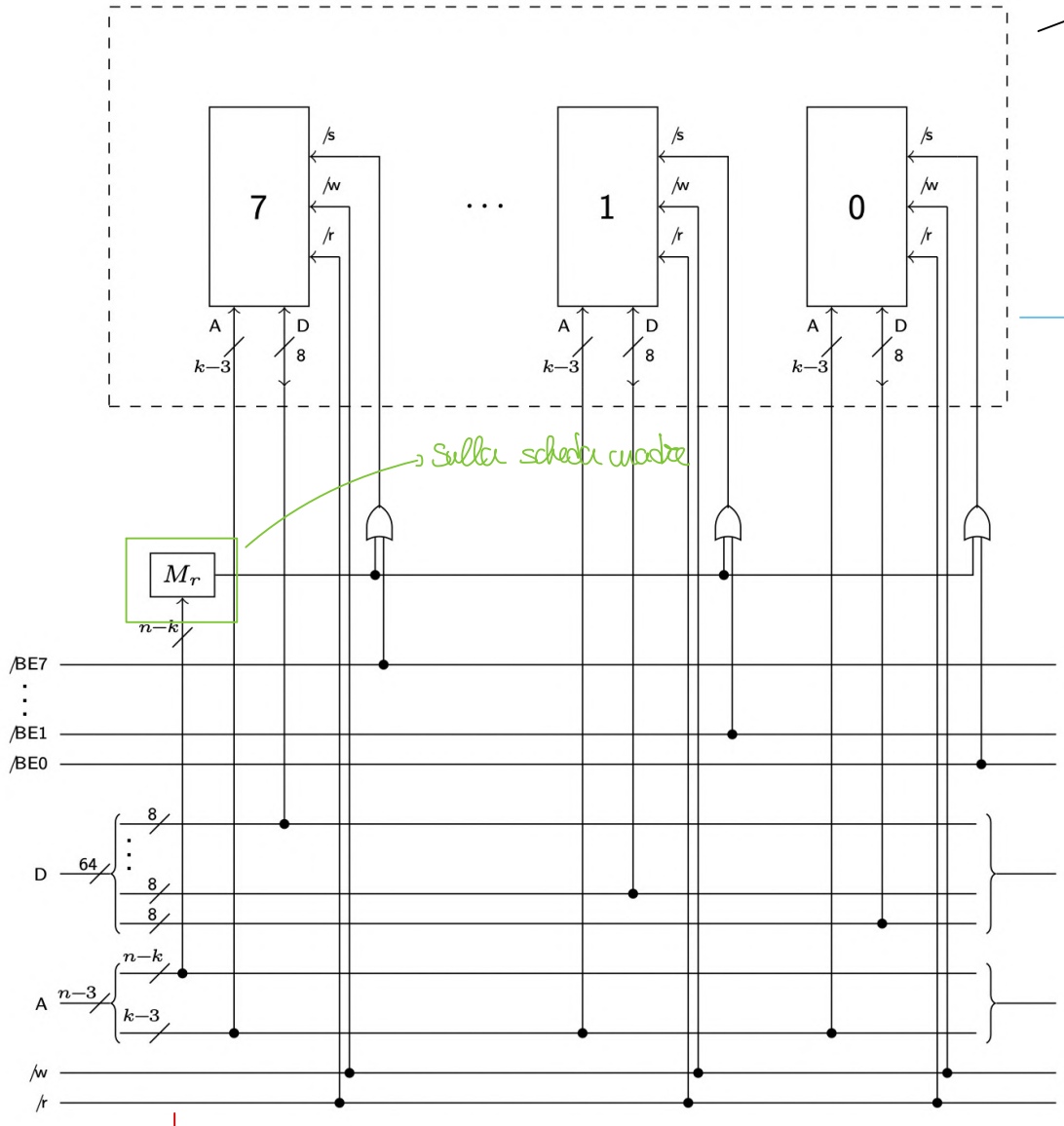
Utilizziamo **8 chip** di RAM da **1 byte** ciascuno



Deve essere verificata sia la condizione dei fili di indirizzo che quella dei byte enable

Supponiamo che la memoria sia da 8 GB : Abbiamo $k = 36$

↓ Scritto meglio



Se vogliamo fare un accesso disallineato dobbiamo fare 2 **operazioni** già dette in precedenza

Parte tratteggiata: si può comprare a parte

⚠ Att: i valori possono essere 0/1 ma non è mai vero che non contengono niente, al massimo **valori non significativi**

⚠ Att: possono fare **un'operazione per volta**

Il compilatore

Utilizziamo lo standard **system V** che definisce le seguenti convenzioni (su Linux)

- char ⟶ 1 byte
- short ⟶ 2 byte
- int ⟶ 4 byte
- long ⟶ 8 byte

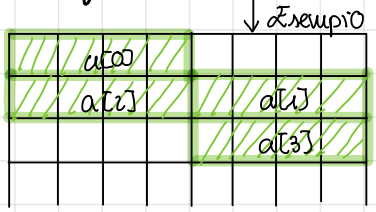
Allineamento **naturale**: ciascuno alla propria dimensione

↓ serve a **semplificare i calcoli**

↳ Tipi derivati: i campi all'interno rispettano l'allineamento dei tipi base

array: int a[...]; ⟶ viene garantito l'allineamento a 4 byte a partire da 0 (primo elemento)

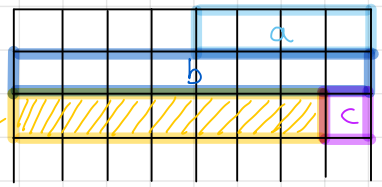
↓ esempio



⟶ array di 4 elementi

strutture: struct { ⟶ **Allineamento**: vincolo degli allineamenti dei suoi elementi

```
int a;
long b;
char c;
};
```



⟶ la struttura inizia all'**inizio** di una riga e i campi vanno allocati **in ordine**

↓

Mettiamo int a al primo offset che rispetta l'allineamento

↳ **Regola generale**

spreccati

La **dimensione dell'oggetto** deve essere un **multiplo dell'allineamento**

puntatori/riferimenti: Allineamento

↓ cosa fa il compilatore?

```
int a = 1
long b = 2;
char c[10] = {0};

int main() {
  b = 3;
}
```

⟶ . data: se all'assemblatore non si dice niente, mette una variabile dietro l'altra

a:

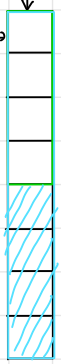
. long 1 ⟶ 4 byte: dimensione di un int

b:

↓ . quad 2 ⟶ 8 byte: dimensione di un long

Quando viene incontrata un'etichetta l'assemblatore si segna il valore del contatore avanti prima del cambio del suo valore

. balign: incrementa il contatore fino ad arrivare ad 8

0 ⟶ 

c = 0 ⟶ Contatori **separati** tra parte data e parte text

c = 4

⟶ Con balign: Vengono spreccati dei bit in modo da allineare ad 8

↓

Ci pensa il **compilatore** a mettere la direttiva in quanto l'assemblatore fa ciò che c'è scritto

↓ Il contatore di b in questo caso vale 3

L'entry point del collegatore è start: definito in un file oggetto automaticamente collegato quando si esegue un file C++. Questo svolge delle azioni e poi chiama main e si preoccupa di dire al sistema operativo che il programma è terminato

↓ in Assembler

main: ⟶ Deve essere dichiarata in modo **globale**: .global main

:

ret

↓

È un'etichetta come le altre posta in .TEXT. Le etichette locali vengono ignorate dal collegatore, mentre vengono considerate solo quelle globali

GESTIONE di FUNZIONI

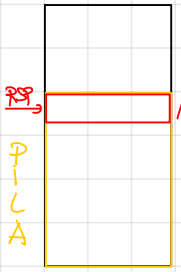
La funzione è un codice che viene chiamato da un'altra funzione. La stessa funzione può essere chiamata da più parti diverse e deve sapere dove ritornare.

↓

f() ⟶ f: ⟶ Utilizziamo le istruzioni **call**, **ret** e lo **stack** oppure un **registro** ⟶ Svantaggio: ci sempre solucito

↓

Memoria ⟵ Utilizzeremo quindi lo stack



RSP contiene l'indirizzo dell'ultima **riga** utilizzata nella pila

↓

CALL destinazione: fa la PUSH di destination e salta

↓

RET: prende la riga puntata da RSP e la trasferisce in RIP

Dobbiamo inoltre gestire **parametri**, **variabili locali** e **valori di ritorno**

a. VARIABILI LOCALI

Per permettere la ricorsività, le variabili locali vengono messe in **pila**.

La funzione viene chiamata e per prima cosa **riserva** spazio nella pila (Sposta RSP più in alto)

↓

Allocazione **automatica**: le variabili vengono distrutte quando la funzione termina

⚠ problema: Le variabili locali si trovano in **punti diversi** in ciascuna esecuzione della funzione

↓

Utilizziamo un registro di riferimento: %RSP

b. PARAMETRI

chi fa cosa o meglio chi fa cosa

chiamante chiamato

↓

compilazione separata

chiamante

```
typer (type1, ..., typen);

f(espr1, espr2...)
```

chiamato

```
typer f(type1,..., typen)
{ ...
  :
  return 1xp0
}
```

↓

importante perché è possibile ricompilare soltanto un file, utile per programmi molto grandi

Cosa sa il CHIAMANTE? Numero degli argomenti, tipo di argomenti, tipo di ritorno, nome della funzione

↓

Deve conoscere queste informazioni prima della chiamata, quindi la funzione deve essere prima dichiarata → È sufficiente per tradurla

↳ Già inclusa di default in C++

COSA SA IL CHIAMATO? Per il chiamato è utile solo sapere types, typen: vengono passati dal chiamante al chiamato

↓

È responsabilità del chiamante passare espressioni del tipo corretto

Tradizionalmente vengono passati in pila, mentre nell'architettura a 64 bit vengono utilizzati i registri → vanno utilizzati **in ordine**: %RDI, %RSI, %RDX, %RCX, %R8, %R9

↓

Ogni argomento ne prende **un numero intero** quindi non si possono mandare più argomenti nello stesso registro → per i tipi base e puntatori cerchiamo prendere un singolo registro e per gli array abbiamo che questi decadono a puntatori e quindi viene passato il puntatore all'indirizzo del **primo elemento**.

↓

f → g → h: in questo caso può accadere che il contenuto dei registri vada salvato altrove

↓ se non ottimizziamo

Si ha **sempre salvataggio in memoria**

C. **VALORI di RITORNO**

Utilizziamo **registri**: %RAX, %RDX

per raddoppiare la dimensione di %RAX

CONVENZIONI sui REGISTRI

Alcuni registri sono chiamati **scratch**: il chiamante non scrive niente perché è compito del chiamato

%RDI, %RSI, %RDX, %RCX, %R8, %R9 + %RAX, %R10, %R11

Altri sono **preservati**: il chiamato deve preoccuparsi di salvarli prima di scriverci

↓

%RBP, %RBX

Gestione chiamante

chiamato ← inizio funzione

PROLOGO

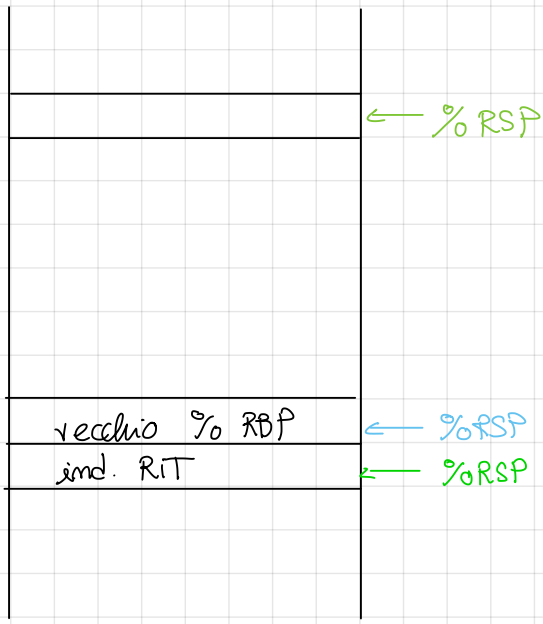
```
PUSH %RBP (precedente)   ——→ Prepara base pointer e alloca spazio per
MOV %RSP, %RBP                variabili locali
SUB $X, %RSP             ——→ lo sposta in modo da poter contenere le variabili locali
SALVATAGGIO registri preservati
COPIA parametri
```

EPILOGO
- RIPRISTINO registri - salvati
- LEAVE

equivale a ↓ equivale a

```
MOV  %RBP, %RSP
POP  %RBP
```

- RET



ESEMPIO

studenti@ce: ~/as

File Edit Tabs Help

```
extern "C" int f(int, int);
int main()
{
        int b = 2;
        int a = f(b+3, b);
        return a;
}
```

serve che la funzione sia dichiarata

↓ corpo di f

```
int f(int a1, int a2)
{
        return a1+a2;
}
```

```
#   +7   +6   +5   +4   +3   +2   +1   +0
# +----+----+----+----+----+----+----+----+
# |             vecchio rbx               | <- rsp
# +----+----+----+----+----+----+----+----+
# |                                       |
# +----+----+----+----+----+----+----+----+
# |       a2          |        a1         | -8
# +----+----+----+----+----+----+----+----+
# |           vecchio rbp                 | <- %rbp
# +----+----+----+----+----+----+----+----+
# |           ind. di ritorno             |
# +----+----+----+----+----+----+----+----+
```

PARAMETRI

CHIAMATA

```
.global f
f:
        push %rbp
        mov %rsp, %rbp
        sub $16, %rsp

        # a1 in %rdi
        # a2 in %rsi

        mov %edi, -8(%rbp)
        mov %esi, -4(%rbp)

        # fine prologo

        mov -8(%rbp), %eax
        add -4(%rbp), %eax
        # epilogo
        leave
        ret
```

.global f → Altrimenti non viene associata ad f nel main

push %rbp
mov %rsp, %rbp
sub $16, %rsp → Vogliamo trovare posto per 2 interi

→ CONVENZIONE SYSTEMS: stack allineato a 16

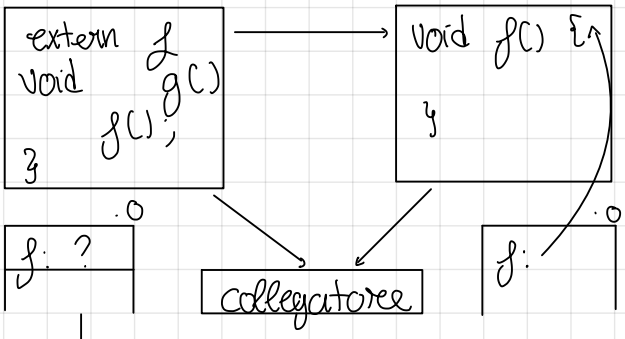
} versione non ottimizzata: tutto in memoria

`extern "C" void f();`

↳ mi dice che la funzione rispetta lo standard del C per la chiamata

↓

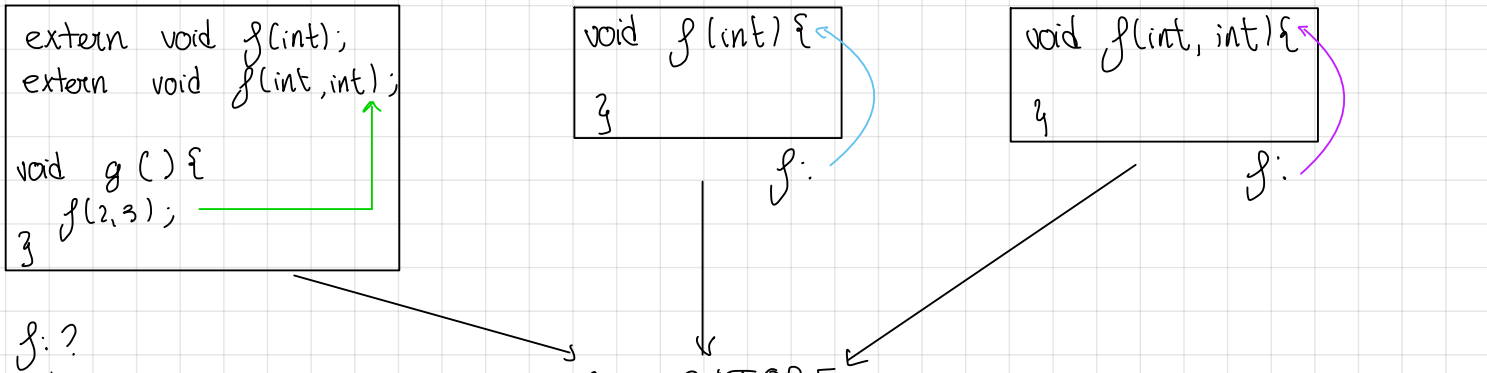
Se mettessi C++ succederebbe davvero: non è sufficiente mettere il nome della funzione come etichetta per permettere overloading → f(int), f(int, int), f(int*)



Nel file .o c'è una **tabella dei simboli**

↓

Per ogni file ove sono mancate come indefinite ovvero cerca un file dove queste sono definite → confronto tra stringhe



↓

Le funzioni vengono tradotte codificando al loro interno numero e tipo di parametri: La prima si chiamerà fi, la seconda fii

↓

REGOLE: fun (type1, ... typen) → _Z + lunghezza stringa + fun + tipi

_MAIUSC o __ sono nomi riservati al sistema

tabella dei tipi: int → i
char → c
long → l
type * → Ptipo : void f(int *) → _Z1f Pi
void f(int **) → _Z1f PPi

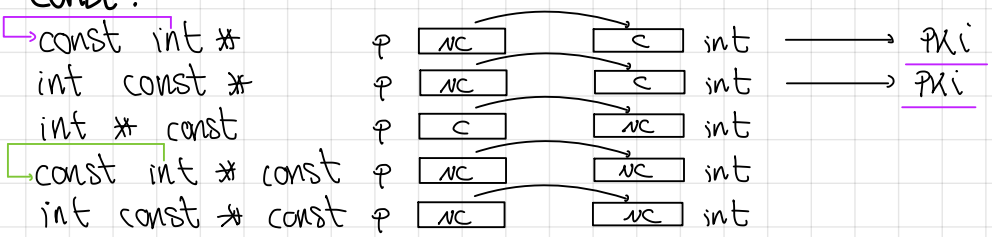
```
struct miastruct {
   :
}
```

si riferisce all'etichetta di struct

void f(miastruct) → _Z1f9miastruct
void f(miastruct *) → _Z1fP9miastruct
void f(int&, const char&) → _Z1fRiRKc (K = const, R = riferito)

array:
void f(int a[100]), void f(int a[100]), void f(int a[]), void f(int *a) → Sono **tutte uguali**: _Z1f Pi

const:



void f(int const) ⟶ Sono uguali: il passaggio è per valore e quindi la funzione non può modificare i parametri (traduzione) _Z1fi : la stringa non può terminare con K
void f(int)

LE CLASSI

class miaclasse { ⟶ Differenza tra classe e struttura: visibilità di default
:

public:
miaclasse();
miaclasse(int);
~miaclasse();
fun1();
fun2(int);
fun1(miaclasse *);
:
};

⟶ void f(){
miaclasse c; ⟶ La forma dipende solo dalle variabili membro e non dalle funzioni membro
c.fun1();
}
↓
chiamata a funzione membro che equivale a scrivere miaclasse_fun1(&c,...)
↓
Nella traduzione abbiamo una normale traduzione come le altre funzioni passando &c con %RDI

↳ Non viene tradotto in quanto se non vengono rispettate le regole non viene creato nessun file eseguibile

Aggiungiamo all'overloading ↳ il fatto che la funzione sta in una classe

class cc {
void f(int); _ZN 2cc 1f E i
void f(int, int); _ZN 2cc 1f E ii
void f(cc); _ZN 2cc 1f E S_
void f(cc &); _ZN 2cc 1f E R S_
};

Nested End
Spazi di nomi per accedere a funzione
La stringa è già presente prima (2cc)

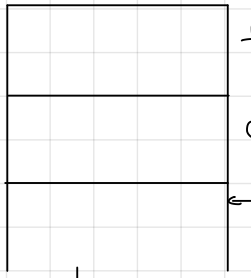
+

costruttori ⟶ C1 caso precedente ⟶ _ZN 2cc C1 f E V void: senza argomenti
distruttori ⟶ D1 caso precedente ⟶ _ZN 2cc D1 f E V
costruttore di copia ⟶ cc(const cc &) : _ZN 2cc C1 E R K S_

Quando il tipo di ritorno è una struttura/classe: architetturalmente grande ⟶
struct ss
class cc
ss f();
cc f();
↓
void f(ss)

Se entra in 16 byte allora la passiamo in %RAX, %RDX a patto che non sia definito un costruttore di copia (anche se passata per valore). Quando è più grande di 16 byte avremo

Es:
```
CC f()
void g() {
  CC mioC;
  mioC = f();
}
```
→ Oggetti temporanei che devono esistere ai fini della traduzione



↓ Il chiamante deve trovare lo spazio perché servono a lui

← Alloca lo spazio e passa l'indirizzo ove esso alloca lo spazio

↓ Avrò quindi
lea tmp1 (%RBP), %RDI
call - Z 1 f V
azioni di f
return

↳ Il parametro nascosto è sempre il primo oggetto, quindi sta in %RDI

PREPROCESSORE

```
#include "lib.h"

long var1 = 8;
long var2 = 4;

#ifdef DEBUG
void debug(const char *msg) {}
#else
#define debug(msg)
#endif

int main()
{
        // un commento
        var1 =    foo(var2);
        debug("questo_e'_un_messaggio_di_debug");
        return var1;
}
```
main.CPP

```
long foo(long);
void bar();
```
lib.h

```
.data
foovar1:
        .quad 5
foovar2:
        .quad 6
.bss
foovar3:
        .quad 0
.text
.global _Z3fool
_Z3fool:
        pushq %rbp
        movq %rsp, %rbp
        movq %rdi, %rax
        movabsq foovar3, %rax
        addq foovar1, %rax
        addq foovar2(%rip), %rax
        leave
        ret
```
foo.S

↓

Il preprocessore fa delle elaborazioni testuali ed i comandi che ci riferiscono a lui iniziano per #. Questi si chiamano **direttive per il preprocessore**. Il compilatore non le vedrà mai perché verranno trasformate in altro dal preprocessore.

↓

#include: Trova il file: " " → prima nella directory di preprocessazione e poi in posti standard
< > → soltanto in posti standard

↓

Una volta che ha trovato il file sostituisce la linea con il codice del file trovato

#define: utilizzato per definire macro

compilazione condizionale: #ifdef / ifndef

```
#ifdef DEBUG
void debug(const char *msg) {}
#else
#define debug(msg)
#endif
```
→ Se la macro DEBUG è definita
→ la definisce se non è già definita → Sparisce la chiamata

output del preprocessore:

g++ -E main.cpp

fermo il preprocessore

```
# 1 "main.cpp"
# 1 "<built-in>"
# 1 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 1 "<command-line>" 2
# 1 "main.cpp"
# 1 "lib.h" 1
long foo(long);
void bar();
# 2 "main.cpp" 2
long var1 = 8;
long var2 = 4;
int main()
{
 var1 = foo(var2);
 ;
 return var1;
}
```

permettono di capire da che file provengono gli estratti

↳ contenuto del file lib.h

↳ la macro debug non era definita

↳ toglie i commenti ed uniforma gli spazi

File che arriva al compilatore

g++ -D DEBUG -E main.cpp

definisco la macro debug

```
# 1 "main.cpp"
# 1 "<built-in>"
# 1 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 1 "<command-line>" 2
# 1 "main.cpp"
# 1 "lib.h" 1
long foo(long);
void bar();
# 2 "main.cpp" 2
long var1 = 8;
long var2 = 4;
void debug(const char *msg) {}
int main()
{
 var1 = foo(var2);
 debug("questo_e'_un_messaggio_di_debug");
 return var1;
}
```

la macro debug era stata definita

La compilazione condizionale viene utilizzata nel caso di compilazione su sistemi operativi diversi esempio, linux: non può esistere un'altra variabile di nome linux → per vederle: -dM

COMPILATORE

Dopo aver preprocessato il file questo va in pasto al compilatore. Questo produce un file assembler che va assemblato

```
.data
.global var1, var2
var1:
        .quad   8
var2:
        .quad   4
.text
.global _start
_start:
        pushq   %rbp
        movq    %rsp, %rbp
        movq    var2(%rip), %rax
        movq    %rax, %rdi
        call    _Z3fool
        movq    %rax, var1(%rip)
        movq    var1(%rip), %rax
        popq    %rbp
        ret
```

↳ il programma non può girare

file main.s

```
.data
foovar1:
        .quad 5
foovar2:
        .quad 6
.bss
foovar3:
        .quad 0
.text
.global _Z3fool
_Z3fool:
        pushq %rbp
        movq %rsp, %rbp
        movabsq foovar3, %rax
        addq foovar1, %rax
        addq foovar2(%rip), %rax
        leave
        ret
```

file foo.s

ASSEMBLATORE

Produce i byte che vanno in memoria e può suddividere più sezioni tra cui DATA, TEXT ed anche altre. L'assemblatore legge il file due volte completamente: 1. Scopre solo dove e quali sono le etichette
2. Assembla effettivamente

È possibile in questo modo fare un salto in avanti. Per ogni sezione tiene un contatore che conta quanti byte occupa questa sezione. Nel primo passaggio scrive da una parte, per ogni etichetta, [etichetta, contatore, sezione]: tabella dei simboli. Essa viene usata nel secondo passaggio. Se deve essere utilizzata un'etichetta non vista nel primo passaggio viene aggiunta un'entrata alla tabella dei simboli per dire che l'etichetta è indefinita e vengono aggiunte le **istruzioni di rilocazione** al file di output, utili per il collegatore che sostituisce l'etichetta che non conosceva con la funzione effettiva

formato ELF (file oggetto, eseguibili, librerie)

hexdump -C

```
00000000  7f 45 4c 46 02 01 01 00  00 00 00 00 00 00 00 00  |.ELF............|
00000010  01 00 3e 00 01 00 00 00  00 00 00 00 00 00 00 00  |..>.............|
00000020  00 00 00 00 00 00 00 00  e0 01 00 00 00 00 00 00  |................|
00000030  00 00 00 00 40 00 00 00  00 00 40 00 08 00 07 00  |....@.....@.....|
00000040  55 48 89 e5 48 89 f8 48  a1 00 00 00 00 00 00 00  |UH..H..H........|
00000050  00 48 03 04 25 00 00 00  00 48 03 05 00 00 00 00  |.H..%....H......|
00000060  c9 c3 05 00 00 00 00 00  00 00 06 00 00 00 00 00  |................|
00000070  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
00000090  00 00 00 00 03 00 01 00  00 00 00 00 00 00 00 00  |................|
000000a0  00 00 00 00 00 00 00 00  00 00 00 00 03 00 03 00  |................|
000000b0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
000000c0  00 00 00 00 03 00 04 00  00 00 00 00 00 00 00 00  |................|
000000d0  00 00 00 00 00 00 00 00  01 00 00 00 00 00 03 00  |................|
000000e0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
000000f0  09 00 00 00 00 00 03 00  08 00 00 00 00 00 00 00  |................|
00000100  00 00 00 00 00 00 00 00  11 00 00 00 00 00 04 00  |................|
00000110  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
00000120  19 00 00 00 10 00 01 00  00 00 00 00 00 00 00 00  |................|
00000130  00 00 00 00 00 00 00 00  00 66 6f 6f 76 61 72 31  |.........foovar1|
00000140  00 66 6f 6f 76 61 72 32  00 66 6f 6f 76 61 72 33  |.foovar2.foovar3|
00000150  00 5f 5a 33 66 6f 6f 6c  00 00 00 00 00 00 00 00  |._Z3fool........|
00000160  09 00 00 00 00 00 00 00  01 00 00 00 03 00 00 00  |................|
00000170  00 00 00 00 00 00 00 00  15 00 00 00 00 00 00 00  |................|
00000180  0b 00 00 00 02 00 00 00  00 00 00 00 00 00 00 00  |................|
00000190  1c 00 00 00 00 00 00 00  02 00 00 00 02 00 00 00  |................|
000001a0  04 00 00 00 00 00 00 00  00 2e 73 79 6d 74 61 62  |..........symtab|
000001b0  00 2e 73 74 72 74 61 62  00 2e 73 68 73 74 72 74  |..strtab..shstrt|
000001c0  61 62 00 2e 72 65 6c 61  2e 74 65 78 74 00 2e 64  |ab..rela.text..d|
000001d0  61 74 61 00 2e 62 73 73  00 00 00 00 00 00 00 00  |ata..bss........|
000001e0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
00000220  20 00 00 00 01 00 00 00  06 00 00 00 00 00 00 00  | ...............|
00000230  00 00 00 00 00 00 00 00  40 00 00 00 00 00 00 00  |........@.......|
00000240  22 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |"...............|
00000250  01 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
00000260  1b 00 00 00 04 00 00 00  40 00 00 00 00 00 00 00  |........@.......|
00000270  00 00 00 00 00 00 00 00  60 01 00 00 00 00 00 00  |........`.......|
00000280  48 00 00 00 00 00 00 00  05 00 00 00 01 00 00 00  |H...............|
00000290  08 00 00 00 00 00 00 00  18 00 00 00 00 00 00 00  |................|
000002a0  26 00 00 00 01 00 00 00  03 00 00 00 00 00 00 00  |&...............|
000002b0  00 00 00 00 00 00 00 00  62 00 00 00 00 00 00 00  |........b.......|
000002c0  10 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
000002d0  01 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
000002e0  2c 00 00 00 08 00 00 00  03 00 00 00 00 00 00 00  |,...............|
000002f0  00 00 00 00 00 00 00 00  72 00 00 00 00 00 00 00  |........r.......|
00000300  08 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
00000310  01 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
00000320  01 00 00 00 02 00 00 00  00 00 00 00 00 00 00 00  |................|
00000330  00 00 00 00 00 00 00 00  78 00 00 00 00 00 00 00  |........x.......|
00000340  c0 00 00 00 00 00 00 00  06 00 00 00 07 00 00 00  |................|
00000350  08 00 00 00 00 00 00 00  18 00 00 00 00 00 00 00  |................|
00000360  09 00 00 00 03 00 00 00  00 00 00 00 00 00 00 00  |................|
00000370  00 00 00 00 00 00 00 00  38 01 00 00 00 00 00 00  |........8.......|
00000380  21 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |!...............|
00000390  01 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
000003a0  11 00 00 00 03 00 00 00  00 00 00 00 00 00 00 00  |................|
000003b0  00 00 00 00 00 00 00 00  a8 01 00 00 00 00 00 00  |................|
000003c0  31 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |1...............|
000003d0  01 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
000003e0
```

readelf -h: decodifica l'inizio del file che contiene info sul contenuto del resto del file

```
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF64
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              REL (Relocatable file)
  Machine:                           Advanced Micro Devices X86-64
  Version:                           0x1
  Entry point address:               0x0
  Start of program headers:          0 (bytes into file)
  Start of section headers:          480 (bytes into file)
  Flags:                             0x0
  Size of this header:               64 (bytes)
  Size of program headers:           0 (bytes)
  Number of program headers:         0
  Size of section headers:           64 (bytes)
  Number of section headers:         8
  Section header string table index: 7
```

va rilocato: mancano dei pezzi

Negli eseguibili: per caricare programma in memoria ← tabella delle sezioni

```
There are 8 section headers, starting at offset 0x1f0:
```

→ readelf -WS

```
Section Headers:
  [Nr] Name              Type            Address          Off    Size   ES Flg Lk Inf Al
  [ 0]                   NULL            0000000000000000 000000 000000 00      0   0  0
  [ 1] .text             PROGBITS        0000000000000000 000040 000023 00  AX  0   0  1
  [ 2] .rela.text        RELA            0000000000000000 000158 000060 18   I  5   1  8
  [ 3] .data             PROGBITS        0000000000000000 000063 000010 00  WA  0   0  1
  [ 4] .bss              NOBITS          0000000000000000 000073 000000 00  WA  0   0  1
  [ 5] .symtab           SYMTAB          0000000000000000 000078 0000c0 18      6   4  8
  [ 6] .strtab           STRTAB          0000000000000000 000138 00001a 00      0   0  1
  [ 7] .shstrtab         STRTAB          0000000000000000 0001b8 000031 00      0   0  1
Key to Flags:
  W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
  L (link order), O (extra OS processing required), G (group), T (TLS),
  C (compressed), x (unknown), o (OS specific), E (exclude),
  l (large), p (processor specific)
```

istruzioni di rilocazione .TEXT — definita dal programmatore — dati globali non inizializzati = 0 — tabella simboli

readelf -s: (mostra tabella simboli)

```
Symbol table '.symtab' contains 8 entries:
   Num:    Value          Size Type    Bind   Vis      Ndx Name
     0: 0000000000000000     0 NOTYPE  LOCAL  DEFAULT  UND
     1: 0000000000000000     0 SECTION LOCAL  DEFAULT    1
     2: 0000000000000000     0 SECTION LOCAL  DEFAULT    3
     3: 0000000000000000     0 SECTION LOCAL  DEFAULT    4
     4: 0000000000000000     0 NOTYPE  GLOBAL DEFAULT    3 var1
     5: 0000000000000008     0 NOTYPE  GLOBAL DEFAULT    3 var2
     6: 0000000000000000     0 NOTYPE  GLOBAL DEFAULT    1 _start
     7: 0000000000000000     0 NOTYPE  GLOBAL DEFAULT  UND _Z3fool
```

non definita — primo settore DATA

readelf -r: (mostra tabella rilocazioni)

```
Relocation section '.rela.text' at offset 0x158 contains 4 entries:
  Offset          Info           Type           Sym. Value    Sym. Name + Addend
000000000007  000500000002 R_X86_64_PC32     0000000000000008 var2 - 4
00000000000f  000700000004 R_X86_64_PLT32    0000000000000000 _Z3fool - 4
000000000016  000400000002 R_X86_64_PC32     0000000000000000 var1 - 4
00000000001d  000400000002 R_X86_64_PC32     0000000000000000 var1 - 4
```

off. 7 — Sez. TEXT

non conosco di preciso offset ma lo ricavo e lo scrivo qua

Dico dove inizia istruzione successiva (var2 - 4 - 7)

COLLEGATORE

Quando chiamiamo il collegatore gli passiamo la lista dei file da collegare e quindi vede il programma per intero → Comando ld lista file. In ordine di file, unisce le sezioni TEXT e DATA e così via. Fatto questo crea un'unica tabella dei simboli aggiustando gli offset e mentre lo crea tiene nota dei simboli non definiti e restituisce un errore se un simbolo è definito più volte in file diversi. A questo punto assegna gli indirizzi alle sezioni e ricava tutti gli offset. Deve adesso gestire i simboli non definiti. Se non vengono aggiustati tutti da errore. Infine applica le rilocazioni. A questo punto il programma può essere eseguito: File ELF con tabella di programma →

```
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF64
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              EXEC (Executable file)
  Machine:                           Advanced Micro Devices X86-64
  Version:                           0x1
  Entry point address:               0x4000b0
  Start of program headers:          64 (bytes into file)
  Start of section headers:          720 (bytes into file)
  Flags:                             0x0
  Size of this header:               64 (bytes)
  Size of program headers:           56 (bytes)
  Number of program headers:         2
  Size of section headers:           64 (bytes)
  Number of section headers:         8
  Section header string table index: 7
```

readelf -wl :
(tabelle di programma)

```
Elf file type is EXEC (Executable file)
Entry point 0x4000b0
There are 2 program headers, starting at offset 64

Program Headers:
  Type           Offset   VirtAddr           PhysAddr           FileSiz  MemSiz   Flg Align
  LOAD           0x000000 0x0000000000400000 0x0000000000400000 0x0000f5 0x0000f5 R E 0x1000
  LOAD           0x0000f5 0x00000000004010f5 0x00000000004010f5 0x000020 0x000028 RW  0x1000

 Section to Segment mapping:
  Segment Sections...
   00     .text
   01     .data .bss
```

PERMESSI
Read
Write
Execute

LIBRERIE STATICHE

Collezioni di file oggetto ⟶ comando **ar** (archive) **cr** nomelib.a lista_file_oggetto

↓

Vantaggio: Quando il collegatore vede una libreria prende da essa soltanto i file oggetto che contengono simboli non definiti

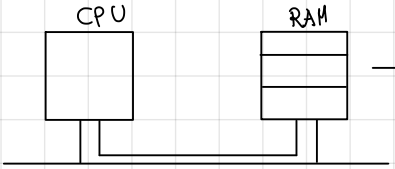
⚠ ATT: Lo fa nel punto in cui è scritta la libreria ⟶ tipicamente **in fondo**

ricerca libreria: Vanno messe in posti standard (/usr/local/lib) ed i nomi devono iniziare per lib ⟶ Posso specificare solo la parte intermedia del nome preceduta da -l

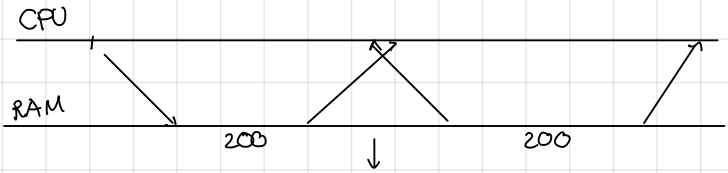
↓

include non è né necessario né sufficiente perché copia tutto il file. È infatti sufficiente la dichiarazione dei simboli (ad esempio cout). Non è neanche sufficiente la include perché la dichiarazione viene fatta nella libreria e per vederla deve collegarla ⟶ Non è compito del preprocessore ma del collegatore che deve collegare la libreria standard di C++. Non ce ne accorgiamo perché nel caso di g++ la libreria standard la collega sempre. Questo non accade per librerie "strane".

La cache



→ impiega 200 - 300 cicli di clock per rispondere. La CPU è invece molto più veloce e lo è diventata nel tempo.



↓

Il tempo di esecuzione è dettato dalla velocità della RAM in questo caso

tipologie di RAM

Viene utilizzata la tecnica della memoria dinamica che prevede di memorizzare i bit in piccoli condensatori che in realtà sono capacità presenti dei transistor. Questo permette di condensare tanti bit in poco spazio anche se la velocità di lettura e scrittura è piuttosto bassa. La coerenza inoltre dura poco e deve essere rinnovata. Abbiamo poi la tecnologica della RAM statica che prevede l'utilizzo di latch e quindi molti transistor. Queste memorie sono molto più veloci rispetto alle memorie dinamiche ma costano molto di più e consumano anche molta più energia elettrica.

↓ Creiamo l'illusione di avere una RAM grande e veloce

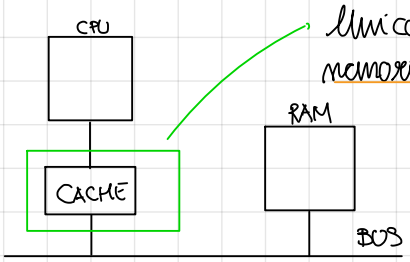
Principio di località temporale: se un programma ha fatto un'operazione di lettura o scrittura ad un certo indirizzo è molto probabile che rifarà un'operazione di lettura o scrittura a quello stesso indirizzo

principio di località spaziale: se un programma ha fatto un'operazione di lettura o scrittura ad un certo indirizzo è molto probabile che in breve tempo farà operazioni di lettura e scrittura ad indirizzi vicini

↓

Questo accade ad esempio nell'incremento di una variabile ed anche perché il codice viene memorizzato e facendo un ciclo preleva sempre le stesse istruzioni. La località spaziale deriva dalla sequenzialità del codice

↓ Modello



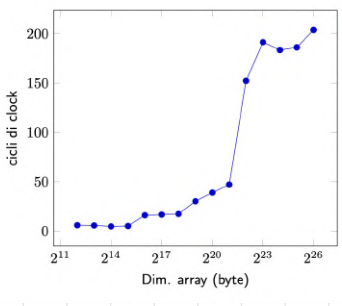
Unica a fare accesi alla RAM: suddivisa in controllore cache e memoria cache realizzata in modo da essere piccola e veloce.

↓

Il controllore cache osserva tutti gli indirizzi e cerca di copiare dalla RAM le informazioni che servono localmente o che potranno servire in futuro → Leggibili più velocemente

Soluzione hardware: fa tutto il controllore cache perché il software funziona indipendentemente dalla presenza della cache. Il controllore è quindi **trasparente al software**.

↓ Efficacia



→ Il tempo aumenta in quanto l'array non entra più nella cache e quindi è necessario un accesso in memoria. Nel caso migliore il tempo di accesso è 4 (cache di primo livello) → Conviene fare le ottimizzazioni con la cache

realizzazione

Utilizzo il principio di località temporale copiando i dati fino al riempimento. Si pone quindi il problema del rimpiazzamento tramite il quale il controllore deve tenere i dati più utili → deve essere rimpiazzata l'informazione che verrà richiesta più in avanti nel futuro → Approssimando che il futuro sarà simile al passato, viene rimpiazzata l'informazione che non è stata richiesta da più tempo: **Least recently used**.

↓

Non ha senso parlare di cache per le operazioni di ingresso ed uscita in quanto per la cache abbiamo posto che la RAM non cambia il proprio contenuto da sola. Le operazioni di lettura e scrittura in I/O hanno indirettamente effetti collaterali che verrebbero azionati in modo errato da parte della cache. Questa deve quindi essere esclusa in caso di operazioni in I/O. Questo è però complicato in quanto anche i registri dell'I/O prendono parte allo spazio di indirizzamento della RAM

scrittura in memoria

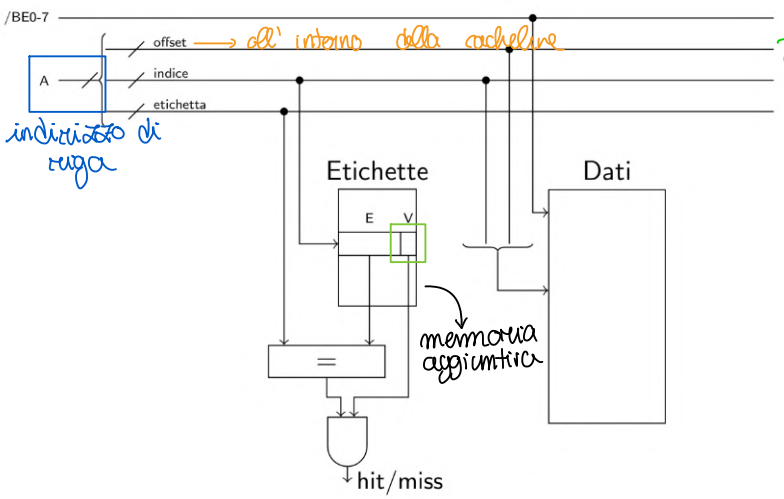
Il controllore copia l'indirizzo a cui deve essere fatta la scrittura e poi la scrittura può essere fatta soltanto nella copia oppure sia nella copia che nella memoria. Si ha **miss** quando l'indirizzo richiesto dal processore non è presente in cache altrimenti si ha **hit**. Nel caso di miss la cache può adottare una politica di **write allocate** o **write no allocate**. Nel secondo caso si procede direttamente alla scrittura in RAM mentre nel primo caso, prima di fare la scrittura, viene copiato il contenuto dell'indirizzo a cui deve essere fatta la scrittura in cache. In questo caso la scrittura può essere fatta solo in cache (politica di **write back**) oppure scrivere in entrambi (politica di **write through**). Il vantaggio di write back è la maggiore velocità di scrittura in cache anche se prima o poi ci dovrà essere anche la scrittura in RAM per non perdere le modifiche. Non c'è quindi un guadagno vantaggioso sotto questo aspetto perché la scrittura in RAM deve comunque esserci. È conveniente quando un indirizzo viene scritto più volte perché in tale caso ci sarà un solo rimpiazzamento. Il secondo vantaggio si ha perché nel caso di un rimpiazzamento la CPU può fare altre operazioni.

↓ Ma come facciamo ad evitare scritture inutili?

Quando c'è una scrittura si associa un flag. Quando non c'è questo flag non c'è stata modifica e quindi non è necessaria una modifica in memoria.

A basso livello

Il controllore deve mappare velocemente un indirizzo di memoria in cache e vedere se a quell'indirizzo c'è quello che il processore sta cercando



Organizzata come la memoria RAM

↓

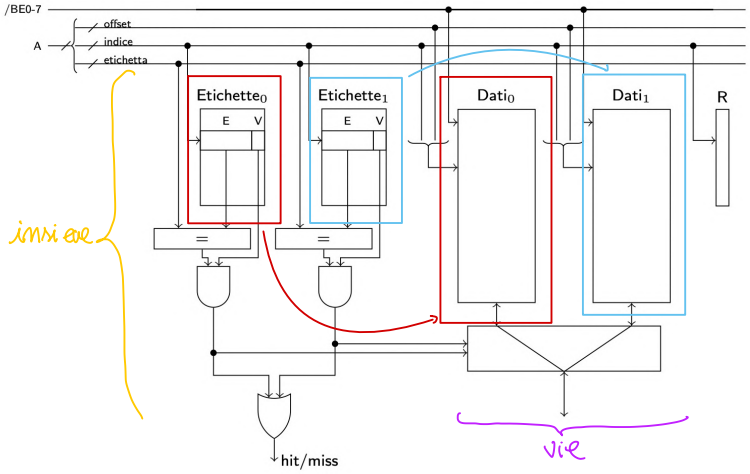
L'indirizzo di cache viene mappato prendendo i bit meno significativi dell'indirizzo del processore tanti quanti ne servono per indirizzare la memoria cache

↓

indirizzamento diretto: l'indirizzo del processore fornisce direttamente l'indirizzo di cache

Ma come faccio a vedere se il contenuto è giusto? con i bit più significativi. Questa parte si chiama **etichetta**. Abbiamo quindi bisogno di una seconda RAM detta RAM delle etichette con la quale confronto l'etichetta data dal processore. C'è poi bisogno di sapere se l'etichetta è valida. Se entrambe le condizioni sono verificate si ha una hit. Ma perché accediamo alla memoria delle etichette solo con l'indice? (indirizzo cache = offset + indice). La memoria delle etichette è overhead e quindi questa deve essere più piccola possibile. Invece di prendere delle righe della RAM vengono prese regioni più grandi riducendo la memoria delle etichette rischiando però di leggere cose inutili della RAM. Possiamo però utilizzare il secondo principio di località per dire che siamo a queste letture. L'unità di trasferimento non è la riga ma la **cacheline** (8 righe / 64 byte tipo), allineata naturalmente. Per quanto riguarda l'indirizzo, i 3 bit meno significativi sono l'offset all'interno della cacheline ed il resto sono il numero di cacheline. Per sapere se il dato c'è o non c'è il processore guarda solo l'indice. Una cacheline di solito è grande 64 byte. Quando il processore preleva una cacheline dalla memoria (miss) la copia nella posizione corrispondente all'indice, indipendentemente dal fatto che ci siano altre posizioni libere. Con la memorizzazione detta, tutte le cacheline con lo stesso indice devono essere nella stessa posizione della memoria perché è utilizzato solo l'indice per effettuare l'accesso. ⟶ ⚠ PROBLEMA: Le cacheline in **posizione** e **posizione + dim. cache** hanno lo stesso indice ↘

È possibile per corrispondere più dati allo stesso indice



⟶ Mettiamo in parallelo più memorie etichette e dati. L'indirizzo del processore viene composto allo stesso modo dell'indirizzamento diretto e la ricerca dell'etichetta viene fatta parallelamente in entrambe le memorie etichetta. Questa viene trovata se sta almeno in una delle due memorie etichetta ⟶ non può stare in entrambe perché il controllore non ha caricato più volte.

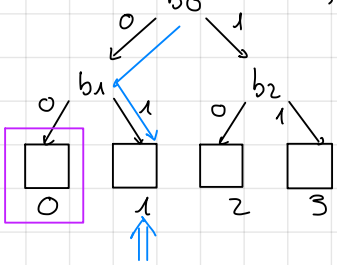
↓

Una volta trovata viene prelevata dalla memoria dati corretta

⚠ Permette la convivenza di cacheline con lo stesso indice

↓

Può essere estesa anche a più vie di cui ogni riga è un **insieme**. Ciascun indice adesso identifica un insieme. Il controllore adesso può anche scegliere cosa rimpiazzare. Con **2 vie** abbiamo un'ulteriore memoria che indica qual è quella in cui è stato fatto l'ultimo accesso. Con 4 vie utilizziamo un algoritmo che utilizza 3 bit (b0, b1, b2) chiamato **pseudo-LRU**



⟶ Pensiamo i hit come un albero binario che ci dice qual è il prossimo elemento da rimpiazzare. Se è 0 si va a sinistra mentre se è 1 si va a destra.
La struttura viene aggiornata cambiando i hit mentre la struttura viene percorsa. Supponiamo di fare un accesso a 1

	b0	b1	b2
PRIMA	0	1	-
DOPO	1	0	-

⟶ va dall'altra parte

L'aggiornamento si può fare con una scrittura di una costante fino a seconda della riga che stiamo rimpiazzando. Questo algoritmo sbaglia in quanto vicini ricevono un "bonus" andando in cima alla lista pur non avendo ricevuto alcuna scrittura

↓

può essere esteso a più vie aggiungendo più cronologie per non farlo diventare troppo costoso

comando lscpu --cache

```
mgiannini@DESKTOP-5HAOP4U:~$ lscpu --cache
NAME ONE-SIZE ALL-SIZE WAYS TYPE        LEVEL
L1d : dati    32K     128K     8 Data          1
L1i : istruzioni 32K  128K     8 Instruction   1
L2        256K      1M     8 Unified       2
L3          8M      8M    16 Unified       3
```

Periferiche

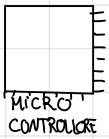
- LIMITAZIONI del SISTEMA OPERATIVO

Non possiamo scrivere sempre programmi che accedono a periferiche. I programmi sono infatti soggetti a limitazioni date dal sistema operativo e NON dal processore. Non vengono permessi cicli infiniti e non vengono permesse le istruzioni IN/OUT. Non è inoltre possibile accedere in memoria ad un indirizzo casuale non compreso nella tabella di caricamento del programma ⟶ In questo caso dobbiamo guardare i permessi di **R W E**. Vengono utilizzati i meccanismi di **interruzione**, **protezione** e **memoria virtuale** per limitare le funzionalità del programma

↓

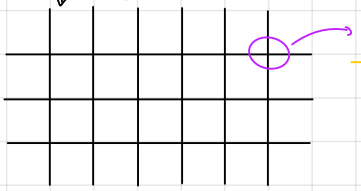
Utilizziamo una macchina virtuale che ci elimina tali meccanismi in modo da poter vedere il funzionamento delle periferiche ⟶ Macchina QEMU con un vantaggio: può caricare nella memoria del programma un eseguibile creato dalla macchina ospite. La libreria **libc** inoltre ci permette di semplificare la gestione inizializzando il processore e permettendo di ricevere input da tastiera e far vedere cose a video. Non è possibile utilizzare la libreria standard ed in particolare iostream in quanto richiede un kernel linux per fare determinate azioni. Lo script **compile** permette di utilizzare libc e dice dove collegare i file.

- TASTIERA



⟶ Piccolo chip che funziona come un computer: ha un processore, una RAM, una ROM e può pilotare dei piedini utilizzandoli come I/O

↓ Sotto i tasti



Collegamenti elettrici collegati ai piedini del microcontrollore collegati elettricamente alla pressione di un tasto. Il microcontrollore invia un impulso ad ogni colonna ciclicamente molto velocemente e poi legge tutte le sue righe dai piedini. Se nessun tasto è premuto l'impulso non torna indietro altrimenti capisce che è premuto il tasto all'intersezione tra riga e colonna

↓

Il microcontrollore associa un codice a ciascun tasto e poi invia l'informazione al microcontrollore sul PC che dialoga con il BUS attraverso il registro RBR che contiene il codice del tasto premuto (codice di **scansione**). Il microcontrollore si ricorda inoltre i tasti che sono premuti e quando un tasto viene rilasciato invia un altro codice che segnala il rilascio del tasto. Abbiamo quindi 2 codici per ciascun tasto che nel caso delle tastiere compatibili IBM si differenziano per il bit più significativo. Sapere se il tasto è stato rilasciato ci permette di continuare a scrivere lo stesso carattere più volte sul computer e di utilizzare le combinazioni di tasti. La comunicazione è inoltre bidirezionale e quindi è presente un TBR. Questo permette di utilizzare i LED e ad impostare alcuni parametri ⟶ È molto più complicato di quanto si pensi perché è implementata come nell'"antichità"

↓

Il microcontrollore nel computer non è più fisicamente presente ma è emulato da un altro pezzo di hardware che emula anche altre cose. I PC IBM inoltre dovevano avere pochi pin per essere economici ⟶ Le interfacce utilizzano l'indirizzo interno attraverso un unico piedino di indirizzo. Per discriminare tra i 4 registri si utilizza quindi il tipo di accesso (R/W). I registri sono visti accoppiati (RTSR, RTBR) agli indirizzi 60 e 64

ESEMPIO

Vogliamo leggere i codici di scansione e mostrarli a video

```
#include <libce.h>

const ioaddr iSTR = 0x64;
const ioaddr iRBR = 0x60;

natb get_code()
{
        natb c;
        do
                c = inputb(iSTR);
        while (!(c & 0x01));
        return inputb(iRBR);
}

int main()
{
        natb c;
        for (;;) {
                c = get_code();
                if (c == 0x01)
                        break;  // make code di ESC
                for (int i = 0; i < 8; i++) {
                        if (c & 0x80)   // bit piu' significativo di c
                                char_write('1');
                        else
                                char_write('0');
                        c <<= 1;
                }
                char_write('\n');
        }
}
```

→ Ci serve per vedere il flag FI (vedi appunti rati)

La code non racchiude compatibile: leggerebbe sempre il primo valore

chiamato da start in libce → quando ritorna riceve l'ASCII inviato

→ codice di scansione di esc

```
.text
        .global inputb
inputb:
        .cfi_startproc
        movw %di, %dx
        inb %dx, %al
        ret
        .cfi_endproc
```

indirizzo > 256

ESEMPIO: eco da tastiera

```
#include <libce.h>

namespace kbd {

const natl MAX_CODE = 29;
bool shift = false;
natb tab[MAX_CODE] = {  // tasti lettere (26), spazio, enter, esc
        0x10, 0x11, 0x12, 0x13, 0x14, 0x15, 0x16, 0x17, 0x18, 0x19,
        0x1E, 0x1F, 0x20, 0x21, 0x22, 0x23, 0x24, 0x25, 0x26,
        0x2C, 0x2D, 0x2E, 0x2F, 0x30, 0x31, 0x32, 0x39, 0x1C, 0x01
};
natb tabmin[MAX_CODE] = {
        'q', 'w', 'e', 'r', 't', 'y', 'u', 'i', 'o', 'p',
        'a', 's', 'd', 'f', 'g', 'h', 'j', 'k', 'l',
        'z', 'x', 'c', 'v', 'b', 'n', 'm', ' ', '\n', 0x1B
};
natb tabmai[MAX_CODE] = {
        'Q', 'W', 'E', 'R', 'T', 'Y', 'U', 'I', 'O', 'P',
        'A', 'S', 'D', 'F', 'G', 'H', 'J', 'K', 'L',
        'Z', 'X', 'C', 'V', 'B', 'N', 'M', ' ', '\r', 0x1B
};
```

→ Si ricorda se shift è premuto

→ Associati al codice di scansione quando shift non è premuto

→ Associamo quando shift è premuto

```
char char_read()
{
        natb c;
        char a;

        do {
                c = get_code();
                if (c == 0x2A) // left shift make code
                        shift = true;
                else if (c == 0xAA) // left shift break code
                        shift = false;
        } while (c >= 0x80 || c == 0x2A); // make code;
        a = conv(c);    // conv() puo' restituire 0
        return a;       // 0 se tasto non riconosciuto
}
```

→ codice premere shift

→ codice rilascio shift

break code di tutti i tasti

→ Restituisce il codice ASCII corrispondente al codice di scansione

```
char conv(natb c)
{
        natb cc;
        natl pos = 0;

        while (pos < MAX_CODE && tab[pos] != c)
                pos++;
        if (pos == MAX_CODE)
                return 0;
        if (shift)
                cc = tabmai[pos];
        else
                cc = tabmin[pos];
        return cc;
}
```

→ Cerco nella tabella

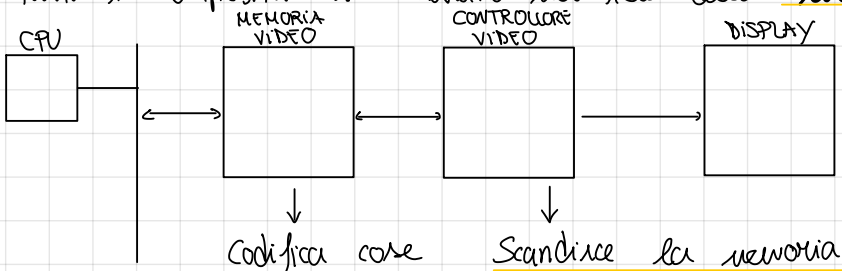
} Utilizzo tabella conv. corrispondente

```
int main()
{
        char c;

        for (;;) {
                c = char_read(); // puo' restituire 0
                if (c == 0x1B) // carattere ASCII esc
                        break;
                char_write(c);  // non effettua azioni se c vale 0
        }
        return 0;
}
```

↳ fa eco

- **VIDEO**

Tutti i dispositivi si basano sull'idea della **memoria video**



Codifica cose che vediamo a video → È presente all'interno dello **spazio di indirizzamento** sotto forma di indirizzi ove il SW può scrivere

Scandisce la memoria video ed interpreta il contenuto in modo da trasformarlo in segnali per il display.

a. **MODO TESTO**: la memoria video contiene i codici ASCII + altre informazioni

b. **MODO GRAFICO**: vengono codificati i colori dei singoli pixel nel display

Come distinguerli ↓

Nel modo testo abbiamo il numero di colonne e righe che vogliamo che vengano visualizzate. Nel modo grafico abbiamo risoluzione e profondità (numero di colori per pixel)

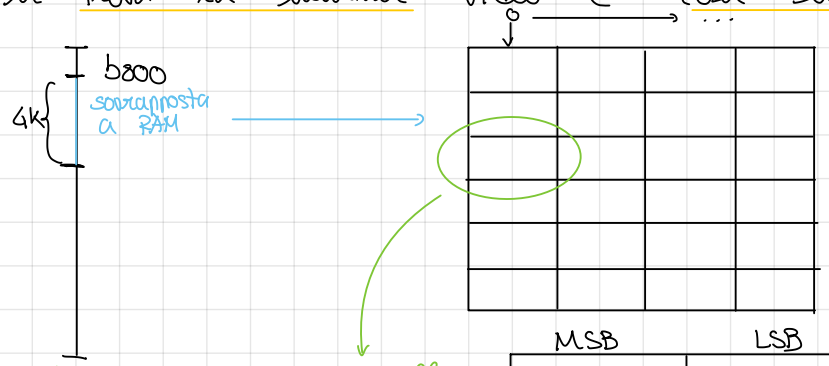
↳ Utilizziamo gli standard **VGA** e **SVGA**

↓

Il controllore video è collegato nello spazio di I/O con alcuni registri, alcuni per capire dove far apparire il cursore in modalità testo. Da fuori si vedono **solo due registri** mentre agli altri si accede indirettamente. Si scrive in uno dei due registri il numero del registro interno con il quale vogliamo parlare ed il secondo diventa una "finestra" sul registro selezionato.

dispositivi compatibili VGA

Partono in modalità testo. Per far vedere qualcosa a schermo basta sapere dove si trova la memoria video e cosa scrivere nelle celle.



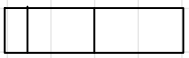
Organizzata ad array

→ Appare a video quando viene scritto

2 byte per ciascuna cella:

attributo che codice ASCII da visualizzare

16 per foreground, 3 per il background e permette di far lampeggiare il carattere



↓ 4 bit foreground : RGB + intensità

3 bit background : RGB → solo colori scuri

1 bit blinking

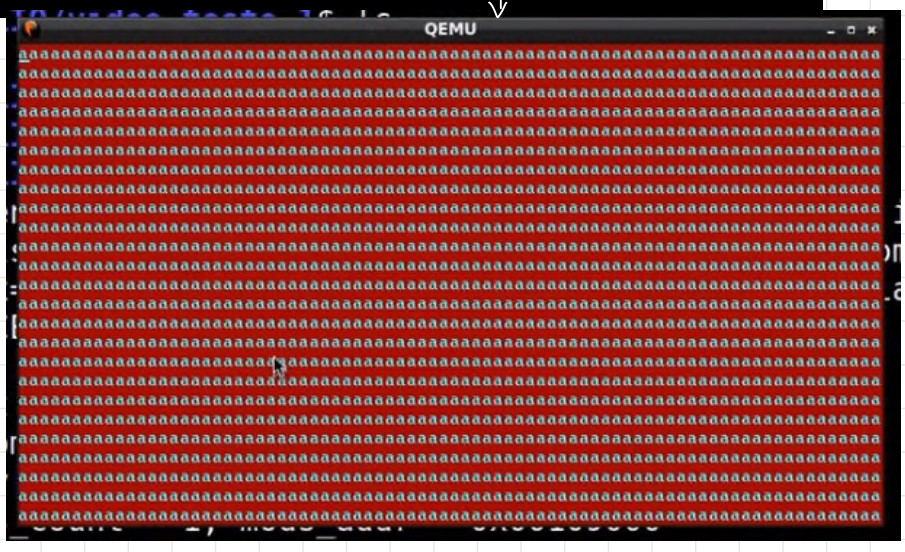
In modalità testo c'è una ROM che contiene i pixel che vanno accesi a seconda del carattere

ESEMPIO: riempimento memoria video

```
#include <libce.h>

namespace vid {
volatile natw* video = reinterpret_cast<natw*>(0xb8000);        // array di 2000
 word;
}

using namespace vid;

int main()
{
        for(int i = 0; i < 2000; i++)
                video[i] = 0x4B00 | 'a';        // sfondo rosso (0100), simbolo
azzurro chiaro (1011)
        for (;;) {
                char c = char_read();
                if (c == 0x1B)
                        break;                  // carattere ASCII esc
        }
        return 0;
}
```

array di unsigned short — inizio memoria video — tradotti con delle MOV — vedi codici RGB — sfondo & foreground codice ASCII di A — termina programma con ESC

OUTPUT



Ma come facciamo a far muovere il cursore?

```
void char_write(natb c)
{
        switch (c) {
        case 0:                          → Non fa nulla
                break;
        case '\n': case '\r':
                x = 0;
                y++;
                if (y >= ROWS)
                        scroll();
                break;
        default:                         → Posizione matrice [x][y]
                video[y * COLS + x] = attr | c;
                x++;                     ← Sposta cursore
                if (x >= COLS) {         → attributo colore, carattere
                        x = 0;           → ultima colonna azzerata x e poi y a capo
                        y++;
                }
                if (y >= ROWS)
                        scroll();        → riporta anche y indietro
                break;
        }
        cursore();
```

```
void scroll()
{                                        → Per tutte le righe tranne ultima
        for (int i = 0; i < VIDEO_SIZE - COLS; i++)
                video[i] = video[i + COLS];   → copia in pos. i quello che c'è in i+1
        for (int i = 0; i < COLS; i++)
                video[VIDEO_SIZE - COLS + i] = attr | ' ';
        y--;
} y in ultima riga Riempie ultima riga con spazi
```

pulitura schermo

```
void clear_screen(natb col)
{
        attr = static_cast<natw>(col) << 8;
        for (int i = 0; i < VIDEO_SIZE; i++)
                video[i] = attr | ' ';
        x = 0;
        y = 0;
        cursore();
}
```

posizione cursore

```
using namespace vid;

void cursore()
{                                        → 2 byte
        natw pos = COLS * y + x;
        outputb(CUR_HIGH, IND);          ← scritto in 2 registri
        outputb(pos >> 0x8, DAT);
        outputb(CUR_LOW, IND);           → finestra su CUR HIGH
        outputb(pos, DAT);
}
```

back space

```
case '\b':
        if (x > 0) {
                x--;
        } else if (y > 0) {
                y--;
                x = COLS - 1;
        }
        video[y * COLS + x] = attr | ' ';
        break;
```

cancello il vecchio carattere mettendoci spazio

modalità grafica (SVGA)

La memoria diventa più grande pur essendo sempre organizzata a matrice e ciascun elemento della matrice è il colore di un pixel

⚠ è complicato passare in modalità grafica → le schede video coesistono con una ROM per impostare le modalità grafiche

↓ Emulazione

```
#include <libce.h>

const natl COLS = 1280;
const natl ROWS = 1024;

volatile natb* framebuffer;

int main()
{
        framebuffer = bochsvga_config(COLS,ROWS);

        for (int i=0; i<COLS; i++)
        for (int j=0; j<ROWS; j++)
                framebuffer[j*COLS + i] = 0x04;          // rosso scuro (vga 0100
)
        char c;
        do
                c = char_read();
        while (c != 0x1B);                               // carattere ASCII esc
        return 0;
}
```

Passa a mod. grafica con quelle righe e colonne ed individua la memoria video

array in cui scriviamo il colore

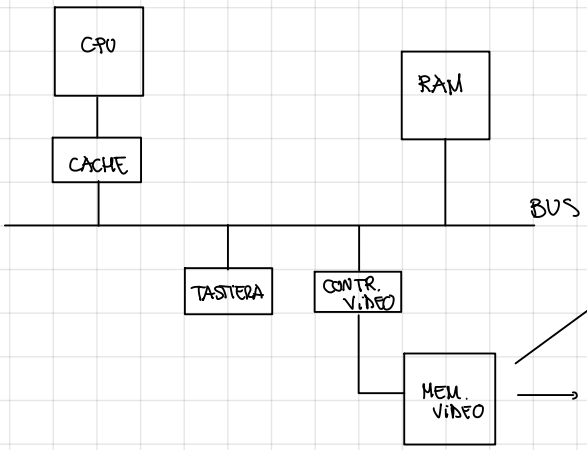
→ La memoria grafica è una **matrice di colori**

ESEMPIO

```
void rettangolo(natb c)
{
        for (int i = 0; i < COLS; i++)
                framebuffer[i] = c;                      // linea in alto
        for (int j = 0; j < ROWS; j++)
                framebuffer[COLS * j + (COLS - 1)] = c; // linea a destra
        for (int i = 0; i < COLS; i++)
                framebuffer[COLS * (ROWS - 1) + i] = c; // linea in basso
        for (int j = 0; j < ROWS; j++)
                framebuffer[COLS * j] = c;               // linea a sinistra
}
void assi(natb c)
{
        for (int i = 0; i < COLS; i++)
                framebuffer[COLS * y0 + i] = c;          // asse cartesiano x
        for (int j = 0; j < ROWS; j++)
                framebuffer[COLS * j + x0] = c;          // asse cartesiano y
}
```

⚠ATT: gli assi cartesiani partono dall'alto a sx e y va in basso

⚠ATT2: Il controllore cache dà fastidio solo se utilizza la politica write back in quanto potrebbe non aggiornare la memoria video



→ Nello spazio di memoria: gli accessi passano prima della cache

→ Può anche essere **letta**: in tal caso il controllore cache lha sempre perché essa non cambia da sola

- **TIMER**

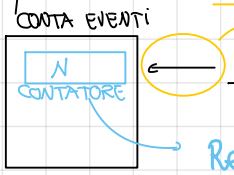
Serve per poter avere una misura del tempo che passa. Un tempo si poteva usare il numero di istruzioni perché esse venivano eseguite in un tempo prestabilito. Non è più vero nei processori moderni e passando da una generazione ed un'altra di processore

↓ potrei pensare, per far passare tempo

`for (int i=0; i<1000000; i++)` ma non posso farlo perché non ho un tempo di esecuzione fisso → il processore svolge le operazioni in parallelo ed il compilatore ottimizza (in questo caso toglie il ciclo perché non fa niente)

↓

Abbiamo bisogno di qualcosa che misura il tempo → interfacce di conteggio che contano gli eventi:

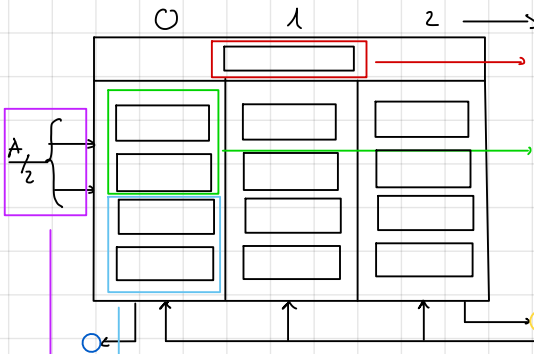


Piedino di ingresso: se periodico contare il tempo

Registro: viene decrementato a partire dal **trigger** che può essere hardware (piedino di ingresso) o software (scrittura registro)

Può generare impulsi in determinati eventi oppure onde quadre e può anche ripartire dal valore iniziale quando il registro arriva a 0 → segnale periodico

↓ Nei PC IBM



→ 3 contatori configurabili

Registro di **configurazione**: da ordine di memorizzazione o può scegliere cosa fare

Registri **contatore**: 16 bit totali (8 per registro) che contengono parte alta e parte bassa del contatore

Circuito: Suona comandato con onde quadre

clock 1 MHz

Controllore delle interruzioni

↳ Servono per **leggere il valore corrente** del contatore perché se il programmatore legge gli altri questi potrebbero variare nel mentre

indirizzamento: Un indirizzo è riservato al registro comune e ci da un indirizzo a ciascun contatore. Questi partono da 40. Per indirizzare i 4 registri di ciascun contatore si utilizza il trucco della tastiera avendo 2 registri nei quali è

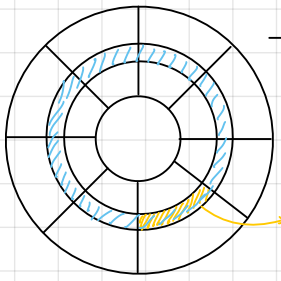
possibile scrivere e gli altri in cui è possibile soltanto leggere. Infine, per fare la scrittura nei 2 registri contatore, prima viene scritta la parte bassa e poi la parte alta → lo fa **automaticamente** il chip

```
#include <libce.h>

namespace tim {

const ioaddr iSPR = 0x61;
const ioaddr iCWR = 0x43;
const ioaddr iCTR2_LSB = 0x42;
const ioaddr iCTR2_MSB = 0x42;

};

using namespace tim;

void avvia_timer(natw N)
{
        outputb(0xB6, iCWR);            // contatore 2, modo 3
        outputb(N, iCTR2_LSB);
        outputb(N >> 8,  iCTR2_MSB);
}
int main()
{
        avvia_timer(1190);              // costante 1000 Hz
        outputb(3, iSPR);
        pause();
        outputb(0, iSPR);
        return 0;
}
```

iSPR → controlla ciclino
iCWR → Control word register
Registri contatore 2 → Problema con cache write back
Il valore serve per dire che vogliamo configurare il contatore 2
genera onda quadra
Da qui in poi si sente il suono
Registro collegato a ciclino che permette di silenziarlo, contano i due LSB. Uno è collegato al circuito del tuner e fornisce il contegno e l'altro è posto in porta AND con il ciclino → Per farlo funzionare si deve mettere 1

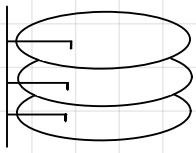
- **HARD DISK**

Ci occupiamo degli hard disk meccanici (con i dischi che girano) → Soluzione più economica per memorizzare grandi quantità di dati



→ Dischi che contengono materiale ferromagnetico organizzati in tracce e settori → parte di dati memorizzati su una traccia
sezione circolare del disco

SETTORE: 512 byte → Dati memorizzati cambiando la polarizzazione del materiale ferromagnetico
↓
Unità di trasferimento: può scrivere / trasferire un intero settore
↓
I programmi vengono eseguiti dalla RAM mentre l'hard disk serve solo per la memorizzazione
↓
È una periferica → non è possibile accedervi direttamente da un programma ma con un'interfaccia
↓
Vi sono 2/3 dischi memorizzati su entrambe le facce con delle testine montate sullo stesso braccio che possono ruotare come un giradischi →



↓
Le testine si muovono tutte insieme sulle facce dei dischi che sono tenuti in rotazione costante da 5000 a 15000 rpm.
Per leggere / scrivere un settore si deve sapere su quale faccia si trova il settore

e quindi con quale testina leggere, la traccia dove si trova per ruotare il braccio delle testine ed infine il settore → numerati negli header

↓

Il controllore fa partire l'operazione che comporta un **seek** + **latency** + **accesso**

- **seek**: di quanto deve ruotare il braccio delle testine per posizionarsi sulla traccia richiesta
  - ↓
  - Dipende da dove era il braccio prima: **millisecondi**
- **latency**: attesa che il settore passi sotto la testina
  - ↓
  - O se sta cercando della rotazione il settore, dipende e della delle rotazioni al minuto per giro completo: **millisecondi**
  - ↓
  - molto più lento del processore
- **accesso**: accesso al settore
  - ↓
  - **ms**: dipende della grandezza del settore
  - ↓
  - migliorato per aumento capacità

tempi **troppo lunghi**: dispositivi inizialmente pensati per accesso casuale poi divenuti sequenziali per la convenienza in termini di tempo

Ma come facciamo a memorizzare più dati in uno spazio sempre più ristretto?

trucco: la testina viene avvicinata di più in modo da creare zone più piccole

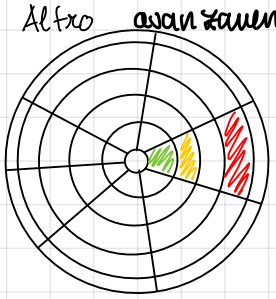
↓ come?

La testina si tiene **sollevata** dal disco e non tocca mai il disco

↓

Il tempo di accesso migliora: è l'unico tempo dichiarato perché è sufficiente memorizzare in sequenza per annullare gli altri due tempi

Altro **avanzamento tecnologico**



→ C'è **spreco di spazio**: i settori interni sono più piccoli di quelli esterni ma contengono sempre la stessa informazione

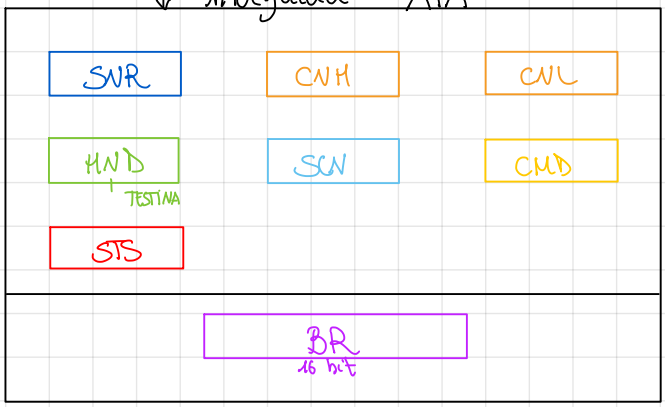
↓

Il disco viene suddiviso in **zone** in modo che i settori abbiano tutti la stessa dimensione aumentando il numero di settori verso l'esterno

↓ Impatta tempo di lettura e scrittura

La velocità per leggere/scrivere un settore all'esterno è maggiore di quella impiegata per settori all'interno ($v = \omega \cdot r$). → I SO riempiono gli hard disk partendo dall'esterno e quindi rallentano quando il disco si riempie

Dal punto di vista del **programmatore**
↳ i settori hanno solo un'indicizzione logica (numerati da 0 al massimo)
↓ Interfaccia ATA



SNR (sector number): indica il numero del settore sulla traccia
CNL/H (cylinder number low / high): Cilindro = insieme delle tracce una sull'altra
SCN (sector count number): Permette di trasferire più di un settore
CMD (comando): Specifica l'azione da fare
STS (stato)
BR: permette lettura / scrittura dei dati

Per fare una **lettura**: Scrivo le coordinate del settore ed il comando di lettura. A questo punto il controllore fa tutte le azioni necessarie e mette il contenuto del settore in una memoria interna e fa sapere tramite il bit data ready in STS che il dato è presente. A questo punto il software deve fare 256 letture di BR che restituisce ogni volta 16 bit successivi del buffer.
Per fare una **scrittura**: Dobbiamo dire in quale settore scrivere, dare il comando di scrittura e scrivere tutti i 512 byte nel buffer interno facendo 256 scritture in BR. A questo punto il controllore fa la scrittura
↓
Adesso le coordinate sono sostituite dal **logical block address** (numero logico del settore) → in questo modo un hard disk può essere grande al massimo 128 GB ($2^{28} \cdot 2^9 = 2^{37}$ byte): modificato con la stessa tecnica di accesso del timer

ESEMPIO

```
void hd_set_lba(natl lba)
{                                   ↳ logical block address
        natb lba_0 = lba,
             lba_1 = lba >> 8,
             lba_2 = lba >> 16,
             lba_3 = lba >> 24;

        outputb(lba_0, iSNR);
        outputb(lba_1, iCNL);
        outputb(lba_2, iCNH);
        natb hnd = (lba_3 & 0x0F) | 0xE0;
        outputb(hnd, iHND);
}
```

altre funzioni (abilita logical numbering)

```
void hd_start_cmd(natl lba, natb quanti, natb cmd)
{
        hd_set_lba(lba);
        outputb(quanti, iSCR);          // numero di set
tori
        outputb(cmd, iCMD);             // comando di sc
rittura
}
```

```
natb buff[8*512];
int main()
{
        natl lba = 1;
        natb quanti = 2;  : quanti settori
        for (int i = 0; i < quanti*512; i++)
                buff[i] = 'f'; → buffer con dati
                → comando scrittura
        hd_start_cmd(lba, quanti, WRITE_SECT);
        for (int i = 0; i < quanti; i++)
                hd_output_sect(&buff[i * 512]);

        str_write("OK\n");
        pause();
        // ...
        return 0;
}
```

```
void hd_wait_for_br()
{
        natb s;
        do
                s = inputb(iSTS);
        while (s & 0x88 != 0x08);
}

void hd_output_sect(natb vetto[])
{
        hd_wait_for_br();
        outputbw(reinterpret_cast<natw*>(vetto), 256, iBR);
}
```

attende che HDD sia pronto

scrive 256 volte in iBR

```
// scrive una sequenza di parole in una porta di I/O
        .global outputbw
outputbw:
        .cfi_startproc
        movq %rsi, %rcx
        movq %rdi, %rsi
        cld
        rep
        outsw
        ret
        .cfi_endproc
```

Le interruzioni

- API per interruzioni semplici

Abbiamo visto in precedenza che vi sono alcuni casi quali la lettura della tastiera o la lettura/scrittura dell'hard disk che richiedono cicli di attesa per avere alcuni bit nel valore desiderato. Questo accade perché abbiamo un'unica CPU con un unico flusso di controllo. Questi bit vanno riletti ogni volta per capire il loro valore attuale

↓

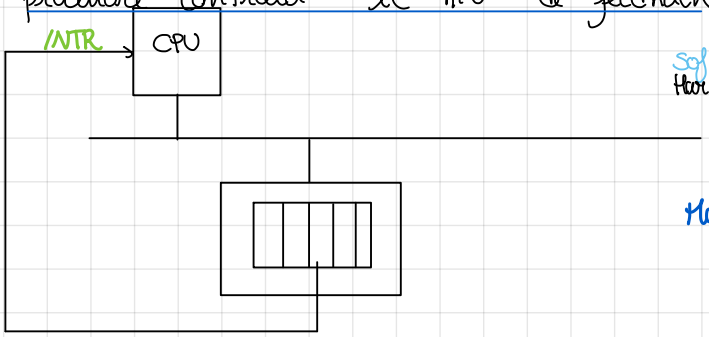
Le **interruzioni** nascono negli anni '60 Dijkstra abbiamo una telescrivente. Per collegarla al computer e farle stampare i caratteri dal computer dobbiamo porre degli elettromagneti sotto ai tasti. Nel computer è sufficiente avere un registro con il codice del tasto da premere ed un demultiplexer collegato a ciascun elettromagnete. Per scrivere una nuova lettera il programmatore deve attendere che l'operazione precedente sia terminata. È necessario un feedback per sapere che l'operazione è finita. Supponiamo invece di voler scrivere un'applicazione che percorra una tabella

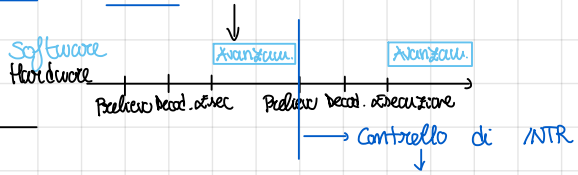
x	f(x)
0.000	val 0
0.001	val 1
0.002	val 3
...	...

→ L'applicazione deve ricalcolare periodicamente i valori della funzione e poi eventualmente stampare il valore nel dispositivo precedente. Questo significa stampare un carattere per volta ed attendere il termine dell'operazione. Vorremmo, mentre si stampa, iniziare i calcoli per la riga successiva

↓

Modifichiamo il processore e le periferiche in modo da portare il bit di feedback dei dispositivi al processore direttamente. Inoltre, il processore nel tempo fa il ciclo ripetuto di fetch-execute. Tra la fine dell'esecuzione dell'istruzione ed il prelievo della successiva il processore controlla il bit di feedback: INTR (richiesta di interruzione).





Hardware: vantaggioso perché il test non grava sul software e non costa quindi nulla

↓

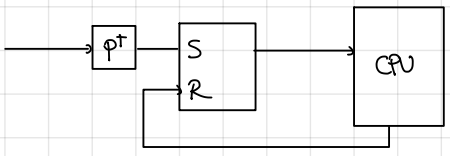
È il programma che gestisce il caso in cui INTR=1 e comunica cosa fare al processore (ripeto). → L'indirizzo della routine è di default e quella zona di RAM deve contenere l'azione da fare in caso di interruzione. Inoltre, è necessario inserire un'altra routine per terminare il programma principale. Questa, nei processori Intel, consiste nel ripristino in pila di IP. Al termine dell'interruzione basta quindi sufficiente fare una POP. → Gestione **asimmetrica**: la routine di interruzione deve ripetere di dover restituire il controllo al programma principale nel minor tempo possibile

⚠ Lo stato della macchina può essere **determinato dalle routine di interruzione.**

↓ Necessarie alcune **correzioni**

a. La CPU **NON** deve vedere la stessa interruzione più volte

↓ Metodo



→ Il pT permette di settare il latch SR in caso di interruzione e questo viene resettato dopo la sua esecuzione

b. Le richieste di interruzione NON possono utilizzare dei registri senza salvarli e ripristinarli → I compilatori C/C++ ignorano le interruzioni e quindi le utilizzano come funzioni standard. Si utilizza quindi l'Assembly:

```
a_routine:
    // salva_registri
    call i_routine
    // carica_registri
    ~~RET~~
```

↙ Non è sufficiente

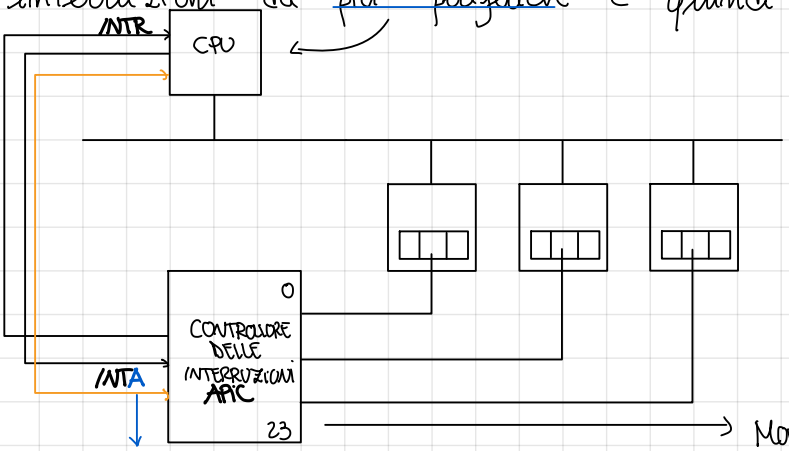
Il programmatore può stabilire di non ammettere interruzioni in dati punti di programma. Per permettere questo, il processore utilizza il bit 9 del registro dei flag (EFLAG). Se è = 1 le interruzioni sono ammesse, altrimenti non lo sono. Il flag può essere messo a 0 con CLI e ad 1 con STI. Dopo aver accettato la richiesta di interruzione inibisce il processore azzera il flag dopo aver salvato il valore dei flag in pila. → IRET Q

↓

Preleva IP ed il valore del registro dei flags. Q si usa per motivi di compatibilità. Senza Q viene usata la IRET a 32 bit.

↓

Limitazione: unica periferica → In situazioni normali però è necessario accettare interruzioni da più periferiche e quindi da più interfacce.



acknowledge: permette l'handshake

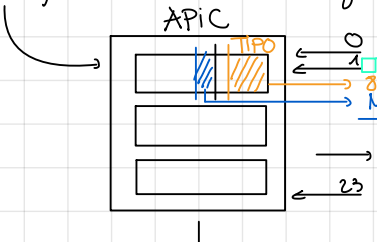
Monitora lo stato dei piedini di ingresso e passa le richieste una per volta alla CPU: 24 piedini che vengono costantemente controllati

↓

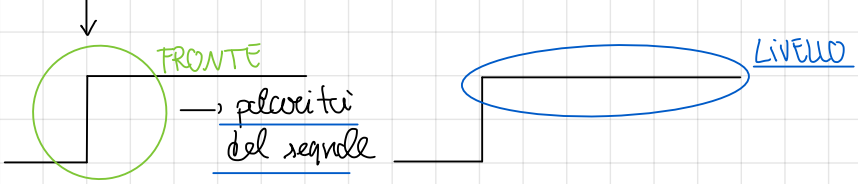
Le interruzioni sono "richieste di attenzione" da parte dei dispositivi e si aspettano risposte → APIC memorizza la richiesta in un registro interno, le prioritizza e la invia al processore. Il controllore associa un numero a ciascuna interruzione, detto tipo dell'interruzione, che viene passato alla cpu attraverso un bus dedicato. Il processore dopo aver ricevuto la richiesta consulta la IDT (interrupt descriptor table). L'indirizzo della tabella si trova in un registro del processore scritto dal programmatore dopo aver compilato la tabella. Questa ha un'entrata per ogni tipo di interruzione. Per ogni entrata abbiamo l'indirizzo a cui saltare → **interruzioni vettorizzate**. IDT ha inoltre un flag per sapere se abilitare o no le interruzioni

↓ Ma come fa APIC ad associare un tipo ad ogni piedino?

Programmabile via software: Vi sono 24 registri (uno per piedino)



→ Vi sono **altri flag** per riconoscere il tipo di segnale che caratterizza l'interruzione



→ fronte: cambiato del segnale

ed anche per determinare il **livello attivo**, che può essere 0 o 1

Il registro **end of interrupt** permette di gestire l'handshake con il processore che comunica ad APIC che l'interruzione è stata gestita. Questo permette di gestire le interruzioni di livello.

ESEMPIO

```
int main()
{
        char spinner[] = { '|', '/', '-', '\\' };

        clear_screen(0x0f);
        apic_set_VECT(1, KBD_VECT);
        gate_init(KBD_VECT, tastiera);
        apic_set_TRGM(1, false);
        apic_set_MIRQ(1, false);
        enable_intr_kbd();

        natq spinpos = 0;
        while (!fine) {
                video[12*80+40] = attr | spinner[spinpos % 4];
                spinpos++;
        }
        return 0;
}
```

tastiera a piedino 1 APIC

scrivere tastiera stampa cod. scansione

barra che ruota

Stabilisco tipo interruzione → `const natb KBD_VECT = 0x40;`

Errori: il programma non scrive più di un tasto perché non comunichiamo la fine dell'interruzione ad APIC e manca la IRETQ perché siamo in C++. Non viene quindi aggiornato IF

L'APIC ha anche 2 registri a 256 bit: **ISR** e **IRR**. Quando arriva una richiesta riconosce il tipo e mette ad 1 il bit corrispondente al tipo di IRR: memorizza le richieste **pending**, cioè quelle non ancora inoltrate al processore. ISR contiene invece quelle **già inoltrate** ma che **NON** hanno ancora ricevuto un **end of interrupt**: richieste in servizio. Può essere riconosciuta una nuova richiesta dello stesso tipo al fronte mentre dobbiamo attendere end of interrupt per quelle di livello

↓ Nel debug (programma già visto precedentemente)

`intr: abilitati` : il bit a di EFLAGS è abilitato

scrivendo **apic** possiamo vedere il suo stato:

```
ISR: 0000:0000:0000:0000:0000:0000:0000:0000:0000:0000:0000:0000:0000:0000:0000:0000
IRR: 0000:0000:0000:0000:0000:0000:0000:0000:0000:0000:0000:0000:0000:0000:0000:0000
```

16 bit per volta

apic_set serve in memoria in quanto questo ha i registri mappati in memoria →

```
[ 1]: polarity=high mode=level vector=40 (masked)
ISR: 0000:0000:0000:0000:0000:0000:0000:0000:0000:0000:0000:0000:0000:0000:0000:0000
IRR: 0000:0000:0000:0000:0000:0000:0000:0000:0000:0000:0000:0000:0000:0000:0000:0000
```

gate_init scrive in entrata della interrupt table →

```
(gdb) idt 0x40
[40]: sys intr                20017f (:tastiera)
```

apic_set: 2 funzioni → **TRGM**: cambia modalità riconoscimento
MIRQ: permette di mascherare interruzioni

`enable_intr_kbd();` → Abilita interruzioni: tipicamente necessario farlo 3 volte. Devono infatti essere abilitate nell'APIC, nell'interfaccia e nel processore

```
using namespace vid;
using namespace kbd;

void enable_intr_kbd()
{
        outputb(0x60, iCMR);
        outputb(0x61, iTBR);
}

void disable_intr_kbd()
```

} in questo modo la tastiera manda le interruzioni

Quando si riceve un'interruzione:

```
(gdb) apic                                                    Bit 40: interruzioni da tastiera
[ 1]: polarity=high mode=edge  vector=40
ISR: 0000:0000:0000:0000:0000:0000:0000:0000:0000:0000:0000:0001:0000:0000:0000:0000
IRR: 0000:0000:0000:0000:0000:0000:0000:0000:0000:0000:0000:0000:0000:0000:0000:0000
```

+

interrupt disabilitati dal processore

+

indirizzo di ritorno in pila e valore dei registri

↓

Alla fine per tornare al programma principale viene fatta una RET

⚠ ERRATO: non viene ripristinato il valore dei registri dei flag e quindi le interruzioni restano disabilitate ⟶ bisogna usare **IRETQ** e per fare questo partiamo dall'assembler aggiungendo soltanto le istruzioni necessarie a far funzionare l'interruzione

↓

File aggiunto:

```
.global a_tastiera
a_tastiera:
        call c_tastiera
        iretq
```

Funzione c_tastiera:

```
extern "C" void a_tastiera();
extern "C" void c_tastiera()
{
        natb c = inputb(iRBR);
        if (c == 0x01)  // make code di ESC
                fine = true;
        for (int i = 0; i < 8; i++) {
                char_write('0' + !!(c & 0x80));
                c <<= 1;
        }
        char_write('\n');
}
```

All'uscita adesso gli interrupt vengono di nuovo abilitati correttamente

Dobbiamo poi dire all'APIC che l'interruzione è stata gestita

↳ aggiungo **apic send EOI();** alla funzione c_tastiera

Notiamo adesso che l'istruzione "torna indietro" ⟶ stiamo mettendo chiamate alla funzione c_tastiera in mezzo al programma che il compilatore non vede. La funzione può quindi usare registri senza salvarli e ripristinarli. Utilizziamo di nuovo l'assembler ed in particolare alcune macro della libce ⟶

```
#include "libce.s"
.global a_tastiera
a_tastiera:
        salva_registri
        call c_tastiera
        carica_registri
        iretq
```

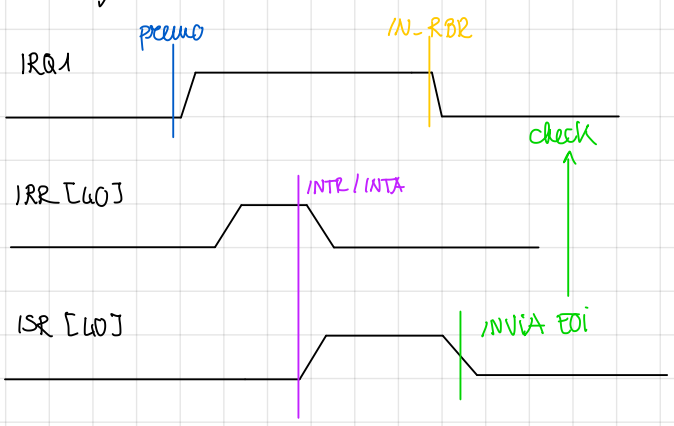
Dobbiamo adesso permettere di uscire dal programma con la pressione del tasto ESC ⟶ ottimizzando, la variabile fine non viene considerata perché il suo valore viene cambiato dalle interruzioni non viste dal compilatore ⟶ si aggiunge la keyword **volatile** nella dichiarazione della variabile.

↓

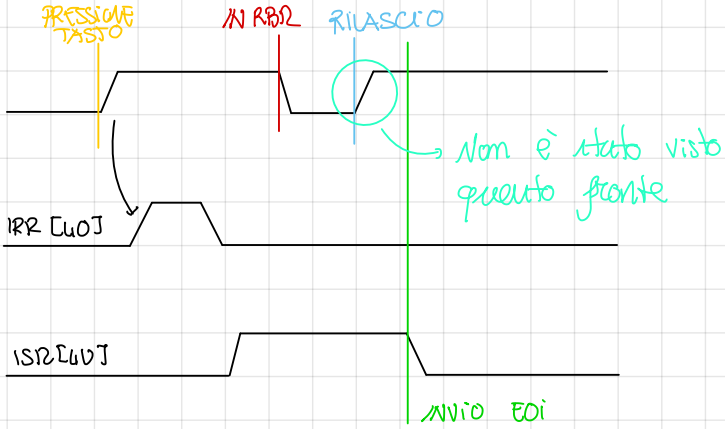
⚠ ATT: avere variabili condivise tra programma principale ed interruzioni può portare alla nascita di errori **tempo - varianti**.

Ma cosa succede succede se avevamo scritto l'interruzione **sul livello** anteriori

sul fronte?



oppure



La coda funziona poiché la tastiera supporta un handshake. Se IRQ1 rimane ad 1 dopo l'EOI viene riconosciuta una nuova interruzione. L'handshake consiste nella lettura di RBR da parte del software che abbassa IRQ1

↓

Il timer non ha un handshake. Le richieste devono essere necessariamente gestite sul fronte

```
volatile natq counter = 0;
extern "C" void c_timer()
{
        counter++;
        apic_send_EOI();
}
```

```
#include <libce.s>

        .global a_timer
a_timer:
        salva_registri
        call c_timer
        carica_registri
        iretq
```

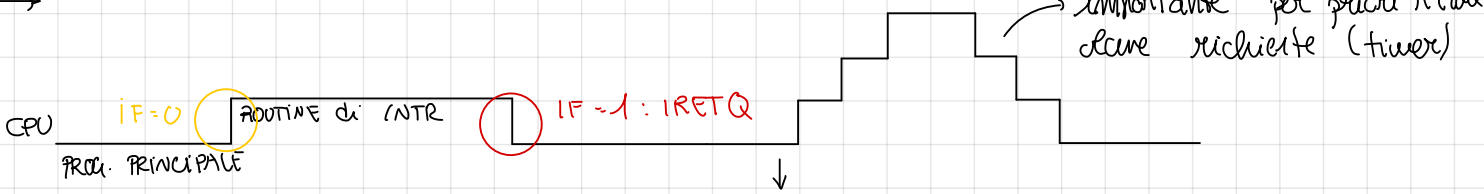
```
void attiva_timer(natw N)
{
        outputb(0b00110111, iCWR); // timer 0, modo 3
        outputb(N, iCTR0_LOW);
        outputb(N >> 8, iCTR0_HIG);
}
```

```
extern "C" void a_timer();
const natb TIM_VECT = 0x50;
int main()
{
        apic_set_VECT(2, TIM_VECT);
        gate_init(TIM_VECT, a_timer);
        attiva_timer(50000); // periodo di circa 50ms
        apic_set_TRGM(2, true); // false: fronte, true: livello
        apic_set_MIRQ(2, false);

        for (volatile int i = 0; i < 100000000; i++)
                ;
        printf("counter = %d\n", counter);
        pause();
}
```

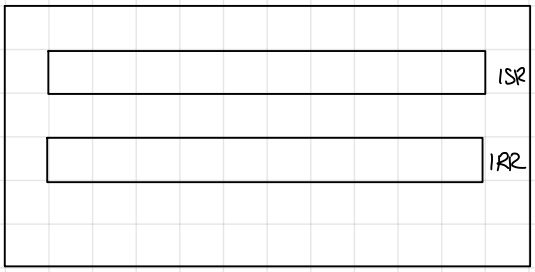
SBAGLIATO

- GESTIONE di PIÙ RICHIESTE di INTERRUZIONE

Cosa succede se vogliamo gestire più richieste di interruzione contemporaneamente? APIC le può gestire in base alla priorità



È possibile non far resettare IF: è possibile annidare le interruzioni che devono chiudersi a stack



→ La priorità della richiesta è associata al tipo: ogni 16 tipi abbiamo una classe di priorità diversa (2ª cifra esadecimale diversa)

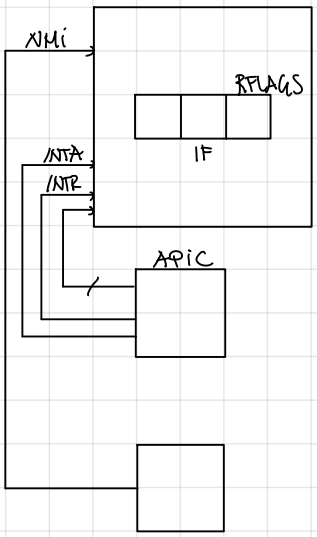
Se avviene una richiesta con classe superiore a quella attualmente in servizio la richiesta viene accettata ed il processore viene di nuovo interrotto. Se invece la richiesta ha classe di priorità inferiore a quella in servizio deve aspettare. Al momento di end of interrupt APIC azzera ISR e guarda IRR. Se c'è un'interruzione con priorità superiore alla seconda presente in ISR questa viene eseguita, altrimenti attende

ESEMPIO

```
int main()
{
        clear_screen(0x0f);

        apic_set_VECT(1, KBD_VECT);
        trap gate_init(KBD_VECT, a_tastiera);
        apic_set_MIRQ(1, false);
        enable_intr_kbd();
        permette annidamento
        apic_set_VECT(2, TIM_VECT);
        trap gate_init(TIM_VECT, a_timer);
        attiva_timer(50000); // periodo di circa 50ms
        apic_set_TRGM(2, false); // false: fronte, true: livello
        apic_set_MIRQ(2, false);

        natq spinpos = 0;
        while (!fine) {
                video[12*80+40] = attr | spinner[spinpos % 4];
                spinpos++;
        }
        return 0;
}
```

```
extern "C" void c_timer()
{
        static natq timer_spinpos = 0;
        video[12*80+60] = attr | spinner[timer_spinpos % 4];
        timer_spinpos++;
        apic_send_EOI();
}
```

- INTERRUZIONI NON MASCHERABILI



NMI: Non-Maskable Interrupts → causano forzatamente il salto alla routine da eseguire indipendentemente dallo stato del processore

↓

tipo implicito: 2 → IDT: 0, 1, 2 → salto alla routine stessa

↓

utilizzato ad esempio quando salta la corrente

Eccezioni

Possiamo replicare il meccanismo delle interruzioni a cose che accadono internamente al processore (ad esempio la divisione per 0) → il programmatore potrebbe avere interesse a sapere il verificarsi di queste situazioni eccezionali in modo da poterle gestire (ad esempio quando il processore non può eseguire alcune istruzioni)

↓

Deve essere interrotto il programma per eseguire un altro decino dal programmatore. In pratica abbiamo un'interruzione proveniente dall'interno. Viene utilizzato, nei processori Intel, lo **stesso meccanismo** delle interruzioni esterne eccetto che il tipo di interruzione è implicito ed è codificato all'interno del microprogramma stesso. Il processore riconosce 32 diversi tipi di queste interruzioni.

↓ Esecuzione

Le interruzioni esterne possono occorrere in qualsiasi momento mentre il processore le riconosce solo dopo la fase di esecuzione dell'istruzione perché il bit IF viene guardato solo dopo l'esecuzione dell'istruzione. Nelle interruzioni interne invece è possibile che la fase di esecuzione non venga portata a termine. Altre possono essere invece rimandate. Nel primo caso viene salvato l'IP dell'istruzione che ha generato l'istruzione mentre nel secondo caso viene salvato l'IP dell'istruzione successiva. Nel primo caso tipicamente l'errore può essere rimediato. Le prime istruzioni sono di tipo **fault** mentre le seconde sono di tipo **trap**. Esiste un terzo tipo detto **abort** che viene usato per errori gravi che bloccano tutto

ESEMPIO

```
#include <libce.h>

int main()
{
        int x = 3, y = 0;

        return x / y;
}
```

```
WRN     -       Eccezione 0, err=0000000000000000, RIP=000000000020014f
```

indirizzo della IDT: fault

↓ Della IDT creata da libce

```
#include "def.h"
.data
.balign 16
.global idt
idt:
        .space 16 * 256, 0
```

+

```
extern "C" bool init_all()
{
        init_gdt();
        load_gdt();
        init_idt(); // chiama gate_init per le prime 32 entrate
        bochsvga_init();
        clear_screen(vid::attr);
        ini_COM1();
        ini_COM2();
        apic_init();
        apic_reset();
        keyboard_init();
        ctors();
        sti();
        return true;
}
```

inizializza la IDT

prima di chiamare main

```
carica_gate     0       exc_divide_error
carica_gate     1       exc_debug
carica_gate     2       exc_nmi
carica_gate     3       exc_breakpoint
carica_gate     4       exc_overflow
carica_gate     5       exc_bound_re
carica_gate     6       exc_invalid_opcode
carica_gate     7       exc_dev_na
carica_gate     8       exc_double_fault
carica_gate     9       exc_coproc_so
carica_gate     10      exc_invalid_tss
carica_gate     11      exc_segm_fault
carica_gate     12      exc_stack_fault
```

```
#include "def.h"
.global exc_divide_error

exc_divide_error:
        .cfi_startproc
        movq $0, %rdi
        movq $0, %rsi
        popq %rdx
        jmp comm_exc
        .cfi_endproc
```

prende l'IP salvato in pila

Se volessimo gestirla in modo diverso:

```
#include <libce.h>

void div()
{
        printf("divisione per zero!!!");
}

int main()
{
        int x = 3, y = 0;

        gate_init(0, div);

        return x / y;
}
```

∞ volte perché la RET allo stesso indirizzo

⚠ trap: eccezione → l'IP salvato è quello dell'istruzione successiva
entrata IDT → tipo interrupt / trap: interruzioni disabilitate/abilitate

eccezione di tipo TRAP: tipicamente generata per tale scopo (int 3)

```c
#include <libce.h>

int main()
{
        int x = 0;

        x++;
        asm("int3");
        x++;
}
```

```
Eccezione 3, err=0000000000000000, RIP=0000000000200149
```

istruzione successiva

↓

Serve per il debugger in quanto funziona come interruzione e poi il controllo ritorna al programma quando si fa continue

```
200144:       83 45 fc 01             addl   $0x1,-0x4(%rbp)
200148:       cc                      int3
200149:       83 45 fc 01             addl   $0x1,-0x4(%rbp)
```

Viene messo CC qui quando c'è un breakpoint e quindi quando il processore passa di qua causa un'eccezione. Per farlo proseguire poi deve essere reinserita l'istruzione originale e poi deve essere decrementato l'IP in pila di 1. Inoltre, va rimesso il breakpoint al passaggio successivo

↓

Eccezione debug

```
 0]: sys intr            201a85 (:exc_divide_error)
 1]: sys intr            201a5d (:exc_debug)
 2]: sys intr            201b05 (:exc_nmi)
 3]: sys intr            201a35 (:exc_breakpoint)
 4]: sys intr            201b19 (:exc_overflow)
 5]: sys intr            201a21 (:exc_bound_re)
 6]: sys intr            201acf (:exc_invalid_opcode)
 7]: sys intr            201a71 (:exc_dev_na)
 8]: sys intr            201a99 (:exc_double_fault)
 9]: sys intr            201a49 (:exc_coproc_so)
 a]: sys intr            201ae3 (:exc_invalid_tss)
 b]: sys intr            201b3b (:exc_segm_fault)
 c]: sys intr            201b5d (:exc_stack_fault)
 d]: sys intr            201b2d (:exc_prot_fault)
 e]: sys intr            201ac1 (:exc_int_tipo_pf)
 f]: sys intr            201b6b (:exc_unknown_exc)
10]: sys intr            201aad (:exc_fp)
11]: sys intr            201a0d (:exc_ac)
12]: sys intr            201af1 (:exc_mc)
```

Viene **emulata** in modalità **single step**: viene generata un'eccezione dopo l'esecuzione di ogni istruzione

↓

⚠ **trap flag**: il processore genera un'eccezione alla fine di ogni istruzione

↓

disabilitato ad ogni entrata della IDT. Questo verrà riabilitato alla fine dell'eccezione

Come gestiamo il trap flag?

abilitiamo

```
.global enable_single_step
enable_single_step:
        pushf
        orw $0x100, (%rsp)
        popf
        ret
```

→ setta il bit 10

contiene l'indirizzo del registro da modificare

disabilitiamo

```
.global disable_single_step
disable_single_step:
        pushf
        andw $0xFEFF, (%rsp)
        popf
        ret
```

```c
#include <libce.h>


extern "C" void enable_single_step();
extern "C" void disable_single_step();
int main()
{
        int x;
        gate_init (1, a_debug); // per eseguire la nostra routine
        enable_single_step();
        x++;
        x++;
        x++;
        x++;
        x++;
        x++;
        disable_single_step();
}
```

→ Nella libce la routine stampa l'eccezione e il suo numero

↓ disassembly

```
.global a_debug
a_debug:
        salva_registri
        call c_debug
        carica_registri
        iretq
```

+

```c
extern "C" void a_debug();
extern "C" void c_debug()
{
        printf("debug\n");
}
```

↓ risultato

```
QEMU
debug
debug
debug
debug
debug
debug
debug
debug
debug
debug
debug
Premere ESC per proseguire
ome/studenti/CE/lib/ce/boot.bin'
```

Se volessimo stampare l'indirizzo dell'interruzione in pila

```
.global a_debug
a_debug:
        salva_registri
        mov 120(%rsp), %rdi
        call c_debug
        carica_registri
        iretq
```

(primo indirizzo)

```c
extern "C" void a_debug();
extern "C" void c_debug(natq rip)
{
        printf("debug rip=%p\n", rip);
}
```

↓ output

```
debug rip=0000000000200176
debug rip=000000000020017a
debug rip=000000000020017e
debug rip=0000000000200182
debug rip=0000000000200186
debug rip=000000000020018a
debug rip=000000000020018e
debug rip=00000000002001a4
debug rip=00000000002001a5
debug rip=00000000002001ab
debug rip=00000000002001ac
Premere ESC per proseguire
```

→ attivata nella POP di enable_single_step: vede trap flag ad 1

→ tutte nel main

→ ha qui in disable_single_step

→ la pop q disattiva il flag ma lo fa in ritardo e quindi viene di nuovo generato

Il breakpoint può essere quindi ricreato al suo posto ad ogni esecuzione togliendo il trap flag prima di cedere il controllo e facendo eseguire al processore l'istruzione per poi settarlo di nuovo ad 1.

step

L'istruzione step deve fare lo step di una singola istruzione in linguaggio macchina. Per fare questo il compilatore costruisce una tabella di corrispondenza tra le righe del sorgente e gli IP. Questo fallisce se il compilatore ottimizza e per fare questo viene utilizzato il single step

Protezione

SISTEMISTI: coloro che mettono i programmi nella macchina

↓

UTENTE 1 : | UTENTE 2 | UTENTE 1 | → caricato da sistemisti → | |

UTENTE 2

JOB1	JOB2	JOB1

: Si suppone che lo stato del programma 1 venga salvato per poter eseguire contemporaneamente il programma 2

↓ limite nostro

↓

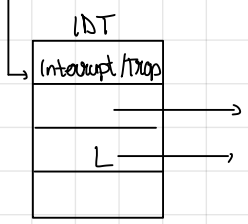
L'utente 1 non ha nessun interesse a cedere la CPU all'utente 2 e l'utente 2 non ci interessa a restituire il controllo all'utente 1 con le interruzioni e quindi potrebbe ad esempio disabilitare le interruzioni. Dobbiamo avere quindi dei modi per far avere ai programmi determinati comportamenti → i linguaggi di programmazione sono troppo espressivi e non è sufficiente in nessun caso analizzare il programma staticamente per capirne il suo comportamento

↓

È necessario l'aiuto del **processore**: sa quale istruzione sta eseguendo in ogni caso → deve sapere **chi** gli sta chiedendo di fare una determinata operazione. Aggiungiamo un **contesto** al resto del programma e modifichiamo il processore in modo da riconoscerlo → **sistema**: eseguite tutte le istruzioni
→ **utente**: alcune istruzioni NON vengono eseguite (STI, CLI, IN, OUT)

↓

È necessario **fornire** al sistema le istruzioni necessarie per dialogare con la macchina in modalità sistema → forniamo delle routine da eseguire in contesto sistema. Abbiamo però bisogno di un'istruzione che cambia il contesto da utente a sistema. → Meccanismo strettamente correlato alle interruzioni: INT $tipo → fa la stessa cosa di un'interruzione esterna o di un'eccezione



→ Routine: primitive di sistema

→ Campo **livello**: indica al processore a quale livello portarsi nell'esecuzione della routine

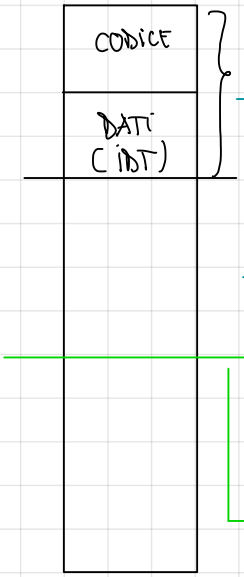
↓

Non viene utilizzata la CALL perché l'utente potrebbe saltare in punti casuali della routine di sistema saltando alcuni controlli. Con l'utilizzo di INT invece l'indirizzo della routine sta in una tabella a cui l'utente non può accedere. Inoltre, vi sono delle zone di memoria a cui l'utente non può accedere in cui sono presenti strutture di sistema o la IDT ad esempio. Se l'utente prova a scrivere su questa memoria viene sollevata un'eccezione con conseguente esecuzione di una routine

↓

In tutti i casi (eccezioni, interruzioni, INT) si salta ad una routine specifica decisa dalla IDT e si passa sempre a livello sistema. Tutti e 3 i meccanismi confluiscono ai gate della IDT che eseguiranno sempre dei controlli

Vi sono alcuni **controlli**



M1: riservata al sistema → il processore parte nella modalità interna e carica il codice ed i dati tra cui la IDT

: il programma prima di lasciare il controllo comunica questo indirizzo e poi mette il processore in modalità utente

M2: utente (il più possibile)

↓

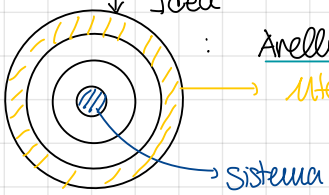
Gli utenti non possono usare le periferiche e le interruzioni esterne. Quando ci sono eccezioni o interruzioni il controllo passa al sistema

Vogliamo proteggere gli utenti tra di loro per non creare conflitti nell'esecuzione di programmi → per ciò supponiamo che non sia possibile avere programmi di utenti diversi in memoria

Nel processore intel

i meccanismi di protezione sono stati aggiunti a partire dal 286 (16 bit).

↓ Idea



: Anelli di protezione → 4 implementati in hardware

Utente: nei livelli più privilegiati possono essere eseguite più istruzioni mentre nel livello utente non ne possono essere eseguite come:

LIDTR

HLT

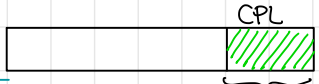
Per IN/OUT, CLI/STI vi è più flessibilità anche a causa dei livelli intermedi

↓

Nel registro RFLAGS c'è un flag (IOPL) che specifica a quale livello possono essere eseguite tali istruzioni → contenendo anche IF questo registro non può essere modificato perché in tal modo potrebbe concedersi da solo privilegi. Per questo la POPF è stata modificata in modo da ignorare le modifiche a livello utente senza però generare alcuna eccezione

↓

La modalità corrente del processore si trova nel registro CS



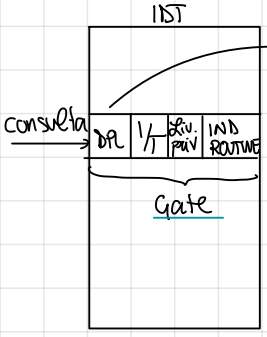
↓ 2 bit meno significativi

Questo, insieme ad altri registri, veniva utilizzato per la segmentazione e poi, a partire dal 386 (32 bit), per la paginazione. Per questa vengono utilizzati solo il livello sistema ed utente.

↓ ⚠ IMPORTANTE: funzionamento

INT

eccezione → in tutti e 3 i casi contiene un tipo



Livello di privilegio che bisogna avere prima di eseguire l'interruzione

↓

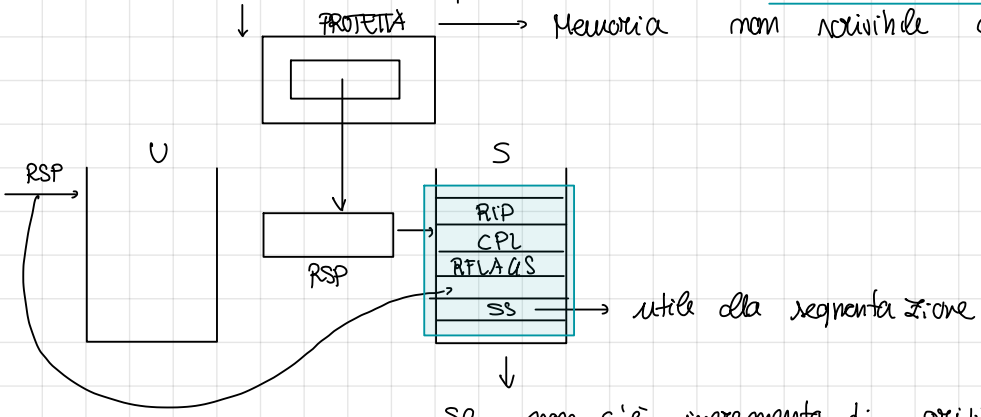
Vale solo per INT perché non ha senso per le interruzioni esterne → si assume che provenga dal sistema (eccetto per INT3)

Su quali gate è necessario mettere un DPL massimo? Le eccezioni devono verificarsi solo esclusivamente al verificarsi effettivo dell'evento che le ha generate. Questo vale anche per le interruzioni esterne.
Il livello di privilegio inoltre, attraversando un gate, non può diminuire. Il motivo è che non possiamo fidarci del codice utente

↓ **Attraversamento** del gate

Se il privilegio aumenta il processore cambia pila il cui indirizzo è scritto all'interno del processore e la pila non può essere scritta dall'utente

↓ PROTETTA → Memoria non scrivibile dell'utente



→ utile alla segmentazione

↓

Se non c'è incremento di privilegio questi elementi vengono salvati nella pila corrente senza cambiarla

Ma perché cambiano pila e non facciamo decidere l'indirizzo all'utente? Ciò che è scritto in RSP è deciso dall'utente e quindi potrebbe puntare alla memoria di sistema. Il processore quindi corromperebbe la memoria di sistema. Inoltre, ci permette di garantire l'affidabilità delle informazioni in memoria sistema.

↓ Restituzione controllo

IRETQ: legge da RSP attuale 5 parole lunghe → dove ritornare, a che livello
↓ di privilegio, come rimettere i flag e come reimpostare la pila
Unico meccanismo per scendere di privilegio e non può salire: eccezione perché l'utente potrebbe cambiare CPL a piacere

```
#include "def.h"
.data
.balign 16
.global sys_tss
.global gdt ——→ come per la segmentazione: parti di memoria contenenti un tipo
.global gdt_pointer descritte nella gdt tramite descrittore per ciascun segmento
gdt:
        .quad 0         //segmento nullo
code_sys_seg: ——→ segmento codice sistema
        .word 0b0               //limit[15:0]   not used
        .word 0b0               //base[15:0]    not used
        .byte 0b0               //base[23:16]   not used
ATTRIB. .byte 0b10011010        //P|DPL|1|1|C|R|A|  DPL=00=sistema
segmento .byte 0b00100000       //G|D|L|-|-------|  L=1 long mode
        .byte 0b0               //base[31:24]   not used
code_usr_seg: ——→ segmento codice utente
        .word 0b0               //limit[15:0]   not used
        .word 0b0               //base[15:0]    not used
        .byte 0b0               //base[23:16]   not used
        .byte 0b11111010        //P|DPL|1|1|C|R|A|  DPL=11=utente
        .byte 0b00100000        //G|D|L|-|-------|  L=1 long mode
        .byte 0b0               //base[31:24]   not used
data_usr_seg:
        .word 0b0               //limit[15:0]   not used
        .word 0b0               //base[15:0]    not used
```

Ad ogni istante il processore usa un segmento codice, un segmento dati ed un segmento pila: registri CS, DS, SS che contengono l'identificatore del segmento corrispondente → identificati da indirizzo di partenza e dimensione → eccezione se accesso fuori da segmento

↓

Scomodi da utilizzare

Per la segmentazione il livello di privilegio del processore è pari a quello del segmento codice corrente. Per avere più livelli di privilegio è necessario definire segmenti di codice con privilegio differente. → Per cambiare CS è necessario usare le regole dei gate

+

```
data_usr_seg:
        .word 0b0           //limit[15:0]   not used
        .word 0b0           //base[15:0]    not used
        .byte 0b0           //base[23:16]   not used
        .byte 0b11110010    //P|DPL|1|0|E|W|A|  DPL=11=utente
        .byte 0b00000000    //G|D|-|-|-------|
        .byte 0b0           //base[31:24]   not used
des_tss:
        .quad 0
        .quad 0
end_gdt:
.balign 16
```

data_usr_seg: ⟶ il registro ss deve avere un valore valido (almeno a livello utente)

des_tss: ⟶ indirizzo TSS nuova pila quando il privilegio del processore cambia

ESEMPIO

```
.global liv_sistema
liv_sistema:
        movq $0x8, 8(%rsp)
        movq $0, 32(%rsp)
        iretq

.data
.balign 16
.global sys_tss, sys_tss_end
sys_tss:
        .fill 1, 4, 0
        .quad sys_stack_end
        .fill 11, 8, 0
        .fill 1, 2, 0
        .word sys_tss_end - sys_tss
sys_tss_end:

.bss
sys_stack:
        .space 4096, 0
sys_stack_end:
/\<sys_stack_end\>
```

.quad sys_stack_end ⟶ bottom pila per puntaggio a livello sistema

.bss ⟵ . align 16

```
#include <libce.h>
void foo()
{
        natb c;
        printf("Provo a leggere RBR\n");
        c = inputb(0x60);
        printf("Ho letto RBR: %2x\n", c);
}

void foo2()
{
        printf("provo ad eseguire hlt\n");
        asm("hlt");
}

void foo3()
{
        printf("disabilito le interruzioni\n");
        asm("cli");
        printf("riabilito le interruzioni\n");
        asm("sti");
}
```

```
extern "C" void liv_utente();
extern "C" void liv_sistema();
int main()
{
        gate_init(0x70, liv_sistema);

        // proviamo ad eseguire foo() prima a livello sistema e poi a livello
        // utente
        foo();
        // portiamo il processore a livello utente
        liv_utente();
        // ora siamo a livello utente
        foo();
        // [provare anche con foo2() e foo3()]
```

Vogliamo una funzione che ci permetta di passare al livello utente: conosciamo solo la IRET per fare questo

```
modifichiamo la pila in modo da poter eseguire una ireqt che porti il
processore a livello utente.  Vogliamo tornare comunque al chiamante e senza
passare ad una nuova pila, quindi effettuiamo la seguente trasformazione:

                                +---------+
                                |   rit   | ---> inserito da CALL
                                +---------+
                                | cs usr  | ---> selettore segmento utente
                                +---------+
finta pila          ==>         | rflags  | ---> valore flags
per IRETQ                       +---------+
+---------+                     |   rsp   .------+ ---> valore nuovo puntatore a pila
|   rit   |                     +---------+      |
+---------+                     | ss usr  | -----+---> per tornare a liv. utente
                                +---------+      |
                                              <--+
```

```
.global liv_utente
liv_utente:
        popq %rax       # rit
        movq %rsp, %rdi
        pushq $0x1b     # selettore dati utente (ss usr)
        pushq %rdi      # rsp salvato in precedenza
        pushq $0x0200   # rflags con IF=1 e IOPL=sistema
                        # provare anche con
                        # $0x3200 (IF=1 e IOPL=utente)
        pushq $0x13     # selettore codice utente (cs usr)
        pushq %rax      # rit salvato in precedenza
        iretq
```

Output

```
QEMU
Provo a leggere RBR
Ho letto RBR: fa
Provo a leggere RBR
```

debug

```
modo: sistema  intr: abilitati   io: permesso   cr3: 0x00001000
```

dobbiamo guardare IOPL ed il modo attuale: in IOPL c'è scritto quando sono abilitate le istruzioni di I/O

↓ dopo liv. utente

```
modo: utente   intr: abilitati   io: vietato    cr3: 0x00001000
```

↳ La printf funziona perché scrive in memoria ma non può spostare il cursore perché dovrebbe usare out

inputb esegue istruzione di I/O: eccezione 13 (general protection fault)

```
[ d]:[sys intr          201d0c (:exc_prot_fault)
```

gate porta a livello sistema

verde: dal livello di privilegio attuale si può attraversare il gate

Chiamando foo2 e foo3: eccezione 13
Impostando IOPL ad utente: pushq $0x3200

↓ eseguo foo

```
QEMU
Provo a leggere RBR
Ho letto RBR: fa
Provo a leggere RBR
Ho letto RBR: fa
Premere ESC per proseguire
```

Proviamo a tornare a livello sistema con IRETQ

```
.global liv_sistema_bad
liv_sistema_bad:
        popq %rax       # rit
        movq %rsp, %rdi
        pushq $0x0      # selettore dati utente (ss usr)
        pushq %rdi      # rsp salvato in precedenza
        pushq $0x0200   # rflags con IF=1 e IOPL=sistema
                        # provare anche con
                        # $0x3200 (IF=1 e IOPL=utente)
        pushq $0x8      # selettore codice utente (cs usr)
        pushq %rax      # rit salvato in precedenza
        iretq
```

→ Ottengo eccezione 13

Proviamo invece ad alternare il livello tramite il passaggio da gate

```
.global liv_sistema
liv_sistema:
        movq $0x8, 8(%rsp)
        movq $0, 32(%rsp)
        iretq
```

→ in generale per tornare al livello sistema il gate è associato a questa funzione

↓ voglio fare la cosa vietata

Modifichiamo la funzione componi_gate:

```
#include "def.h"

.global componi_gate_utente
componi_gate_utente:
        .cfi_startproc
        movq %rsi, %rax         // offset della routine

        movw %ax, (%rdi)        // primi 16 bit dell'offset
        movw $SEL_CODICE_UTENTE, 2(%rdi)
                                a code normali è sistema
        movw $0, %ax
        movb $0b11101110, %ah           // byte di accesso
                                        // (presente, 32bit, tipo interrupt)
        andq $0x1, %rdx
        orb %dl, %ah
        movb $0, %al                    // la parte bassa deve essere 0
        movl %eax, 4(%rdi)      // 16 bit piu' sign. dell'offset
                                        // e byte di accesso
        shrq $32, %rax                  //estensione a 64 bit dell'offset
        movl %eax, 8(%rdi)
        movl $0, 12(%rdi)               //riservato

        ret
        .cfi_endproc
```

+

```
void gate_init_usr(natl num, void routine())
{
        gate gg;

        componi_gate_utente(gg, routine, false /* interrupt */);
        idt[num] = gg;
}

void trap_init_usr(natl num, void routine())
{
        gate gg;

        componi_gate_utente(gg, routine, true /* trap */);
        idt[num] = gg;
}
```

↓

```
void gate_init_usr(natl, void());
int main()
{
        gate_init_usr(0x70, liv_sistema);

        // proviamo ad eseguire foo() prima a livello sistema e poi a livello
        // utente
        foo();
        // portiamo il processore a livello utente
        liv_utente();
        // ora siamo a livello utente
        //foo3();
        // [provare anche con foo2() e foo3()]

        // nel caso in cui riusciamo ad arrivare fin qui (per esempio con foo()
        // o foo3() se IOPL=utente) dobbiamo poi ritornare alla funzione start
        // della libce. Questa vorrà eseguire alcune operazioni privilegiate e,
        // se torniamo mentre siamo ancora a livello utente, si genererà
        // un'eccezione di protezione.  Per tornare a livello sistema abbiamo
        // predisposto un gate della IDT (numero 0x70) ed eseguiamo una INT per
        // attraversarlo
        asm("int $0x70");
        pause();
}
```

← eccezione 13

Multiprogrammazione

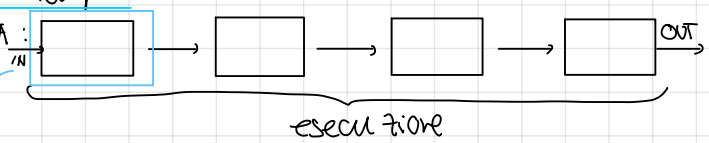
Il sistema multiprogrammato tenta di eseguire più programmi contemporaneamente in modo però concorrente dato che abbiamo una sola CPU.

UTENTE 1	SYS	UTENTE 2	SYS	UTENTE 1

: in questo il processore può saltare tra più programmi

processo: programma in esecuzione su dati ben precisi. È qualcosa che si svolge nel tempo

STATI del SISTEMA:



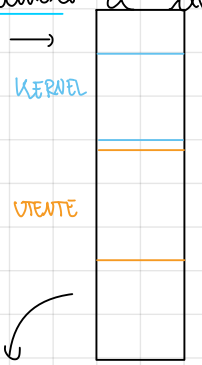
Contiene tutto il necessario per far proseguire l'esecuzione: registri del processore, memoria del sistema → ciascuno stato rappresenta un cambiamento di questi elementi e permette di stabilire il successivo (c'è IP e codice) → **permette di saltare da un processo ad un altro**: il sistema salva (ad esempio sul disco) la foto del processo da cui salta e carica in memoria una foto di un altro processo presente sul disco e fa avanzare il processore. Con la stessa metodologia si può tornare al processo precedente

↓

tutti i passaggi richiedono il transito attraverso il sistema perché è l'unico che ha l'accesso al disco ed alle foto → il **sistema** quindi **gestisce i processi**

- PRIMITIVA (CHIAMATA di SISTEMA / SYSTEM CALL)

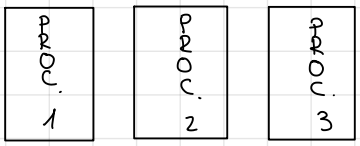
Mattoni fondamentali forniti dal sistema non scomponibili o modificabili. Nei sistemi operativi viene detta **nucleo o kernel** e sta sempre in memoria e sono primitive le funzioni che offre ai programmi o all'utente. Nei sistemi multiprogrammati permettono di creare o terminare processi e sono implementate come sys call perché utilizzano il meccanismo della protezione per innalzare il livello del processore al livello sistema in modo da avere accesso a determinate strutture dati →



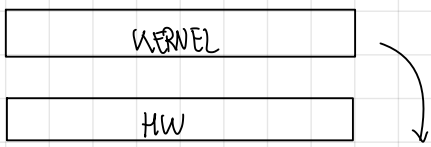
Va in esecuzione per prima all'avvio quando il processore è ancora a livello sistema. Imposta la idt, la gdt ed altro in modo tale da fornire i gate necessari e poi cede il controllo all'utente portando il processore a livello utente. Deve rimanere in memoria ed acquisirà il controllo quando viene attraversato un gate (eccezione, interruzione, INT)

↓

⚠ ATT: il processore fa una cosa per volta



→ Un processo può accedere all'hardware solo tramite il kernel: i processi ottengono la CPU dal kernel e controllo cosa fa il processo tramite le eccezioni comunque "insegnate" dal kernel



Il kernel protegge i processi l'uno dall'altro → Riprendiamo il concetto di contesto: `ADD % RAX, 1000` contesto: in un sistema multiprogrammato il significato dipende da quale processo viene eseguita. Se la eseguono P1 e P2 il contenuto di RAX e di 1000 dipendono dal processo stesso. Ogni **processo** è quindi associato ad un **contesto**. Il kernel quindi **gestisce i contesti** dei vari

processi nella macchina virtuale

```
studenti@ce:~$ cd nucleo-6.6.1/
studenti@ce:~/nucleo-6.6.1$ ls
build  debug  include  io  Makefile  run  sistema  utente  util
```

io → funzioni di I/O

sistema → modalità atomica (livello sistema e INTR disabilitati)

utente → modello utente: modalità utente con INTR abilitate

→ file separati tra di loro

```
studenti@ce:~/nucleo-6.6.1$ ls sistema/
sistema.cpp  sistema.s
```

↓

Vengono compilati insieme e caricati in memoria dal bootloader e riceve il controllo. A questo punto fa le sue azioni, porta il processore a livello utente e viene eseguito il modello utente compilato a parte

↓

Il sistema è multiprocesso ma non multiutente e quindi i processi hanno una parte condivisa con gli altri ed un'altra non condivisa.

utente.cpp

Per scrivere un programma utente: possiamo usare le funzioni di libreria e le primitive di sistema (equivalgono a chiamate di funzioni). Per dire al kernel che il processo è terminato si utilizza la primitiva **terminate_p()**. Si aggiunge **pause()** per far attendere la pressione di un tasto dato che la macchina virtuale viene chiusa al termine di tutti i processi

```
#include <all.h>

int main()
{
        writeconsole("Hello, world!\n", 14);
        pause();
        terminate_p();
}
```

scrive a schermo

lunghezza stringa

↓ nella cartella build

```
→ nucleo git:(master) ✗ ls build
io  mkstart  mkstart.dSYM  parse  sistema  utente
```

Possiamo creare nuovi processi con la primitiva **activate_p**. Questa richiede alcuni parametri. Il primo indica cosa deve fare il processo: per questo è necessario definire una funzione **void** con un **unico argomento natq**. Quando si vuole che tale processo termini si utilizza di nuovo **terminate_p**. Dobbiamo poi specificare la **priorità** con il **secondo parametro** (priorità fissa) ed infine il **livello utente / sistema** tramite il **terzo parametro**. La funzione restituisce un valore numerico che rappresenta l'**identificatore** del processo che può essere utilizzato per dialogare con tale processo

```
int main()
{
        writeconsole("Hello, world!\n", 14);
        activate_p(processo1, 10, 10, LIV_UTENTE);
        terminate_p();
}
```

```
void processo1(natq a)
{
        writeconsole("sono un altro processo\n", 24);
        pause();
        terminate_p();
}
```

La macchina può continuare ad eseguire altri processi anche se uno contiene un errore. Per inviare un messaggio sulla porta seriale si usa

la funzione **flog** specificando la **severità** del log (LOG_+ INFO / WARNING / ERROR) e poi il messaggio

Nel debugger:

```
processi:   1
esecuzione: [0xFFFF, MAX_PRIO]
pronti:     [1, MAX_PRIO] → [0, DUMMY]
p_sospesi:  massima priorità   minima priorità
```

→ Per ogni processo viene fornito un identificatore e la sua priorità

↓

Vengono gestiti dalla funzione **schedulatore()**: fra tutti i processi pronti sceglie il prossimo da eseguire (in ordine di priorità). Nella IDT troviamo inoltre colorate in verde le primitive comuni del sistema e del modello I/O

sistema

```
[20]: sys intr      20070c (sistema:a_activate_p)
[21]: sys intr      20071d (sistema:a_terminate_p)
[22]: sys intr      20072e (sistema:a_sem_ini)
[23]: sys intr      20073f (sistema:a_sem_wait)
[24]: sys intr      200750 (sistema:a_sem_signal)
[25]: sys intr      200761 (sistema:a_delay)
[26]: sys intr      200772 (sistema:a_do_log)
[27]: sys intr      200783 (sistema:a_getmeminfo)
```

I/O

```
[40]: sys trap      10000000272 (io:a_readhd_n)
[41]: sys trap      10000000279 (io:a_writehd_n)
[42]: sys trap      10000000280 (io:a_dmareadhd_n)
[43]: sys trap      10000000287 (io:a_dmawritehd_n)
[44]: sys trap      1000000025d (io:a_readconsole)
[45]: sys trap      10000000264 (io:a_writeconsole)
[46]: sys trap      1000000026b (io:a_iniconsole)
[47]: sys trap      1000000028e (io:a_getiomeminfo)
```

organizzazione memoria

```
---sistema/condiviso---
0000000000000000  S-000-000-000-000: unmapped
0000000000001000  S-000-000-000-001: S W
00000000000b8000  S-000-000-000-270: S W PWT A D
00000000000b9000  S-000-000-000-271: S W
0000000000100000  S-000-000-000-400: S W A D
0000000000101000  S-000-000-000-401: S W
0000000000107000  S-000-000-000-407: S W A D
0000000000108000  S-000-000-000-410: S W
0000000000200000  S-000-000-001-000: S W PS A D
0000000000600000  S-000-000-003-000: S W PS
0000000002000000  S-000-000-020-000: unmapped
00000000fec00000  S-000-003-766-000: S W PWT PCD PS A D
00000000ff000000  S-000-003-770-000: S W PWT PCD PS
0000000100000000  S-000-004-000-000: unmapped
```

mem. video

memorizzare primitive

```
---sistema/privato---
000000ffffff000  S-001-777-777-777: S W A D
---IO/condiviso---
0000010000000000  S-002-000-000-000: S R A
0000010000004000  S-002-000-000-004: unmapped
0000010000010000  S-002-000-000-020: S W A
0000010000011000  S-002-000-000-021: S W A D
0000010000012000  S-002-000-000-022: S W
0000010000020000  S-002-000-000-040: S W A D
0000010000022000  S-002-000-000-042: S W
0000010000121000  S-002-000-000-441: unmapped
---utente/condiviso---
ffff800000000000  U-400-000-000-000: U R A
ffff800000002000  U-400-000-000-002: U W A
ffff800000003000  U-400-000-000-003: U W A D
ffff800000005000  U-400-000-000-005: U W
ffff800000104000  U-400-000-000-404: unmapped
---utente/privato---
fffffffffffff0000  U-777-777-777-760: U W
ffffffffffffff000  U-777-777-777-777: U W A D
```

Rosso: accesso vietato
Giallo: accesso in lettura
Verde: accesso in lettura e scrittura

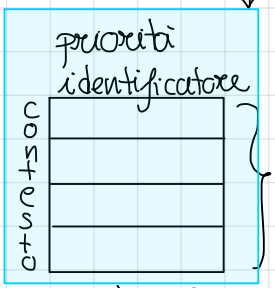
codice / dati

pila

- **GESTIONE dei PROCESSI**

Un processo è una **struttura dati**: il Kernel ha un descrittore che contiene tutte le informazioni utili a descriverlo

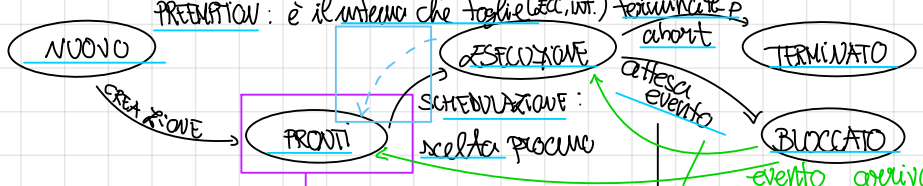
↓ descrittore



Registri del processo (copia privata registri CPU)

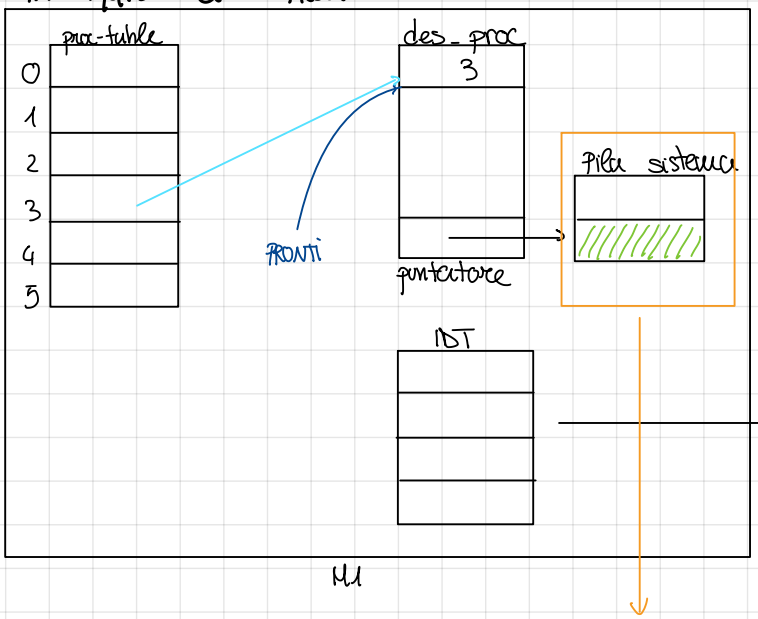
↳ I descrittori sono posizionati nella "zona rossa" della memoria e sono organizzati in liste: stato del processo

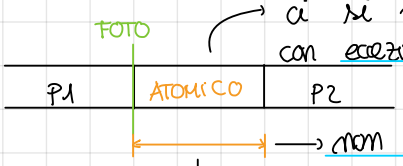
↓ in quali stati può trovarsi



potrebbero proseguire se avvenuto a disposizione le code sono divise per eventi la CPU

- **PASSAGGIO di PROCESSI**





ci si può entrare solo con eccezioni, interruzioni e INT

→ non possono avvenire eccezioni

Ha tutte quante le foto dei registri, sceglie quella da caricare e ne fa il dispatch → funzione activate

Tutte le routine della IDT devono avere la stessa struttura

```
a_routine:
    CALL salva_stato        ┐
    CALL C_routine          │ atomico
    CALL carica_stato       ┘
    IRETQ
```

salva il contenuto di tutti i registri del processore: contengono tutti i valori come erano stati lasciati dal processo. Vengono copiati nel campo contesto del processo in esecuzione

opposto di salva_stato: prende tutti i registri del contesto del processo in esecuzione e li mette nei registri del processore

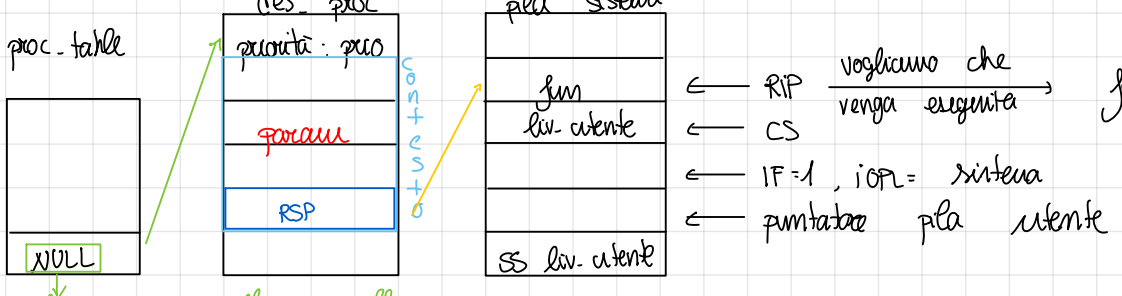
Carica IP, CS, RSP, Flag

È necessaria una pila sistema diversa per ogni processo. Quando un processo entra nel nucleo vengono salvate 5 informazioni: id, livello, precedente, punt_nucleo, contesto e puntatore. Queste sono memorizzate nella pila sistema. Avendo più pile di sistema possiamo ordinarle a piacere i processi. La pila viene decisa automaticamente in quanto viene utilizzato RSP che viene salvato e caricato ogni volta

→ ma siamo sicuri che la pila utilizzata da P2 contenga tutte le informazioni necessarie ad eseguirlo?

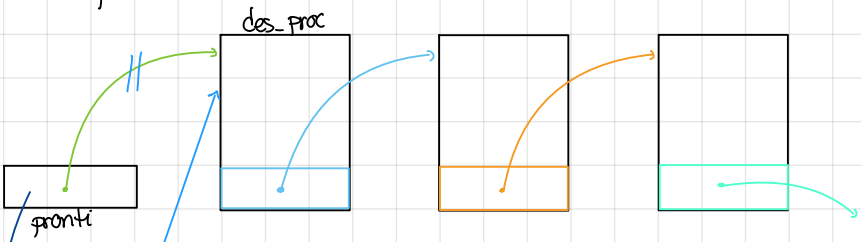
Chiediamoci la storia passata di P2. Supponiamo che non è in esecuzione e quindi è possibile che non ci sia mai stato oppure è possibile che sia già uscito. Nel secondo caso deve essere stata un'eccezione, un'interruzione o una INT. In tutti e 3 i casi ha attraversato la IDT e quindi sicuramente avrà salvato tutte le informazioni necessarie. Nel primo caso è stato creato da activate_p. È necessario inizializzare lo stato di tale processo in modo da renderlo compatibile con la IRETQ ed in particolare lo si rende simile ad un processo appena entrato nel nucleo.

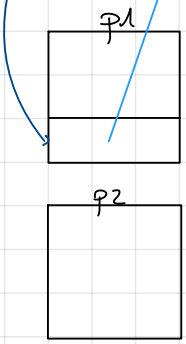
↓ activate_p (fun, param, prio, liv)



Scelto sempre quello a null

Ma perché abbiamo bisogno dell' **atomicità** ?

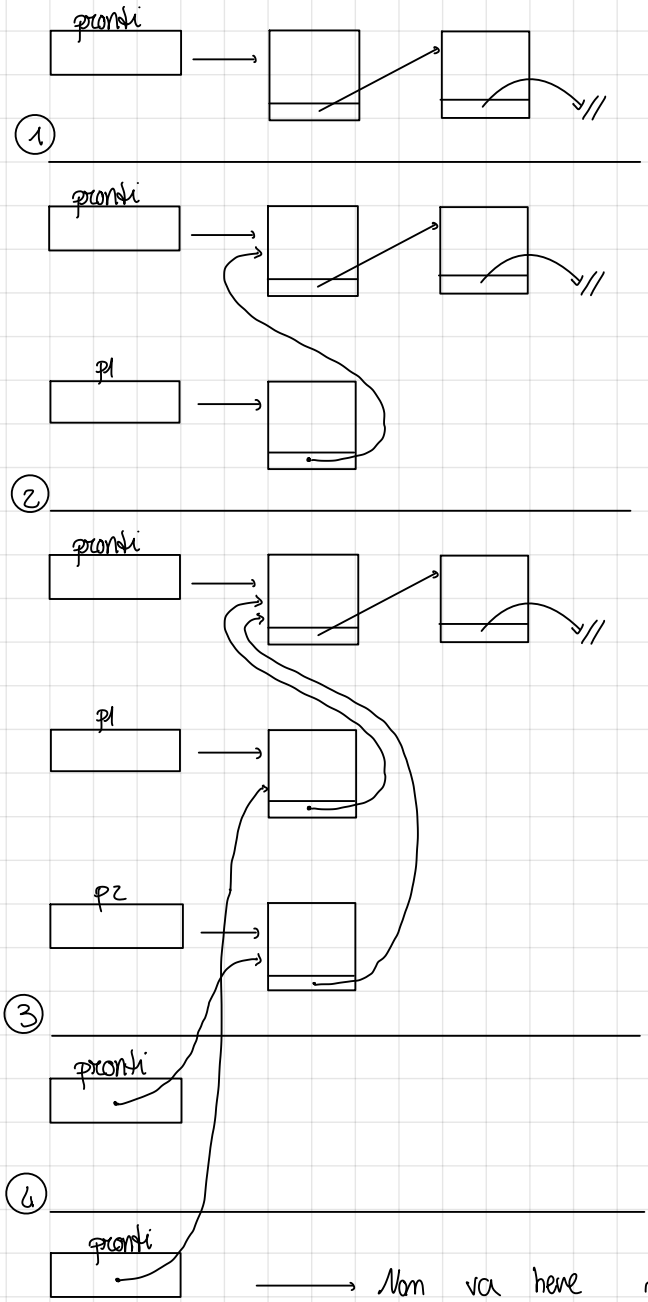




cosa può succedere se durante un inserimento in testa un'altra routine vuole inserire un altro elemento in testa nello stesso posto ? Per inserire in testa devo fare p1 → next = pronti e pronti = p1. Niente vieta che un'interruzione arrivi in mezzo alle due operazioni

1. p1 → next = pronti
4. pronti = p1

1. p2 → next = pronti
3. pronti = p2



→ Non va bene perché abbiamo perso p2

↓

Impediamo quindi che vi possano essere interruzioni. Nel passaggio tra stati consistenti della lista si passa da stati inconsistenti ma all'inizio dell'esecuzione delle funzioni

la struttura deve essere in uno stato consistente. Per questo il codice deve essere eseguito in modo atomico

L'atomicità "trasforma" una serie di più istruzioni in una sola istruzione di linguaggio macchina: si vede soltanto il risultato finale

codice modello sistema

Si ha subito l'inizializzazione della IDT e di alcune parti di memoria

```
#include <costanti.h>
#include <tipo.h>
#include <libce.h>
#include <sys.h>
#include <sysio.h>
```

```
// indirizzo virtuale e indirizzo fisico nello
 spazio di memoria
typedef unsigned short ioaddr;
// indirizzo (fisico) nello spazio di I/O
typedef unsigned char natb;
typedef unsigned short natw;
typedef unsigned int natl;
#if defined(__x86_64__) || defined(CE_UTILS)
typedef unsigned long size_t;
typedef unsigned long natq;
typedef unsigned long vaddr;
typedef unsigned long paddr;
#else
typedef unsigned int size_t;
typedef unsigned long long natq;
typedef unsigned long long vaddr;
typedef unsigned long long paddr;
#endif
```

per utilizzare il bootloader a 32 bit

Processi:

```
struct des_proc {
        natw id;
        natw livello;
        natl precedenza;
        vaddr punt_nucleo;
        natq contesto[N_REG];
        paddr cr3;

        des_proc *puntatore;

        // per debugging
        void (*corpo)(natq);
        natq  parametro;
};
```

descrittore di processo

indice tabella processi

utente o sistema

indirizzo della pila sistema del processo utente: va copiato nell'TSS

Array di natq con tante entrate quanti sono i registri del processore

per fare le liste di processi

debugging

```
des_proc *proc_table[MAX_PROC];
```

```
volatile natl processi;
```

→ numero di processi attivi: a 0 si fa lo shutdown

```
enum { I_RAX, I_RCX, I_RDX, I_RBX,
       I_RSP, I_RBP, I_RSI, I_RDI, I_R8, I_R9, I_R10,
       I_R11, I_R12, I_R13, I_R14, I_R15 };
```

→ costanti indici nell'array contesto

```
des_proc *esecuzione;
des_proc *pronti;
```

```
void inserimento_lista(des_proc *&p_lista, des_proc *p_elem)
+-- 20 lines: {----------

// rimuove da p_lista il processo a più altra priorità e
// lo restituisce
des_proc *rimozione_lista(des_proc *&p_lista)
+-- 12 lines: {----------
```

→ inserimento mantenendo ordine di precedenza (vedi codice da nucleo)

→ Estrazione della testa

```
// inserisce esecuzione in testa alla lista pronti
extern "C" void inspronti()
{
        esecuzione->puntatore = pronti;
        pronti = esecuzione;
}
```

→ Inserisce sempre in testa. Diversa dalla precedente in quanto l'altra nella condizione specificava >= e quindi a parità di priorità inseriva nella posizione successiva

⚠ NON fa il dispatch

```
// sceglie il prossimo processo da mettere in esecuzione
extern "C" void schedulatore(void)
{
// poiché la lista è già ordinata in base alla priorità,
// è sufficiente estrarre l'elemento in testa
        esecuzione = rimozione_lista(pronti);
}
```

→ C'è sempre almeno un processo: dummy

```
void dummy(natq i)
{
        while (processi)
                halt();
        flog(LOG_INFO, "Shutdown");
        end_program();
}

end_program:
        call reboot
        cli
        hlt
```

guarda le interruzioni dopo l'esecuzione

```
// alloca un id non utilizzato.
natl alloca_proc_id(des_proc *p)
{
        static natl next = 0;

        // La funzione inizia la ricerca partendo dall'id successivo
        // all'ultimo restituito (salvato nella variable statica 'next'),
        // saltando quelli che risultano in uso.
        natl scan = next, found = 0xFFFFFFFF;
        do {
                if (proc_table[scan] == nullptr) {
                        found = scan;
                        proc_table[found] = p;
                }
                scan = (scan + 1) % MAX_PROC;
        } while (found == 0xFFFFFFFF && scan != next);
        next = scan;
        return found;
}
```

cerca di partire dall'ultimo id utilizzato

Braccitiva activate_p → entry point nella IDT inserito dalla carica_gate

```
.macro carica_gate num routine dpl
        movq $\num, %rdi
        movq $\routine, %rsi
        movq $\dpl, %rdx
        xorq %rcx, %rcx
        call init_gate
.endm
```

→ sostituisce il testo sopra con le definizioni sotto

crea IDT

```
//                indice          routine                 dpl
// gestori eccezioni:
carica_gate     0               exc_div_error           LIV_SISTEMA
carica_gate     1               exc_debug               LIV_SISTEMA
carica_gate     2               exc_nmi                 LIV_SISTEMA
carica_gate     3               exc_breakpoint          LIV_SISTEMA
carica_gate     4               exc_overflow            LIV_SISTEMA
carica_gate     5               exc_bound_re            LIV_SISTEMA
carica_gate     6               exc_inv_opcode          LIV_SISTEMA
carica_gate     7               exc_dev_na              LIV_SISTEMA
carica_gate     8               exc_dbl_fault           LIV_SISTEMA
carica_gate     9               exc_coproc_so           LIV_SISTEMA
carica_gate     10              exc_inv_tss             LIV_SISTEMA
carica_gate     11              exc_segm_fault          LIV_SISTEMA
carica_gate     12              exc_stack_fault         LIV_SISTEMA
carica_gate     13              exc_prot_fault          LIV_SISTEMA
carica_gate     14              exc_page_fault          LIV_SISTEMA
```

activate_p

```
//primitive comuni (tipi 0x2-)
carica_gate     TIPO_A      a_activate_p     LIV_UTENTE
carica_gate     TIPO_T      a_terminate_p    LIV_UTENTE
carica_gate     TIPO_SI     a_sem_ini        LIV_UTENTE
carica_gate     TIPO_W      a_sem_wait       LIV_UTENTE
carica_gate     TIPO_S      a_sem_signal     LIV_UTENTE
carica_gate     TIPO_D      a_delay          LIV_UTENTE
carica_gate     TIPO_L      a_do_log         LIV_UTENTE
carica_gate     TIPO_GMI    a_getmeminfo     LIV_UTENTE
```

```
a_activate_p:
        .cfi_startproc
        .cfi_def_cfa_offset 40
        .cfi_offset rip, -40
        .cfi_offset rsp, -16
        call salva_stato
        call c_activate_p
        call carica_stato
        iretq
        .cfi_endproc

        .extern c_terminate_p
```

→ Salva registri in campo contesto processo precedente

→ Carica lo stato del processo attualmente puntato da esecuzione

Ripone del punto dove il processo si era bloccato

salva_stato

```
// offset dei vari registri all'interno di des_proc
.set PUNT_NUCLEO, 8
.set CTX, 16
.set RAX, CTX+0
.set RCX, CTX+8
.set RDX, CTX+16
.set RBX, CTX+24
.set RSP, CTX+32
.set RBP, CTX+40
.set RSI, CTX+48
.set RDI, CTX+56
.set R8,  CTX+64
.set R9,  CTX+72
.set R10, CTX+80
.set R11, CTX+88
.set R12, CTX+96
.set R13, CTX+104
.set R14, CTX+112
.set R15, CTX+120
.set CR3, CTX+128
```

→ Offset all'interno di un descrittore di processo

+

```
salva_stato:
        // salviamo lo stato di un paio di registri in modo da poterli
        // temporaneamente riutilizzare. In particolare, useremo %rax come
        // registro di lavoro e %rbx come puntatore al des_proc.
        .cfi_startproc
        .cfi_def_cfa_offset 8
        pushq %rbx
        .cfi_adjust_cfa_offset 8
        .cfi_offset rbx, -16
        pushq %rax
        .cfi_adjust_cfa_offset 8
        .cfi_offset rax, -24
```

```
movq esecuzione, %rbx
movq %rbx, esecuzione_precedente
```

→ l'indirizzo è minore di 4GB e quindi non serve usare %RIP

```
// copiamo per primo il vecchio valore di %rax
movq (%rsp), %rax
movq %rax, RAX(%rbx)
// usiamo %rax come appoggio per copiare il vecchio %rbx
movq 8(%rsp), %rax
movq %rax, RBX(%rbx)
// copiamo gli altri registri
movq %rcx, RCX(%rbx)
movq %rdx, RDX(%rbx)
```

→ Copiamo i registri del processore nel descrittore

```
// salviamo il valore che %rsp aveva prima della chiamata a salva stato
// (valore corrente meno gli 8 byte che contengono l'indirizzo di
// ritorno e i 16 byte dovuti alle due push che abbiamo fatto
// all'inizio)
movq %rsp, %rax
addq $24, %rax
movq %rax, RSP(%rbx)
movq %rbp, RBP(%rbx)
movq %rsi, RSI(%rbx)
movq %rdi, RDI(%rbx)
movq %r8,  R8 (%rbx)
movq %r9,  R9 (%rbx)
movq %r10, R10(%rbx)
movq %r11, R11(%rbx)
movq %r12, R12(%rbx)
movq %r13, R13(%rbx)
movq %r14, R14(%rbx)
movq %r15, R15(%rbx)
```

→ Vogliamo salvare lo stato giallo

↓

La CALL salva l'indirizzo di ritorno e quindi %RSP punta alla cella sopra a quella del processo

un attimo dopo l'attraversamento del gate: punta il %RIP salvato dal processore

```
popq %rax
.cfi_adjust_cfa_offset -8
.cfi_restore rax
popq %rbx
.cfi_adjust_cfa_offset -8
.cfi_restore rbx

ret
.cfi_endproc
```

} Ripristina registri

carica_stato:

direttive per assemblatore: produce tabelle utili al debugger

esempio: funzione x chiamata y che chiama z (record di attivazione)

```
carica_stato:
        .cfi_startproc
        .cfi_def_offset 8
        movq esecuzione, %rbx
        popq %rcx    //ind di ritorno, va messo nella nuova pila
        .cfi_adjust_cfa_offset -8
        .cfi_register rip, rcx

        // nuovo valore per cr3
        movq CR3(%rbx), %r10
        movq %cr3, %rax
        cmpq %rax, %r10
        je 1f                   // evitiamo di invalidare il TLB
                                // se cr3 non cambia
```

→ fa il dispatch: la RET in fondo può funzionare e può riportare l'esecuzione dopo la CALL salva_stato

usiamo pila sistema

```
movq %r10, %rax
movq %rax, %cr3         // il TLB viene invalidato


// anche se abbiamo cambiato cr3 siamo sicuri che l'esecuzione prosegue
// da qui, perché ci troviamo dentro la finestra FM che è comune a
// tutti i processi
movq RSP(%rbx), %rsp    //cambiamo pila
pushq %rcx              //rimettiamo l'indirizzo di ritorno
.cfi_adjust_cfa_offset 8
.cfi_offset rip, -8
```

: carichiamo il nuovo valore di %RSP

```
// se il processo precedente era terminato o abortito la sua pila
// sistema non era stata distrutta, in modo da permettere a noi di
// continuare ad usarla. Ora che abbiamo cambiato pila possiamo
// disfarci della precedente.
cmpq $0, ultimo_terminato
je 1f
call distruggi_pila_precedente
```

→ Dobbiamo distruggere tutte le strutture dati di un processo terminato → usiamo terminate_p

→ salta ad etichetta 1 davanti: sintassi utile per le macro

```
// aggiorniamo il puntatore alla pila sistema usata dal meccanismo
// delle interruzioni
movq PUNT_NUCLEO(%rbx), %rcx
movq %rcx, tss_punt_nucleo

movq RCX(%rbx), %rcx
movq RDI(%rbx), %rdi
movq RSI(%rbx), %rsi
movq RBP(%rbx), %rbp
movq RDX(%rbx), %rdx
movq RAX(%rbx), %rax
movq R8(%rbx), %r8
movq R9(%rbx), %r9
movq R10(%rbx), %r10
movq R11(%rbx), %r11
movq R12(%rbx), %r12
movq R13(%rbx), %r13
movq R14(%rbx), %r14
movq R15(%rbx), %r15
movq RBX(%rbx), %rbx
```

Aggiorna puntatore a pila sistema nel TSS: trasferisce le 5 informazioni importanti nella pila del nuovo processo

```
movq RCX(%rbx), %rcx
movq RDI(%rbx), %rdi
movq RSI(%rbx), %rsi
movq RBP(%rbx), %rbp
movq RDX(%rbx), %rdx
movq RAX(%rbx), %rax
movq R8(%rbx), %r8
movq R9(%rbx), %r9
movq R10(%rbx), %r10
movq R11(%rbx), %r11
movq R12(%rbx), %r12
movq R13(%rbx), %r13
movq R14(%rbx), %r14
movq R15(%rbx), %r15
movq RBX(%rbx), %rbx


retq
```

caricamento registri

funzione activate_p

```
c_activate_p(void f(natq), natq a, natl prio, natl liv)
{
    des_proc *p;                    // des_proc per il nuovo processo
    natl id = 0xFFFFFFFF;           // id da restituire in caso di fallimento
    // non possiamo accettare una priorità minore di quella di dummy
    // o maggiore di quella del processo chiamante
    if (prio < MIN_PRIORITY || prio > esecuzione->precedenza) {
        flog(LOG_WARN, "priorita' non valida: %d", prio);
        c_abort_p();
        return;
    }

    // controlliamo che 'liv' contenga un valore ammesso
    // [segnalazione di E. D'Urso]
    if (liv != LIV_UTENTE && liv != LIV_SISTEMA) {
        flog(LOG_WARN, "livello non valido: %d", liv);
        c_abort_p();
        return;
    }
```

1: controllo tutti i parametri passati

: controllo priorità

distrugge processo corrente

: controllo livello

```
// non possiamo creare un processo di livello sistema mentre
// siamo a livello utente
if (liv == LIV_SISTEMA && liv_chiamante() == LIV_UTENTE) {
        flog(LOG_WARN, "errore di protezione");
        c_abort_p();
        return;
}
```

ci serve sapere il livello di privilegio di chi ha chiamato la primitiva

serve una funzione perché può variare dinamicamente → vedo il livello di provenienza della pila (CS)

```
// restituisce il livello a cui si trovava il processore al momento
// in cui è stata invocata la primitiva. Attenzione: funziona solo
// se è stata chiamata salva_stato.
int liv_chiamante()
{
        // salva_stato ha salvato il puntatore alla pila sistema
        // subito dopo l'invocazione della INT
        natq *pila = reinterpret_cast<natq*>(esecuzione->contesto[I_RSP]);
        // la seconda parola dalla cima della pila contiene il livello
        // di privilegio che aveva il processore prima della INT
        return pila[1] == SEL_CODICE_SISTEMA ? LIV_SISTEMA : LIV_UTENTE;
}
```

```
p = crea_processo(f, a, prio, liv);
```

```
if (p != nullptr) {          se fallisce
        inserimento_lista(pronti, p);
        processi++;
        id = p->id;                     // id del processo creato
                                        // (allocato da crea_processo)
        flog(LOG_INFO, "proc=%d entry=%p(%d) prio=%d liv=%d", id, f, a, prio, liv);
}
```

```
esecuzione->contesto[I_RAX] = id;
```

→ Non posso fare return id perché verrebbe salvato in %RAX che viene ripristinato

crea_processo:

```
des_proc* crea_processo(void f(natq), natq a, int prio, char liv)
{
        des_proc*       p;              // des_proc per il nuovo processo
        paddr           pila_sistema;   // pila_sistema del processo
        natq*           pl;             // pila sistema come array di natq
        natl            id;             // id del nuovo processo

        // allocazione (e azzeramento preventivo) di un des_proc
        p = new des_proc;
        if (!p)
                goto err_out;
```

→ new chiama una funzione nonostante sia fornita dalla libreria standard di C

→ Esco e restituisco un errore

```
memset(p, 0, sizeof(des_proc));
```

→ Mette a 0 l'area di memoria puntata da p per la grandezza di des_proc.

```
// rimpiamo i campi di cui conosciamo già i valori
p->precedenza = prio;
p->puntatore = nullptr;
// il registro RDI deve contenere il parametro da passare alla
// funzione f
p->contesto[I_RDI] = a;
```

→ inizializzo i campi noti di des_proc

```
// selezione di un identificatore
id = alloca_proc_id(p);
if (id == 0xFFFFFFFF)
        goto err_del_p;
p->id = id;
```

→ Errore se identificatore non trovato

goto cercando a etichette definite dopo

```
err_del_sstack: distruggi_pila(p->cr3, fin_sis_p, DIM_SYS_STACK);
err_rel_tab:    clear_root_tab(p->cr3);
                rilascia_tab(p->cr3);
err_rel_id:     rilascia_proc_id(p->id);
err_del_p:      delete p;
err_out:        return nullptr;
}
```

disfa le cose in modo inverso

```
// creazione della pila sistema
if (!crea_pila(p->cr3, fin_sis_p, DIM_SYS_STACK, LIV_SISTEMA))
        goto err_rel_tab;
// otteniamo un puntatore al fondo della pila appena creata.  Si noti
// che non possiamo accedervi tramite l'indirizzo virtuale 'fin_sis_p',
// che verrebbe tradotto seguendo l'albero del processo corrente, e non
// di quello che stiamo creando.  Per questo motivo usiamo trasforma()
// per ottenere il corrispondente indirizzo fisico. In questo modo
// accediamo alla nuova pila tramite la finestra FM.
pila_sistema = trasforma(p->cr3, fin_sis_p - DIM_PAGINA) + DIM_PAGINA;

// convertiamo a puntatore a natq, per accedervi più comodamente
pl = reinterpret_cast<natq*>(pila_sistema);
```

→ Dobbiamo simulare una INT

```
if (liv == LIV_UTENTE) {
        // inizializziamo la pila sistema.
        pl[-5] = reinterpret_cast<natq>(f); // RIP (codice utente)
        pl[-4] = SEL_CODICE_UTENTE;         // CS (codice utente)
        pl[-3] = BIT_IF;                    // RFLAGS
        pl[-2] = fin_utn_p - sizeof(natq);  // RSP
        pl[-1] = SEL_DATI_UTENTE;           // SS (pila utente)
        // eseguendo una IRET da questa situazione, il processo
        // passerà ad eseguire la prima istruzione della funzione f,
```

```
// creazione della pila utente
if (!crea_pila(p->cr3, fin_utn_p, DIM_USR_STACK, LIV_UTENTE)) {
        flog(LOG_WARN, "creazione pila utente fallita");
        goto err_del_sstack;
}

// inizialmente, il processo si trova a livello sistema, come
// se avesse eseguito una istruzione INT, con la pila sistema
// che contiene le 5 parole lunghe preparate precedentemente
p->contesto[I_RSP] = fin_sis_p - 5 * sizeof(natq);

p->livello = LIV_UTENTE;

// dal momento che usiamo traduzioni diverse per le parti sistema/private
// di tutti i processi, possiamo inizializzare p->punt_nucleo con un
// indirizzo (virtuale) uguale per tutti i processi
p->punt_nucleo = fin_sis_p;
```

Come funzionano le primitive ?

```
#include <all.h>

void mioproc(natq a)
{
        terminate_p();
}

void main()
{
        activate_p(mioproc, 10, 10, LIV_UTENTE);
        pause();
        terminate_p();
}
```

Converte questo in una CALL che non è adatta per attraversare i gate della INT. Formiamo quindi un programma Assembler che fa una INT →

```
activate_p:
        .cfi_startproc
        int $TIPO_A
        ret
        .cfi_endproc

        .global terminate_p
```

Fornite dal programma sistema per facilitare

La funzione Assembler funziona come già visto, mentre per la funzione C è importante notare come i parametri ricevuti sono quelli passati dall'utente.

```
c_activate_p(void f(natq), natq a, natl prio, natl liv)
```
→ inseriti nei registri usuali per il passaggio di parametri

Vogliamo scrivere una primitiva che restituisce la precedenza del primo processo in coda pronti

↓ non posso fare

```
void main()
{
        activate_p(mioproc, 10, 10, LIV_UTENTE);
        printf("la precedenza del primo processo in coda pronti e': %d\n", pronti->precedenza);
        pause();
        terminate_p();
}
```
sconosciuta ↓ Risolvere

```
extern des_proc *pronti;
```
sconosciuto

↓ Risolvo

```
struct des_proc {
        natw id;
        natw livello;
        natl precedenza;
        vaddr punt_nucleo;
        natq contesto[N_REG];
        paddr cr3;

        des_proc *puntatore;

        // per debugging
        void (*corpo)(natq);
        natq  parametro;
};
```
→ Il linker non può collegare perché non conosce pronti

↓ Risolvo

```
des_proc *pronti = reinterpret_cast<des_proc*>(0x210230);
```
↳ Metto l'indirizzo del file

Adesso il file utente viene compilato ma la CPU lancia la ma eseguire in quanto questa memoria è protetta da hardware

↓

È necessaria una primitiva

Passo 1. Scegliere un numero non usato

```
#define TIPO_GP                    0x28
```

Passo 2. Caricamento gate con tale tipo

```
carica_gate     TIPO_GP         a_getprec       LIV_UTENTE
```

Passo 3. Scrivere parte assembler primitiva

```
a_getprec:
        .cfi_startproc
        .cfi_def_cfa_offset 40
        .cfi_offset rip, -40
        .cfi_offset rsp, -16
        call salva_stato
        call c_getprec
        call carica_stato
        iretq
        .cfi_endproc
```

Passo 4. Scrivere la primitiva in C++ nel modello sistema

```
extern "C" void c_getprec()
{
        esecuzione->contesto[I_RAX] = pronti->precedenza;
}
```
→ dobbiamo restituirlo nel contesto

Passo 5. La inseriamo in sys.h per poter essere chiamata facilmente

```
extern "C" natl getprec();
```

Punto 6. Dichiariamo la primitiva in utente.S

```
        .global getprec
getprec:
        .cfi_startproc
        int $TIPO_GP
        ret
        .cfi_endproc
```

Punto 7. Utilizzo

```
void main()
{
        activate_p(mioproc, 10, 10, LIV_UTENTE);
        natl prec = getprec();
        printf("la precedenza del primo processo in coda pronti e': %d\n", prec);
        pause();
        terminate_p();
}
```

↓ Debugger

proces list:

```
(gdb) process list
==> Processo 0
    livello          : sistema
    corpo            : sistema:dummy(0)
    rip              :          0x202b9b <dummy(unsigned long)>
==> Processo 3
    livello          : sistema
    corpo            : io:estern_kbd(0)
    rip              :      0x1000000055d <estern_kbd(int)>
==> Processo 4
    livello          : sistema
    corpo            : io:esternAta(0)
    rip              :      0x10000000d80 <esternAta(int)>
==> Processo 5
    livello          : utente
    corpo            : utente:start(0)
    rip              : 0xffff8000000001d4 <activate_p+2>
==> Processo 6
    livello          : utente
    corpo            : utente:mioproc(10)
    rip              : 0xffff8000000000e8 <mioproc(unsigned long)>
==> Processo 7
    livello          : utente
    corpo            : utente:mioproc(11)
    rip              : 0xffff8000000000e8 <mioproc(unsigned long)>
==> Processo 8
    livello          : utente
    corpo            : utente:mioproc(12)
    rip              : 0xffff8000000000e8 <mioproc(unsigned long)>
```

Mostra i proces attivi

p proc_table:

```
$1 = {
  100010 → { id: 0, corpo: "sistema:dummy(0)", prec: 0 0x0, rax: 0 0x0 },
  null,
  null,
  100250 → { id: 3, corpo: "io:estern_kbd(0)", prec: 1104 0x450, rax: 0 0x0 },
  100310 → { id: 4, corpo: "io:esternAta(0)", prec: 1120 0x460, rax: 0 0x0 },
  100190 → { id: 5, corpo: "utente:start(0)", prec: 4294967295 0xffffffff, rax: 8 0x8 },
  1000d0 → { id: 6, corpo: "utente:mioproc(10)", prec: 10 0xa, rax: 0 0x0 },
  1003d0 → { id: 7, corpo: "utente:mioproc(11)", prec: 55 0x37, rax: 0 0x0 },
  100490 → { id: 8, corpo: "utente:mioproc(12)", prec: 5 0x5, rax: 0 0x0 },
  null <repeats 1015 times>
```

proces dump:

```
livello          : utente
corpo            : utente:start(0)
-- pila sistema (000000ffffffffd8 → 453fd8)
rip              : 0xffff800000000204 <getpr
cs               : [SEL_CODICE_UTENTE]
rflags           : [IF SF IOPL=sistema]
rsp              : 0xffffffffffffffd8
ss               : [SEL_DATI_UTENTE]
-- contesto:
rax              : 0xa
rcx              : 0x3
rdx              : 0x1
rbx              : 0x0
rsp              : 0xffffffffd8
rbp              : 0xfffffffffffffff0
rsi              : 0xe
rdi              : 0xffff8000000000e8
r8               : 0xffff800000103010
r9               : 0x0
r10              : 0x0
r11              : 0x0
r12              : 0x0
r13              : 0x0
r14              : 0x0
r15              : 0x0
cr3              : 0x0044f000
-- prossima istruzione:
file: utente/utente.s function: None
129
```

⚠ possibili ottimizzazioni: possiamo non chiamare salva_stato e carica_stato se la primitiva non modifica la variabile esecutore. possiamo in questo modo restituire un parametro in %RAX → non vale la pena ed ha poco senso ed inoltre il chiamante può trovare nei registri scratch dei valori lasciati dal sistema. Inoltre, queste sono utili in caso di errori

↓

Modifichiamo la primitiva in modo che ci restituisca la precedenza di un processo di cui forniamo noi l'identificatore

```
extern "C" natl getprec(natl id);
```

```
natl id = activate_p(mioproc, 13, 100, LIV_UTENTE);
activate_p(mioproc, 14, 1, LIV_UTENTE);
natl prec = getprec(id);
```

```
extern "C" void c_getprec(natl id)
{
        esecuzione->contesto[I_RAX] = proc_table[id]->precedenza;
}
```

→ potrebbe essere errato

↓

Otteniamo in tal caso un errore nel sistema perché in proc_table all'indirizzo c'è NULL

↓

Vogliamo prevenire questa tipologia di errori, quindi controlliamo prima il parametro

```
extern "C" void c_getprec(natl id)
{
        if (proc_table[id]) {
                natl prec = proc_table[id]->precedenza;
                esecuzione->contesto[I_RAX] = prec;
        } else {
                esecuzione->contesto[I_RAX] = (natq)-1;
        }
}
```

→ L'utente può ancora fare qualunque cosa e quindi può accedere ad un indirizzo qualsiasi in memoria purché mappabile (quindi anche fuori dalla proc_table)

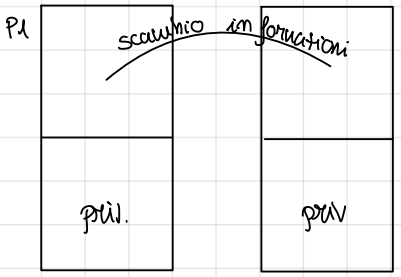
Aggiungiamo

```c
if (id >= MAX_PROC) {
        flog(LOG_WARN, "id %d non valido (max %d)", id, MAX_PROC);
        c_abort_p();
        return;
}
```

È necessaria la carica_stato in quanto esecutore viene modificata

- **SEMAFORI**

Dobbiamo prevedere che gli utenti possano organizzare i programmi in più processi. Per questo dobbiamo prevedere una **memoria condivisa** in cui i processi possono scrivere →

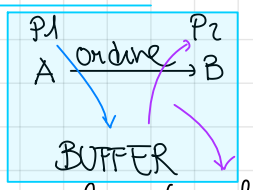


. data
. bss
. text
heap
→ Permette la condivisione di strutture dati

PILA UTENTE

Questo crea problemi di **interferenze** che in questo caso avvengono perché in modo guidato → Non possiamo dare agli utenti delle primitive per usare cli() e sti() perché non ci fidiamo dell'utente

Vogliamo quindi che ci sia un principio di **mutua esclusione** che riguarda le sezioni critiche delle strutture dati condivise. È inoltre possibile che sia richiesta invece una **sincronizzazione**:

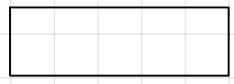


: tipico nel caso in cui un processo produce dati per l'altro

P2 deve leggere dal buffer solo dopo che P1 ci ha scritto e questo non deve sovrascrivere il buffer finché P2 non ci ha letto

↓

Semafori:



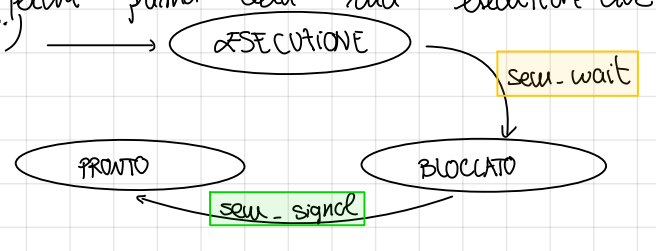
Scatola che contiene gettoni → operazioni: inserire un gettone, prelevare un gettone: STOP fino a che qualcuno non ne inserisce uno

↓

Dobbiamo stabilire il numero di scatole, i gettoni presenti inizialmente e chi può prendere e lasciare i gettoni

↓

mutua esclusione: Immaginiamo di avere un bagno in cui può andare una persona per volta. Per rendere il problema impossibile che chi deve andare in bagno deve avere un gettone e mettiamo in solo un gettone nella scatola. Ad aspettare possono avere più persone → per quanto riguarda i processi, se questi "non trovano il gettone", possono nello stato bloccato: il processo non può andare avanti e non è pronto perché prima della sua esecuzione deve verificarsi un evento (processo "mette il gettone") →



Se vi sono tanti processi bloccati si sceglie quello con precedenza più alta. La soluzione è di tipo **cooperativo**: tutti i processi devono eseguirla. È una soluzione di basso livello che è facile sbagliare. È comunque meglio di disabilitare le interruzioni in quanto gli eventuali errori sono locali al programma. Inoltre, la disabilitazione delle interruzioni impatta anche altri processi che non avevano alcuna relazione con quelli interessati alla mutua esclusione

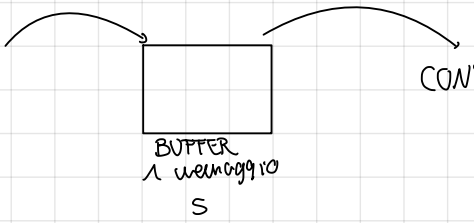
Sincronizzazione: In termini di scatola, ne prendiamo una all'inizio vuota ed A inserisce il gettone alla fine del processo e B lo prende quando deve iniziare. Funziona sempre in quanto inizialmente A ha il gettone.
Forniamo all'utente 3 primitive:

- **sem_ini (nome, v)** → blocca un nuovo semaforo con v gettoni iniziali e ne restituisce l'identificatore
- **sem_wait (id)** → prende un gettone da ID. Blocca il processo solo se la scatola è vuota
- **sem_signal (id)** → inserisce un gettone e quindi potenzialmente far passare un processo da bloccato a pronto o in esecuzione se ha priorità maggiore dei processi pronti. Il processo attualmente in esecuzione viene messo in pronti (prelemption)

↓ casi tipici

Verde: un gettone; rosso: nessun gettone

ESERCIZIO



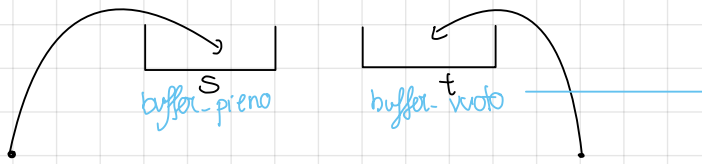
→ dobbiamo garantire che il consumatore non legga mai due volte lo stesso messaggio e che il produttore non sovrascriva il messaggio prima che venga letto dal consumatore

↓ utili zero e semafori

```
sem_wait (t)
PRODUCO
sem_signal (s)
```

```
sem_wait (s)
CONSUMO
sem_signal (t)
```

: faccio una sorta di lis DAV_ / RFD



→

```
sem_wait (b.v.)
PRODUCO
sem_signal (b.p.)
```

```
sem_wait (b.p.)
CONSUMO
sem_signal (b.v.)
```

↓ generale

Sincronizzazione: Pa A —semafori→ A

```
sem_signal (A_eseguita)
```

```
sem_wait (A_eseguita)
B
```

Pb B

mutua esclusione: P_1 P_2 .. P_n

```
sem_wait (libero)
```

A_1 A_2 A_n

```
sem_signal (libero)
```

realizzazione (struttura dati):

```
int contatore
des_proc * puntatore
```

Le primitive che utilizziamo per i semafori non devono creare problemi di mutua esclusione in quanto appunto primitive atomiche non interrompibili

```
#define TIPO_A          0x20    // activate_p
#define TIPO_T          0x21    // terminate_p
#define TIPO_SI         0x22    // sem_ini
#define TIPO_W          0x23    // sem_wait
#define TIPO_S          0x24    // sem_signal
#define TIPO_D          0x25    // delay
#define TIPO_L          0x26    // log
#define TIPO_GMI        0x27    // getmeminfo (debug)
#define TIPO_GP         0x28    // getprec
```

```
a_sem_wait:
        .cfi_startproc
        .cfi_def_cfa_offset 40
        .cfi_offset rip, -40
        .cfi_offset rsp, -16
        call salva_stato
        call c_sem_wait
        call carica_stato
        iretq
        .cfi_endproc

        .extern c_sem_signal
```

```
struct des_sem {
        int counter;
        des_proc *pointer;
};
```

→ struttura dati semaforo vero e proprio

```
// Usiamo due insiemi separati di semafori, uno per il livello utente e un
// altro per il livello sistema. I primi MAX_SEM semafori di array_dess sono
// per il livello utente, gli altri MAX_SEM sono per il livello sistema.
des_sem array_dess[MAX_SEM * 2];
```

sem_ini

```
extern "C" void c_sem_ini(int val)
{
        natl i = alloca_sem();
        if (i != 0xFFFFFFFF)
                array_dess[i].counter = val;
        esecuzione->contesto[I_RAX] = i;
}
```

non si possono distinguere

inizializza sem. i-esimo con valore utente

→ restituisce indice semaforo

```
natl alloca_sem()
{
        int liv = liv_chiamante();
        natl i;
        if (liv == LIV_UTENTE) {
                if (sem_allocati_utente >= MAX_SEM)
                        return 0xFFFFFFFF;
                i = sem_allocati_utente;
                sem_allocati_utente++;
        } else {
                if (sem_allocati_sistema >= MAX_SEM)
                        return 0xFFFFFFFF;
                i = sem_allocati_sistema + MAX_SEM;
                sem_allocati_sistema++;
        }
        return i;
}
```

distingue semafori utente e sistema

sem_wait

```
extern "C" void c_sem_wait(natl sem)
{
        // una primitiva non deve mai fidarsi dei parametri
        if (!sem_valido(sem)) {
                flog(LOG_WARN, "semaforo errato: %d", sem);
                c_abort_p();
                return;
        }

        des_sem *s = &array_dess[sem];
        s->counter--;

        if (s->counter < 0) {
                inserimento_lista(s->pointer, esecuzione);
                schedulatore();
        }
}
```

controlliamo il parametro passato dall'utente

puntatore al semaforo specificato

può diventare negativo

Non c'erano gettoni

⚠ NO cambio processo

il processo viene inserito in una lista perché deve attendere

sem_signal

```
extern "C" void c_sem_signal(natl sem)
{
        // una primitiva non deve mai fidarsi dei parametri
        if (!sem_valido(sem)) {
                flog(LOG_WARN, "semaforo errato: %d", sem);
                c_abort_p();
                return;
        }

        des_sem *s = &array_dess[sem];
        s->counter++;

        if (s->counter <= 0) {
                des_proc* lavoro = rimozione_lista(s->pointer);
                inspronti();    // preemption
                inserimento_lista(pronti, lavoro);
                schedulatore(); // preemption
        }
}
```

inseriamo un gettone

C'è qualcuno che aspetta

scegliamo quello con precedenza maggiore

Si occupano di fare l'eventuale preemption

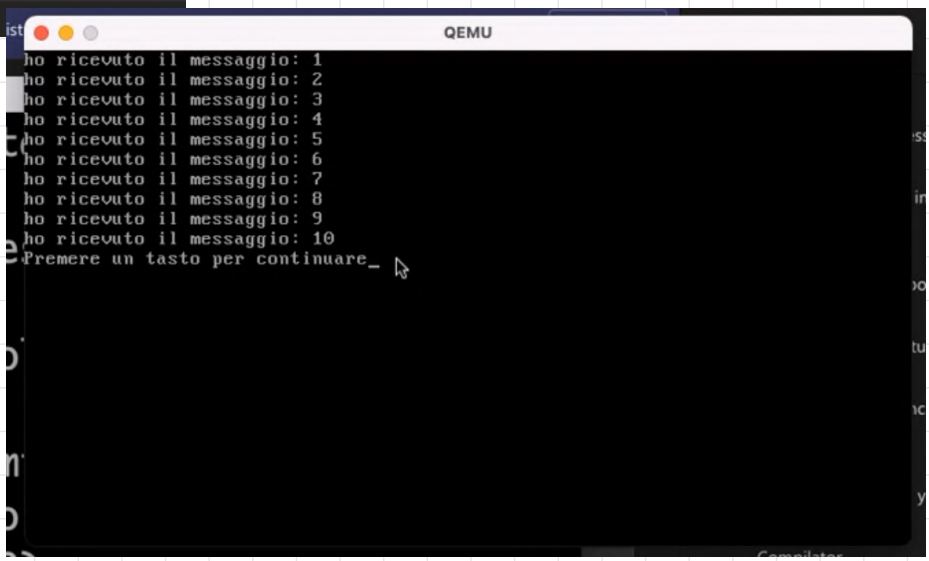
Non toglie dall'esecuzione un eventuale processo con lo stesso livello di precedenza.

ESEMPIO:

```
void prod(natq a)
{
        for (int i = 0; i < 10; i++) {
                sem_wait(buffer_vuoto);
                buffer++; // nuovo messaggio
                sem_signal(buffer_pieno);
        }
        terminate_p();
}

void cons(natq a)
{
        for (int i = 0; i < 10; i++) {
                sem_wait(buffer_pieno);
                printf("ho ricevuto il messaggio: %d\n", buffer);
                sem_signal(buffer_vuoto);
        }
        pause();
        terminate_p();
}
```

```
void main()
{
        buffer_pieno = sem_ini(0);
        buffer_vuoto = sem_ini(1);
        activate_p(prod, 0, 10, LIV_UTENTE);
        activate_p(cons, 0, 20, LIV_UTENTE);
        terminate_p();
}
```

} funziona anche se cambiamo la priorità



debugger:

```
processi:   2
esecuzione: [6, 10]
pronti:     [0, DUMMY]
sem[    1]: { -1, [7, 20] }
p_sospesi:
```

→ Code di ciascun semaforo

delay: sospende un processo per un certo tempo

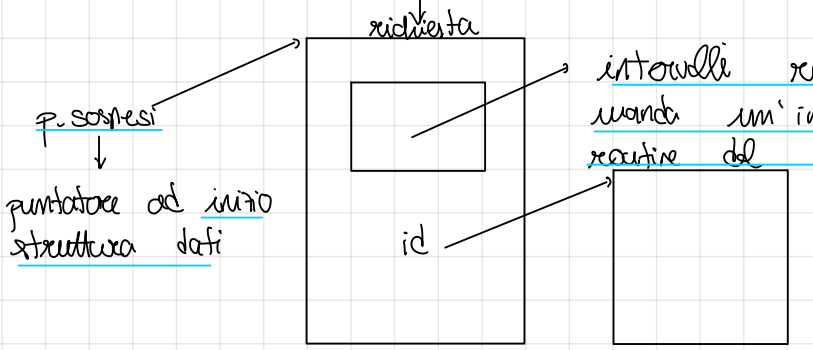
```
#include <all.h>

void main()
{
        for (int i = 0; i < 10; i++) {
                printf("hello, world\n");
                delay(100);
        }
        pause();
        terminate_p();
}
```

100 è definito in intervalli di tempo definiti altrimenti il timer è l'evento da attendere: passaggio del tempo

il processo viene BLOCCATO →

Nel mentre può andare in esecuzione altro

↓ Coda in struttura dati



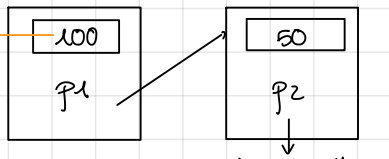
intervalli rimanenti: periodicamente il timer manda un'interruzione che fa partire una routine del kernel

↓

esamina la coda e vede quali processi vanno in coda pronti

↓ ottimizziamo

Memorizziamo quanti intervalli in più mancano rispetto al precedente



deve aspettare 150

Se arriva un processo che richiede di aspettare meno di quelli presenti questo viene messo in testa e viene aggiornato il contatore del processo che è scalato in seconda posizione. Quando un processo chiama delay esso deve capire dove va messo facendo lo scorrimento della lista. Se la somma dei contatori è < del numero di intervalli richiesti si continua a scorrere, altrimenti viene inserito ed aggiornato il contatore del processo successivo. Quando si scorre, il contatore del processo inserito viene sempre decrementato sottraendo ogni volta il contatore di ciascun processo "attraversato"

↓

Il tutto è facilitato in quanto deve decrementare solo il primo contatore ed eventualmente inserire in lista pronti estraendo dalla testa

```
// la priorità massima è riservata al driver del timer di sistema
carica_gate     TIPO_TIMER      driver_td       LIV_SISTEMA
```

→ Routine che deve essere eseguita periodicamente (interruzione esterna)

↓

```
        .extern c_driver_td
driver_td:
        .cfi_startproc
        .cfi_def_cfa_offset 40
        .cfi_offset rip, -40
        .cfi_offset rsp, -16
        call salva_stato
        call c_driver_td        (*)
        call apic_send_EOI
        call carica_stato
        iretq
        .cfi_endproc
```

Deve cambiare in esecuzione il processo con maggiore priorità che deve quindi eventualmente essere cambiato (preemption)

↓

```
a_delay:
        .cfi_startproc
        .cfi_def_cfa_offset 40
        .cfi_offset rip, -40
        .cfi_offset rsp, -16
        call salva_stato
        call c_delay ──────────→
        call carica_stato
        iretq
        .cfi_endproc
```

```
extern "C" void c_delay(natl n)
{
    richiesta *p;

    p = new richiesta;
    p->d_attesa = n;
    p->pp = esecuzione;

    inserimento_lista_attesa(p);
    schedulatore();
}
```

```
struct richiesta {
        natl d_attesa;
        richiesta *p_rich;
        des_proc *pp;
};
```

→ Algoritmo visto prima

↓

```
void inserimento_lista_attesa(richiesta *p)
{
        richiesta *r, *precedente;

        r = p_sospesi;
        precedente = nullptr;

        while (r != nullptr && p->d_attesa > r->d_attesa) {
                p->d_attesa -= r->d_attesa;
                precedente = r;
                r = r->p_rich;
        }
        p->p_rich = r;
        if (precedente != nullptr)
                precedente->p_rich = p;
        else
                p_sospesi = p;

        if (r != nullptr)
                r->d_attesa -= p->d_attesa;
}
```

} → r punta a successivo e precedente a precedente all'uscita

→ inserimento in testa

→ aggiorna contatore elemento successivo

(*)

```
extern "C" void c_driver_td(void)
{
        inspronti();

        if (p_sospesi != nullptr) {
                p_sospesi->d_attesa--;
        }

        while (p_sospesi != nullptr && p_sospesi->d_attesa == 0) {
                inserimento_lista(pronti, p_sospesi->pp);
                richiesta *p = p_sospesi;
                p_sospesi = p_sospesi->p_rich;
                delete p;
        }

        schedulatore();
}
```

decrementa contatore ad ogni interrupt

inserisce quelli con contatore 0 in cima a coda

punta a nuovo primo elemento : funzione standard su lista

Esempio

```
esecuzione:  [6, 10]
pronti:      [0, DUMMY]
p_sospesi:   {100, [5, MAX_PRIO]}
```

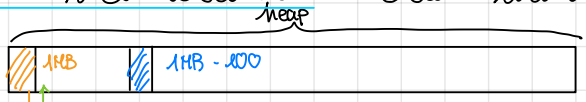
introducendo il timer abbiamo il **non determinismo**: le interruzioni del timer sono imprevedibili ⟶ i programmi devono essere corretti qualunque sia la sua dilatazione

⚠ NOTA: new e delete ⟶ funzioni di libreria del C++

```
void* operator new(size_t size)
{
        return alloca(size);
}
```

```
void operator delete(void *p)
{
        dealloca(p);
}
```

funzionano in quanto all'inizializzazione del sistema si inizializza lo heap (1 MB) ⟶ le informazioni sull'allocazione della memoria vengono scritte nello heap stesso:



info memoria : spazio consecutivo in memoria disponibile

alloca (100) : nuovo descrittore e puntatore a zona di memoria allocata (inizio)

dealloca (p) : dal puntatore si ottengono le informazioni necessarie dal descrittore semplicemente facendo una sottrazione

⟵ le parti libere sono contenute in una lista di cui alloca fa lo scorrimento fino a trovare una sufficientemente grande.

La paginazione

Stato di un processo = stato CPU + stato memoria

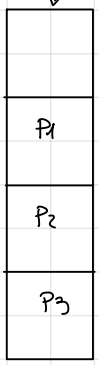
il cambio di processo implica il cambio della memoria privata del processo uscente con quella del processo entrante

→ prima soluzione rudimentale

↓

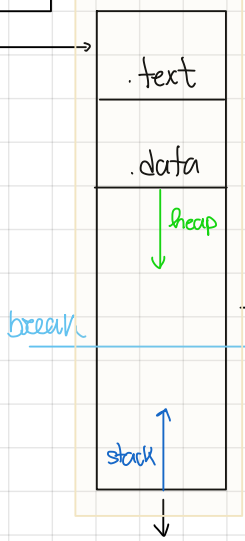
costo insostenibile

vogliamo



→ Più processi in memoria: evito accesso ad HDD

Memoria di un processo →



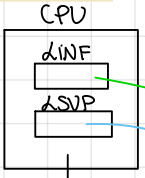
→ ⚠ Pericolo di collisione: non c'è una soluzione efficace perché le sezioni, crescendo, possono oltrepassare il break

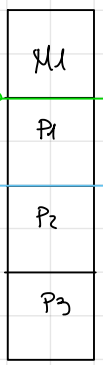
Allocata con dimensione uguale

⚠ Che problema c'è ad avere più processi in memoria? Un processo può scrivere in una parte di memoria appartenente ad un altro processo

↓ Necessario più **hardware**: soluzioni

1. registri aggiuntivi →





→ Viene modificata la CPU perché può controllare processo attualmente in esecuzione e vedere se rispetta i "confini" della sua memoria

Contenuto non modificabile dall'utente dentro descrittore di processo: cambiato al cambio di processo

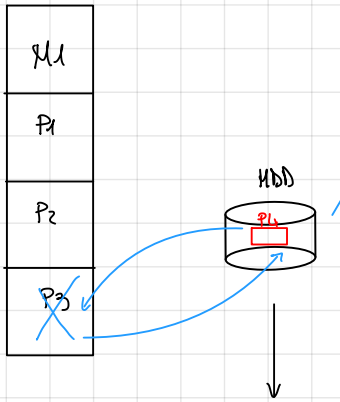
Scomodo: gli indirizzi a cui vengono allocati i processi vengono decisi al momento della loro creazione

rilocazione

1. Caricatore rilocante: usa le informazioni di rilocazione del linker per aggiustare

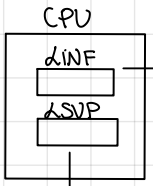
gli indirizzi al momento del caricamento del programma
2. Scrivere tutto il programma con indirizzi relativi (non dipendenti da posizione)

⚠ Problema se abbiamo più processi che memoria



→ tratta la memoria utente come una cache copiando in HDD il processo partizionato
↓
Molto meno frequente di farlo ogni sostituzione

Se ora abbiamo in memoria P4 e ci libera spazio per P3, non possiamo ricaricarlo da un'altra parte perché, ad esempio, gli indirizzi caricati del processo in pila sono riferiti alla posizione precedente del processo in memoria e non possono essere riaggiustati. → Dobbiamo quindi ricaricare i processi allo **stesso indirizzo** da cui sono stati tolti
↓ Modifica



→ interpretato come **base**
↓
Ogni volta che c'è un accesso in memoria, l'indirizzo generato dal programma in esecuzione viene sommato alla base prima di accedere alla RAM
↓
Pi: a ⟶ base + a ⟶ RAM (rimane)

Limite superiore

Siamo tutti gli indirizzi sono relativi alla base, quindi basta modificare quella: si mette in punto della RAM dove c'è spazio
↓
soluzione trasparente: il programmatore può collegare sempre il suo programma all'indirizzo 0 e non deve renderlo indipendente dalla posizione (perché la base viene sommata automaticamente)

⚠ PROGRAMMAZIONE = decidere cosa c'è in memoria inizialmente: lo posso fare indipendentemente dalla base (che può essere qualsiasi). La RAM vede **sempre** l'indirizzo risultante della **somma** dell'indirizzo specificato dal programma con la base → il processo ha quindi un proprio spazio riservato e non ci interessa sapere quale è perché i suoi indirizzi vanno da 0 al massimo: **memoria virtuale** indipendente dagli altri processi
↓
La rilocazione viene fatta durante l'esecuzione del programma dalla CPU.

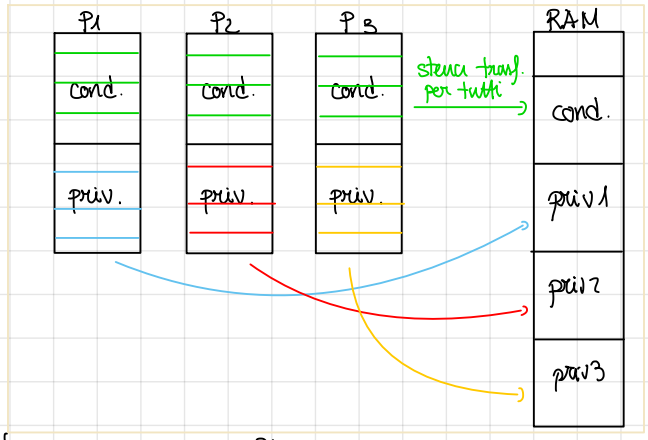
Difetto: la trasformazione è poco flessibile. Se dei processi terminano, i processi adiacenti non vengono modificati e si produce **frammentazione** (RAM libera sparsa a tratti), che può portare ad avere processi in attesa nonostante lo spazio libero totale in RAM sia sufficiente → soluzione costosa: Ricompattamento attraverso lo spostamento dei processi

Difetto 2: la soluzione permette di isolare bene i processi ma questo rende difficile farli

comunicare attraverso le sezioni a comune.

↓ Paginazione

Permette di comporre la memoria di un processo in pezzi più piccoli e ciascuno di essi metterlo dove voglio senza che il processo se ne accorga. È possibile fare questo per ciascuna sezione ovunque ci sia spazio. Risolve il problema della frammentazione e della condivisione



↓

le porzioni singole sono chiamate **pagine**

Spazio di indirizzamento del processo diviso in regioni notevoli della stessa grandezza (4 KiB per Intel) ⟶ **pagine**

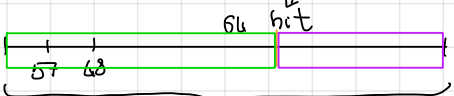
Spazio di indirizzamento fisico

⟶ Frame (cornice): 4 KiB

↓

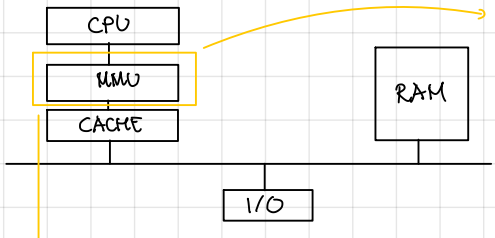
A ciascuna pagina può essere associato un frame (o nessuno) e a ciascun frame possono essere associate più pagine

trasformazione



L'**indirizzo virtuale** ancora da trasformare può essere diviso in 2. La parte più significativa è il numero di pagina mentre la meno significativa è l'offset all'interno della pagina, che su 4 KiB è grande 12 bit. Una pagina può essere **mappata** su qualunque frame sostituendo il numero di pagina con il numero di frame e lasciando l'offset invariato (corrispondenza 1:1) con una **funzione di trasformazione** ⟶ ricerca non gestibile dal sistema ed applicabile dall'hardware: struttura dati esempio ⟶ a[n. pagina] = n. frame

↓ nuovo dispositivo: **memory management unit**



tutti gli indirizzi generati dalla CPU vengono tradotti dalla MMU e quindi tutti gli indirizzi della CPU sono indirizzi virtuali. Dopo la MMU ci sono gli indirizzi fisici

Deve esserci una tabella per ogni processo. Questa aiuta ad indicare perché il processo può accedere solo agli indirizzi facenti parte del dominio della funzione. Il sistema inoltre permette una condivisione selettiva applicando la stessa trasformazione per più indirizzi virtuali. Viene inoltre risolto il problema della frammentazione potendo disporre le pagine liberamente.
La MMU inoltre li traduce in ogni accesso in memoria e quindi può essere utile per altri scopi:

nº pagina —associati→ traduzione (nº frame)

- P: specifica se traduzione esiste o no → il processo non può accedere ad alcune pagine, senza eccezione: utile per puntatori nulli per restituire errore e per proteggere lo stack e lo heap attraverso l'inserimento di guard pages che li intervallano
- R/W: protegge le pagine dalla scrittura (.text è R/O). Basilare di proteggere il programma e di condividere sezioni
- U/S: livello necessario per accedere a pagina: proteggere dati sistema perché alcune info di sistema devono essere mappate per permettere al processore di eseguire alcune funzioni (interconnessioni). Per ottimizzare consideriamo tutta la memoria mappata

PCD
PWT
A/D

Il bit PCD permette di disabilitare la cache e quindi far puntare l'indirizzo sul BUS direttamente. Il bit PWT (page write through) abilita la modalità write through per la scrittura in cache. Ha senso disabilitare la cache per l'I/O ed impostare la modalità WT per la memoria video. I bit A/D sono aggiornati dalla MMU stessa → A (access): 1 se viene fatta traduzione per quel numero di pagina

D: 1 se l'accesso è fatto in scrittura

↓

Permettono di realizzare statistiche da parte del sistema

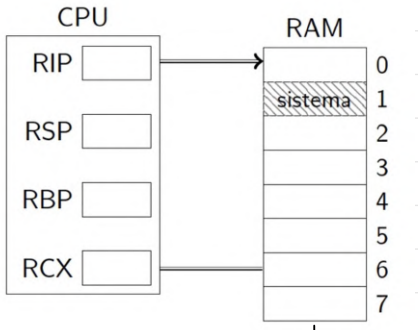
ESEMPIO di programma con MMU attiva

```
char buf[0x2000] = { 2, 6, -1, 200, ...,
  15, 3, -32, 1};
int main()
{
  int sum = 0;
  for (int i = 0; i < 0x2000; i++)
    sum += buf[i];
  return sum;
}
```

Semplificazioni

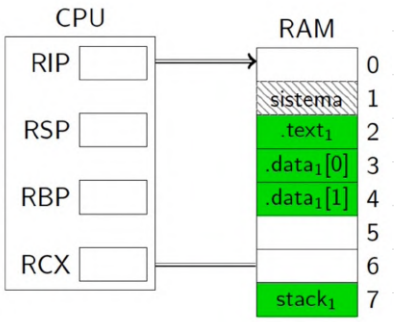
1. Spazio di memoria = 32 KiB quindi indirizzi da 0000 a 7fff
2. Spazio diviso in 8 pagine da 4 KiB
3. Pagine 0 e 1 riservate al sistema
4. Codice nella pagina 2, variabile buf nelle pagine 3 e 4 e la pila nella pagina 7

```
                .text
                .global main
2000  main:     pushq %rbp              55
2001            movq %rsp, %rbp         48 89 e5
2004            subq $8, %rsp           48 83 ec 08
2008            movl $0, -8(%rbp)       c7 45 f8 00 00 00 00
200f            movl $0, -4(%rbp)       c7 45 fc 00 00 00 00
2016  for:      cmpq $0x2000, -4(%rbp)  48 81 7d fc 00 20 00 00
2013            jge fine                7d 14
2020            movslq -4(%rbp), %rcx   48 63 4d fc
2024            movsbl buf(%rcx), %eax  0f be 81 00 30 00 00
202b            addl %eax, -8(%rbp)     01 45 f8
202e            addl $1, -4(%rbp)       83 45 fc 01
2032            jmp for                 eb e2
2034  fine:     movl -8(%rbp), %eax     8b 45 f8
2037            popq %rbp               5d
2038            ret                     c3
                .data                   ...
3000  buf:      .byte 2, 6, -1, 200     02 06 ff c8
                ...                     ...
4ffc            .byte 15, 3, -32, 1     0f 03 e0 01
```

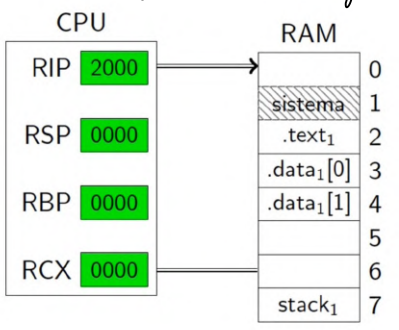


prima che il programma parta solo sistema

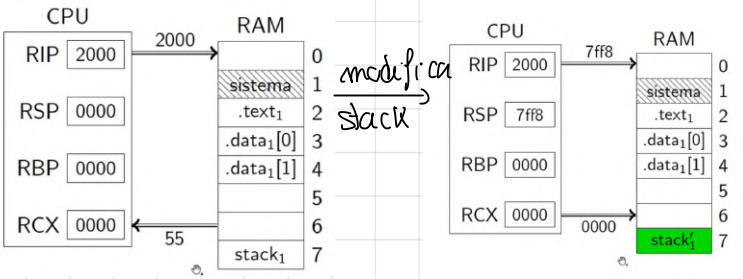
↓ dopo caricamento



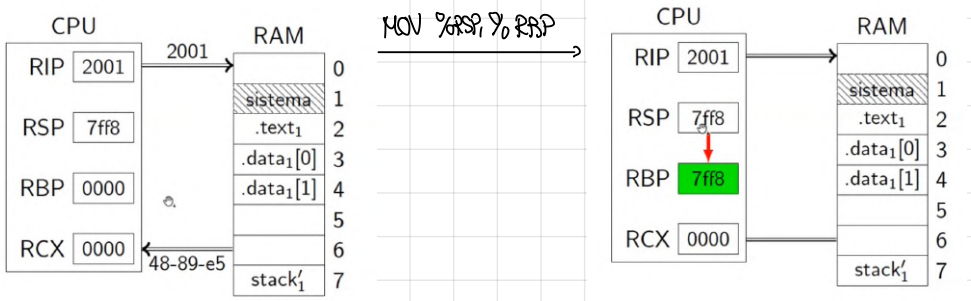
con inizializzazione registri



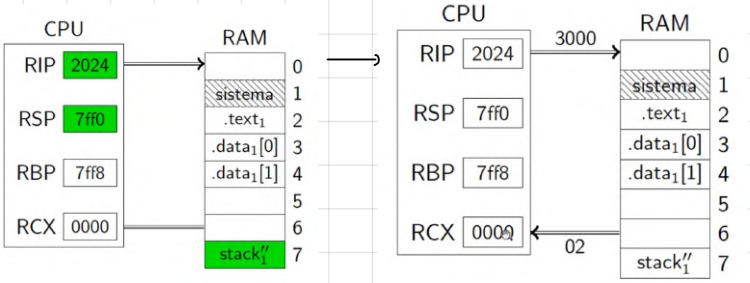
Passo 1



Passo 2

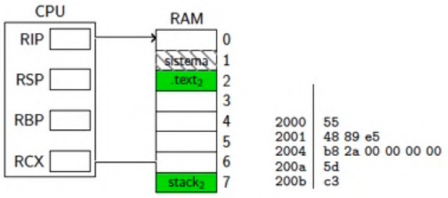


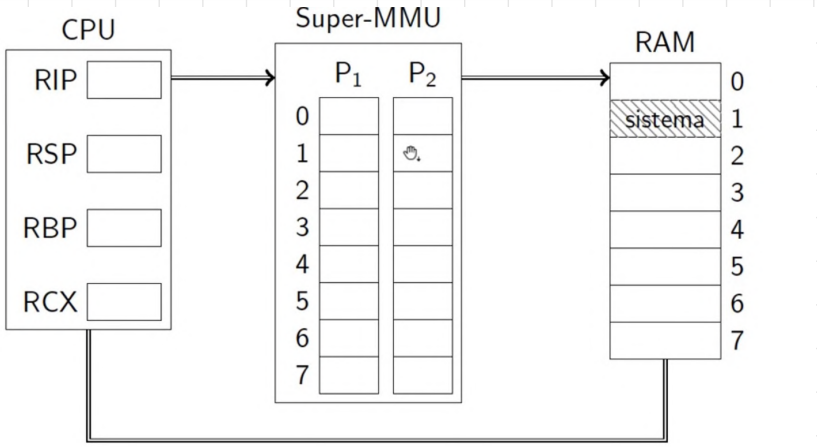
Passo 24



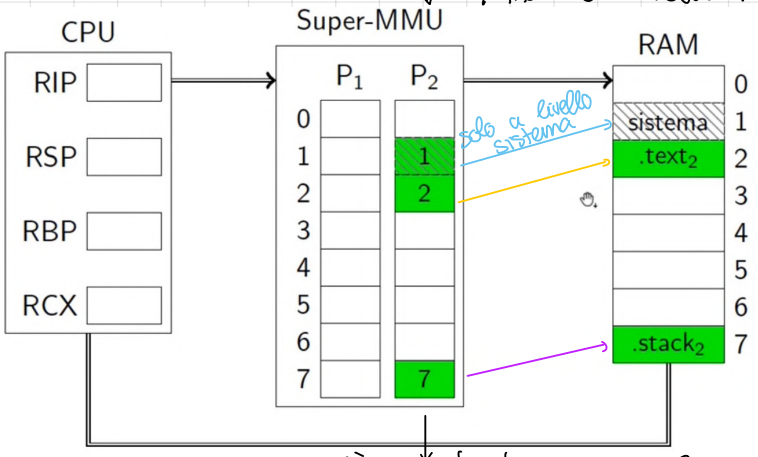
Il processo non sa che ci sono altri processi che condividono con lui la RAM

↓ secondo processo



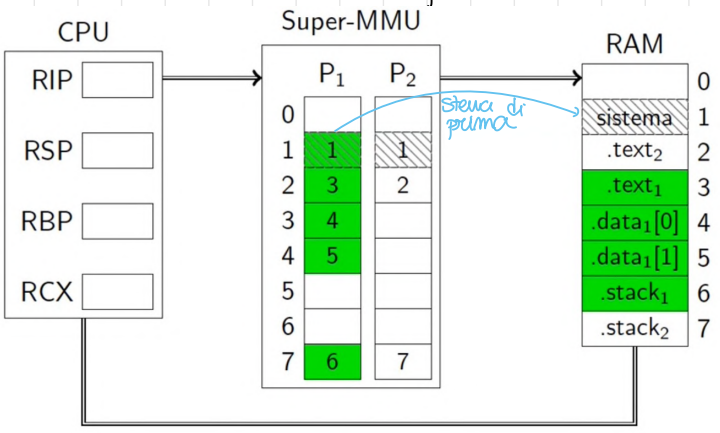


↓ Atto di caricamento

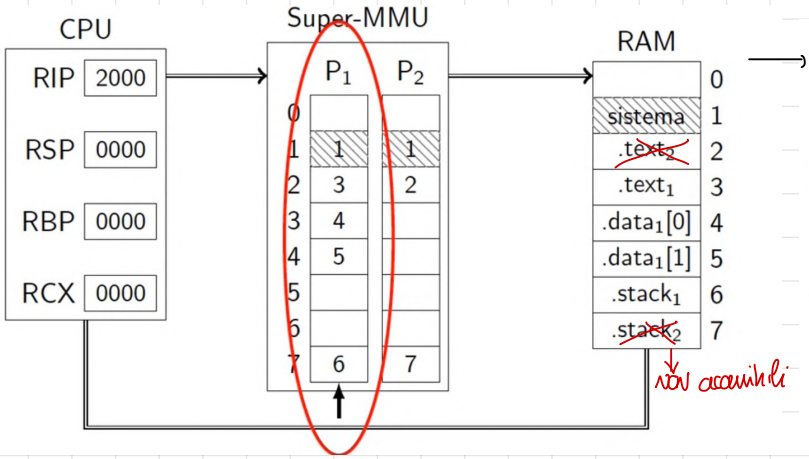


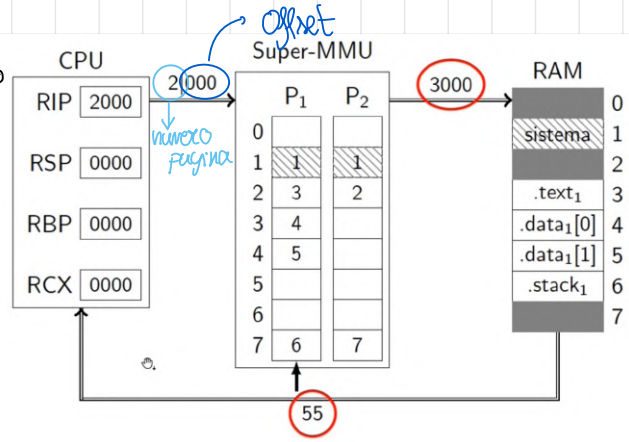
è importante ripetere la corrispondenza delle tabelle

Caricando il secondo processo

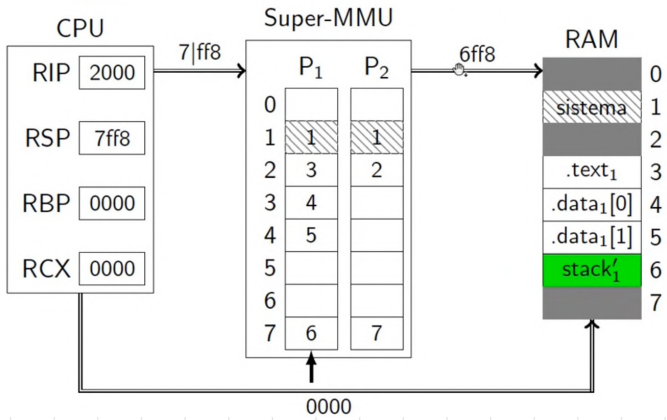


Esecuzione del programma ⟶ deve rimanere invariata



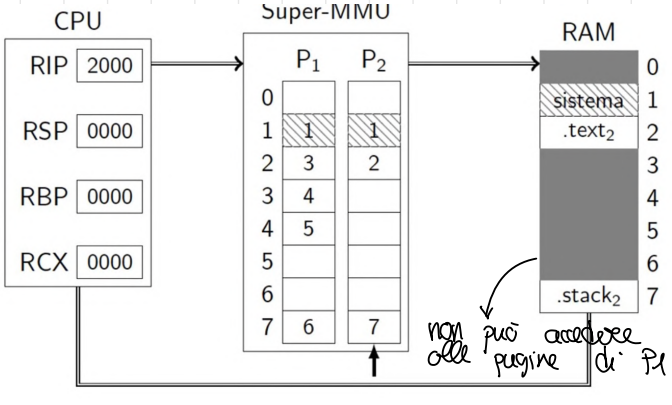


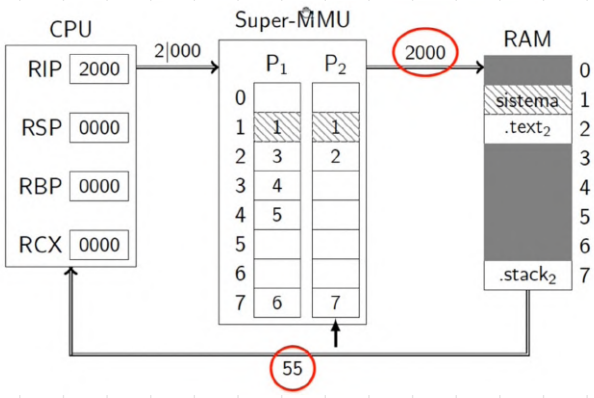
⟶



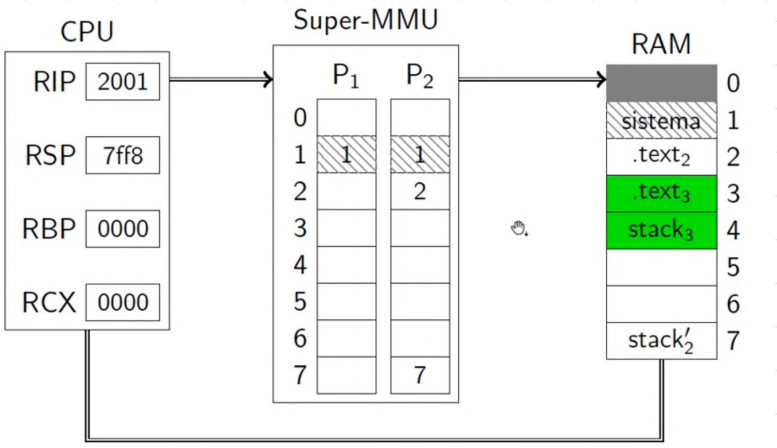
Stesso funzionamento di prima (anche per passaggi successivi)

Supponiamo che ci sia un'interruzione che fa cambiare processo





Supponiamo che P2 venga tolto



E che, dopo la terminazione di P2 si voglia reiniziare P2: alcuni frame sarai liberi sono occupati → non importa: possono essere riallocati

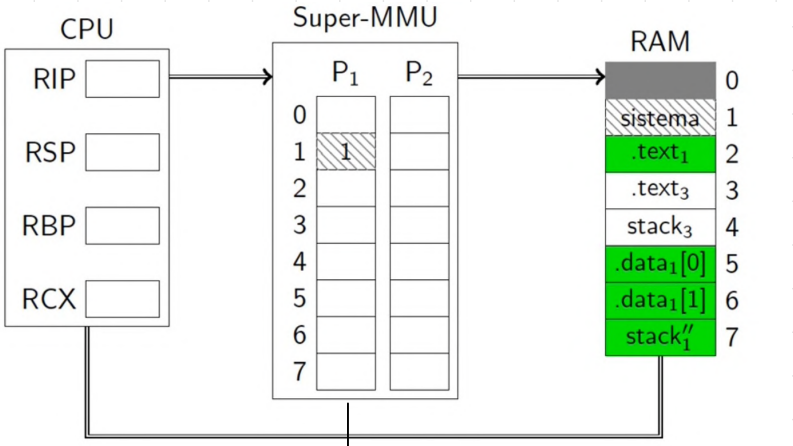
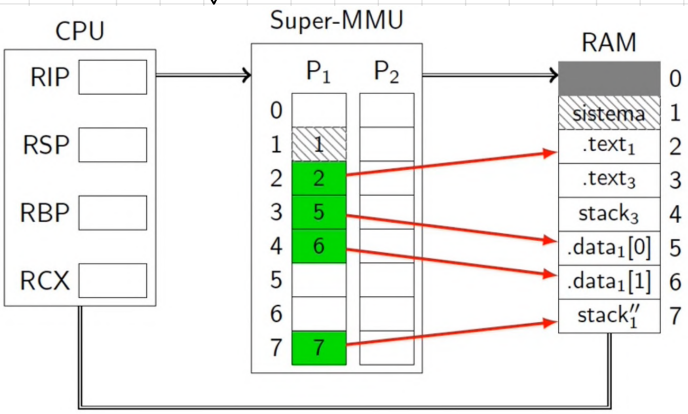
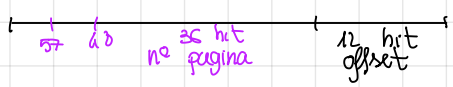


tabella aggiornata



: P1 non si accorge di nulla e quindi continua l'esecuzione normale

La super MMU non può essere realizzata in questo modo: Quanto è grande la tabella di corrispondenza?



→ la tabella, nel nostro caso l'array, deve crescere ogni frame ad ogni numero di pagina

2^{36} righe che devono contenere il numero di frame che può arrivare a 52 bit e gli altri bit → arrotondando, ci servono 8 byte per riga

$2^{36} \cdot 2^3 = 2^{39} = 512$ GiB: non è possibile

Utilizziamo una nuova struttura dati: TRIE → permette di associare chiavi e valori

Esempio
KEY: parole inglesi
VALUES: traduzione in italiano

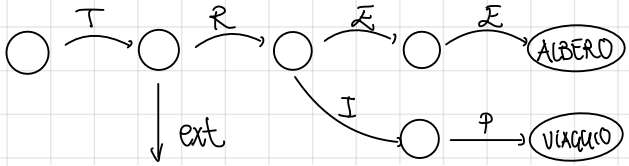
↓

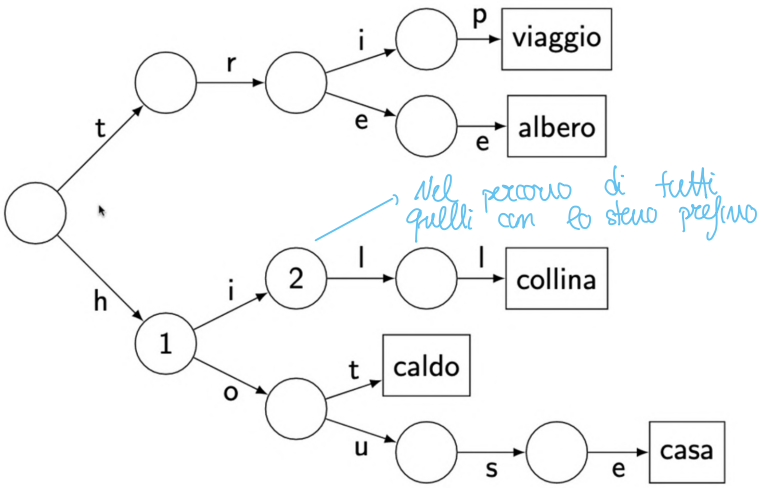
Albero nei cui archi ci sono le singole lettere della chiave che guidano il percorso e nei nodi abbiamo i valori

↓ Esempio

Tree ⟶ Albero:

Trip ⟶ Viaggio





Nel percorso di tutti quelli con lo stesso prefisso

⟶ La chiave non è memorizzata in nessun nodo ma è lungo il percorso

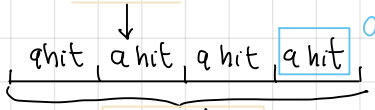
↓

accesso di puntatori in questo caso indirizzati da codice ASCII

✓: le chiavi assenti non occupano spazio

Variazione: sui rami ci sono blocchi hit della chiave

↓



36 hit
nº pagina

analogo alle lettere di prima: in base 8

↓

in ogni nodo c'è una tabella con 512 entrate che punta ad un'altra tabella. Nell'ultima tabella si trova la traduzione (nº di frame)

↓

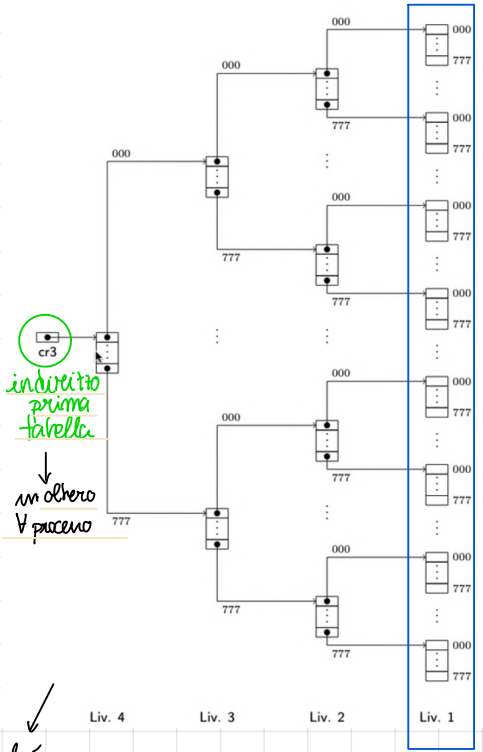


tabella di corrispondenza da 512 GiB

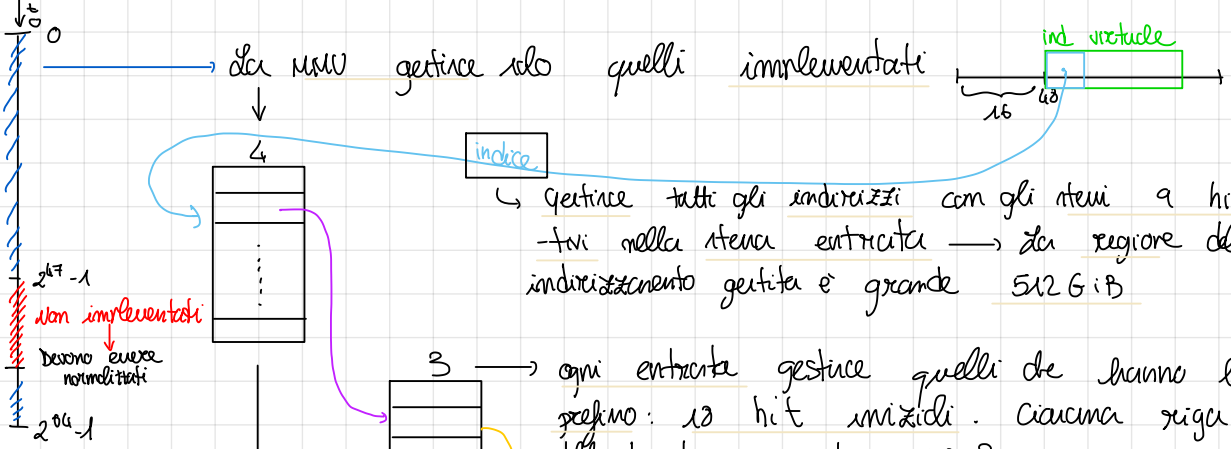
↓

Essendo che anche le altre colonne occupano spazio, abbiamo più spazio occupato rispetto all'accesso.

↓

La struttura dati ci aiuta perché se un processo non usa alcuni indirizzi è possibile non ricostruire quella parte dell'albero

Studio Cicconardi



↳ gestisce tutti gli indirizzi con gli stessi 9 bit più significativi nella stessa entrata → la regione dello spazio di indirizzamento gestita è grande 512 GiB

→ ogni entrata gestisce quelli che hanno lo stesso prefisso: 18 bit iniziali. Ciascuna riga si occupa della traduzione di 1 GiB

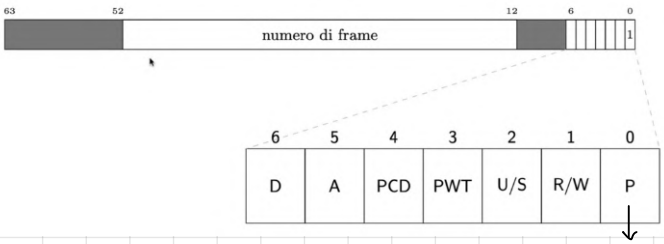
La granularità delle regioni permette di non allocare molti sottoalberi in caso di piccoli gigli che usano la stessa sezione di memoria

→ 2MB

Per pezzi più grandi possono usare pagine più grandi. Ad esempio, usiamo pagine da 2MB utilizzando offset da 12 byte e facendo scomparire un livello. L'organizzazione permette inoltre di non avere tutta la struttura in memoria

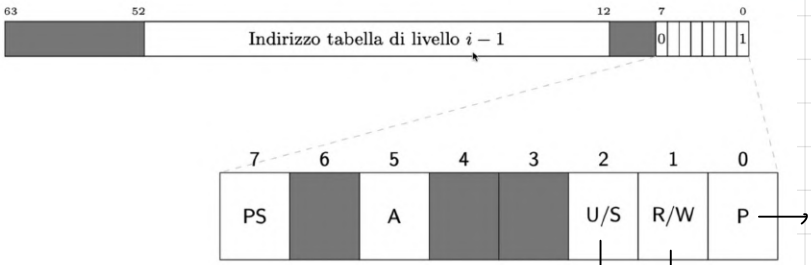
→ 4 KiB: una pagina

↓ Formato



Può non essere valida: in tal caso la MMU si disinteressa della pagina che può essere utilizzata per altri scopi dal programmatore

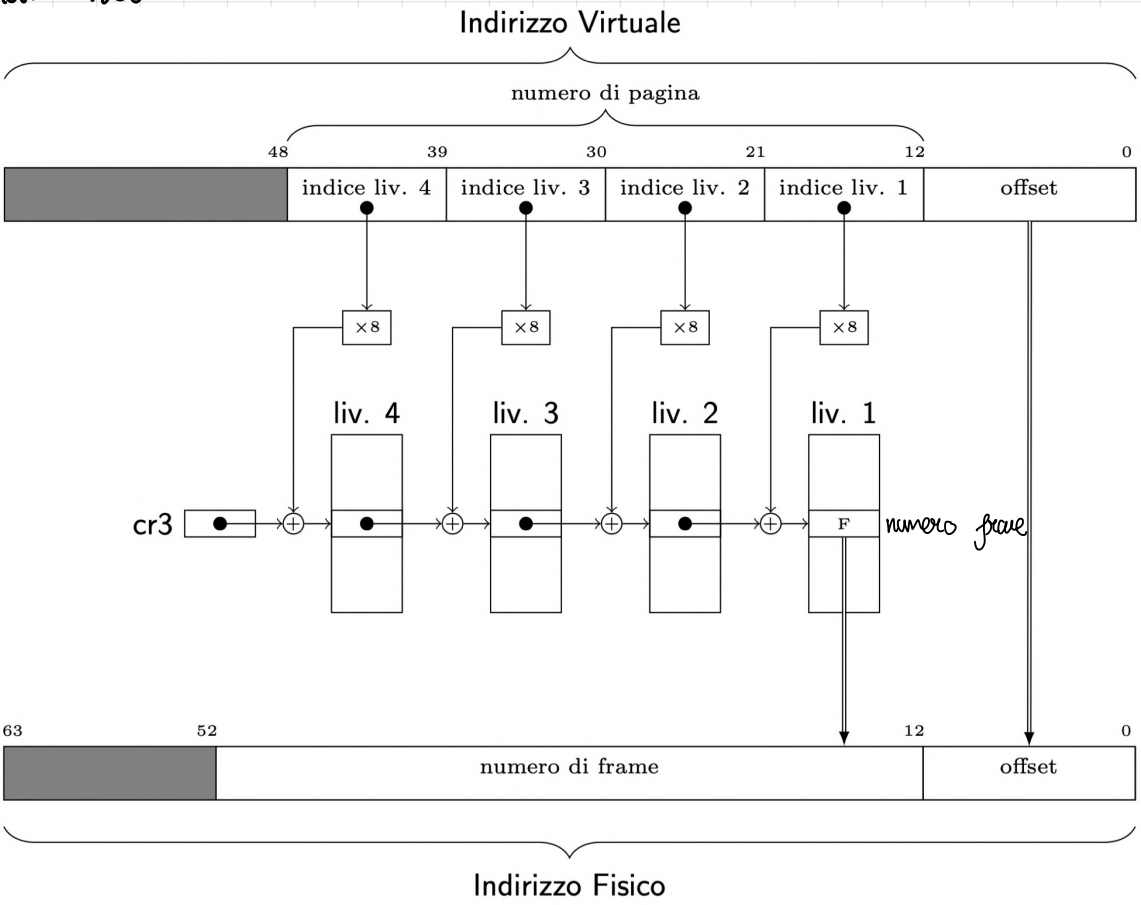
≠ da tab intermedie



→ P=0 se manca la traduzione della pagina: interrompe la traduzione

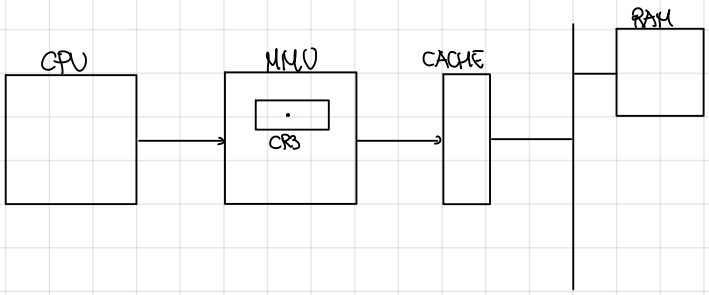
↓ se permesso di scrittura deve essere concesso da tutti i bit stessa cosa di R/W

Riavvendo



→ Arriva alla CPU

Avviciniamoci alla MMU reale

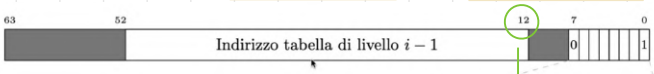


: contiene le tabelle → comodo perché create dal sistema e accessibili senza interventi speciali. La MMU va a dov'è la tabella di livello 4. Per sapere dove sono le altre fa accessi in memoria

↓

La MMU deve sapere l'**indirizzo fisico** della tabella di livello 4. Anche questa contiene l'indirizzo fisico della tabella di livello 3 e così via

ricordiamo le righe di tali tabelle :



Mancano bit meno significativi: limiti posizionamento in RAM → Avendo 12 zeri meno significativi le tabelle devono essere allineate naturalmente (a 4KiB)

Inoltre, gli accessi in memoria diventano troppi → introduciamo una cache della MMU: efficace perché associa al numero di pagina direttamente F saltando i passaggi intermedi

Creazione di un albero di traduzione

```c
#include <libce.h>

void main()
{
        pause();
}
```

→

```
                                                        [protezione]
modo: sistema  intr: abilitati     io: permesso  cr3: 0x00001000
```

contenuto corrente registro cr3: puntatore alla radice struttura di traduzione ~~fissiamo~~ fare traduzione identità : numero frame = numero pagina

↓ vm maps: riassunto albero di traduzione

```
(gdb) vm maps                                   Flag settati
0000000000000000  S-000-000-000-000: U W PWT PS A D
0000000000200000  S-000-000-001-000: U W PS A D
0000000000400000  S-000-000-002-000: U W PS
0000000040000000  S-000-001-000-000: U W PWT PCD PS
00000000fec00000  S-000-003-766-000: U W PWT PCD PS A D
00000000fee00000  S-000-003-767-000: U W PWT PCD PS
0000000100000000  S-000-004-000-000: unmapped
```

da
a (non compreso)

2^{32} : Traduzioni per 4GB

+

```
(gdb) vm path 0 → percorso traduzione
cr3: 0x00001000 : partenza indirizzo 0
000: WU--A--  flag 0x00002000 : livello 3
        000: WU--A--  → 0x00003000 :
                000: WUw-ADs  → 0x00000000
```

implementiamo traduzione identità per 8MB

```
.data
.align 4096
```

→ Allineamento naturale a 4 byte

↓ 8 MB

da 0000 0000 a 0008 0000

offset

Scomposti in gruppi di 3 cifre : utilizziamo la base 8, quindi il numero di pagina sono 12 cifre ottali mentre l'offset è dato da 4 cifre ottali

↓

	4	3	2	1	offset
in base 8 :	000	000	000	000	0000 : da
	000	000	004	000	0000 : a → primo non tradotto

↓ tabella di livello 4

Avendo tutti 0, passano tutti da una sola entrata, il che implica la presenza di una sola tabella di livello 3. Anche in questa tabella tutti gli indirizzi passano dalla prima entrata e quindi ci sarà un'unica tabella di livello 2. Nel caso di questa tabella avremo bisogno di 4 entrate (000, 001, 002, 003).

```
tab_4
0 ->   tab_3
       0 -> tab_2
              0 -> tab_1_0
              1 -> tab_1_1
              2 -> tab_1_2
              3 -> tab_1_3
```

→

```
.data
.align 4096
tab_4:
        .space 4096, 0
tab_3:
        .space 4096, 0
tab_2:
        .space 4096, 0
tab_1_0:
        .space 4096, 0
tab_1_1:
        .space 4096, 0
tab_1_2:
        .space 4096, 0
tab_1_3:
        .space 4096, 0
```

inizializzazione delle tabelle

Sintassi righe

tab 4

```
init_vm:
        movq $tab_3, tab_4
        movb 0b00000111, SWP
```

→ Posso scrivere $tab_3 perché bootloader fa traduzione 1:1
→ setto hit

tab 3

```
movq $tab_2, tab_3
movb $0b00000111, tab_3
```

tab 2

```
movq $tab_1_0, tab_2
movb $0b00000111, tab_2

movq $tab_1_1, tab_2 + 8
movb $0b00000111, tab_2 + 8

movq $tab_1_2, tab_2 + 8 * 2
movb $0b00000111, tab_2 + 8 * 2
movq $tab_1_3, tab_2 + 8 * 3
movb $0b00000111, tab_2 + 8 * 3
```

grandezza riga tab 2

tab 1 : contiene le traduzioni (ricordiamo, traduzione identità)

```
mov $0, tab_1_0
movb $0b00000111, tab_1_0
```

→ Alla tabella si accede con indice 0

```
mov $4096, tab_1_0 + 8
```

: seconda riga

Scrivo 1 seguito da 12 zeri

```
mov $(2 * 4096), tab_1_0 + 8 * 2
```

: terza riga

2 seguito da 12 zeri

per crearle tutte insieme

```
        movq $tab_1_0, %rdi
        xorq %rax, %rax
.Loop:
        movq %rax, (%rdi)
        movb $0b00000111, (%rdi)
        addq $0x1000, %rax
        addq $8, %rdi
        cmp $last, %rdi
        jb .Loop

        ret
```

→ faccio le cose di prima per tutte le tabelle

```
#include <libce.h>

extern "C" void init_vm();
void main()
{
        init_vm();
        pause();
}
```

Albero generato

```
(gdb) vm tree &tab_4
0x00204000
000: WU+----  → 0x00205000
          000: WU-----  → 0x00206000
                    000: WU-----  → 0x00207000
                              000: WU-----  → 0x00000000
                              001: WU-----  → 0x00001000
                              002: WU-----  → 0x00002000
                              003: WU-----  → 0x00003000
                              004: WU-----  → 0x00004000
                              005: WU-----  → 0x00005000
                              006: WU-----  → 0x00006000
                              007: WU-----  → 0x00007000
                              010: WU-----  → 0x00008000
                              011: WU-----  → 0x00009000
                              012: WU-----  → 0x0000a000
                              013: WU-----  → 0x0000b000
                              014: WU-----  → 0x0000c000
```

Per far sì che questa traduzione venga usata scriviamo l'indirizzo di tab4 in cr3

```
// carica il registro cr3
// parametri: indirizzo fisico del nuovo direttorio
        .global loadCR3
loadCR3:
        .cfi_startproc
        movq %rdi, %cr3
        retq
        .cfi_endproc
```

funzionamento

```
#include <libce.h>

extern "C" void init_vm();
extern natq* tab_4;
natq var = 3;
void main()
{
        init_vm();
        var = 5;
        pause();
}
```

→

```
(gdb) p &var
$1 = (natq *) 0x205000 <var>
(gdb) vm path &var
cr3: 0x00206000
000: WU--A--  → 0x00207000
          000: WU--A--  → 0x00208000
                    001: WU--A--  → 0x0020a000
                              005: WU-----  → 0x00205000
```

proviamo a togliere il diritto di scrittura alla pagina contenente var

```
205
    2          1
000/000/001 000/000/101
```

```
movb $0, tab_1_1 + 8 * 0b101
```

↳ proviamo togliendo tutti i diritti:

```
(gdb) vm path &var
cr3: 0x00206000
000: WU--A--  → 0x00207000
          000: WU--A--  → 0x00208000
                    001: WU--A--  → 0x0020a000
                              005: RS----- b: 517
```

→ manca tradurre

Ottengo eccezione 14 (page fault)

```
exc_int_tipo_pf:
```

proviamo a eseguire un clear (altro indirizzo virtuale tradotto nello stesso indirizzo di var)

```
    4           3           2           1
000/000/000 000/000/000 000/000/011  777
```

: Lo vogliamo far corrispondere a var (0x 205)

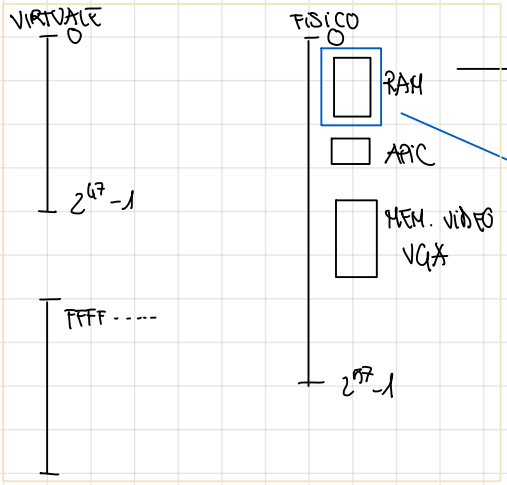
```
movq $0x205000, tab_1_3 + 8i* 0777
```

+ flag

```
(gdb) vm path &var
cr3: 0x00206000
000: WU--A--  → 0x00207000
        000: WU--A--  → 0x00208000
                001: WU--A--  → 0x0020a000
                        005: WU-----  → 0x00205000
```

```
(gdb) vm path 0x7ff000
cr3: 0x00206000
000: WU--A--  → 0x00207000
        000: WU--A--  → 0x00208000
                003: WU-----  → 0x0020c000
                        777: WU-----  → 0x00205000
```

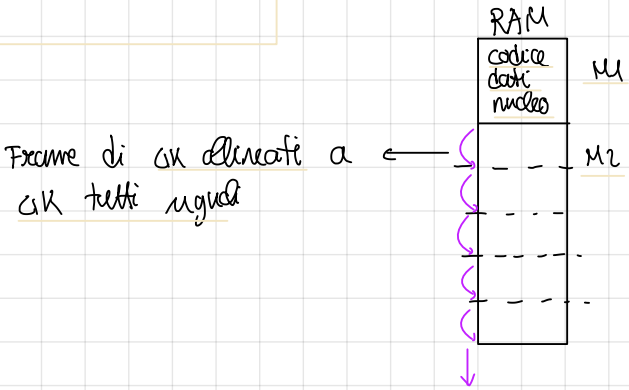
Spazio di indirizzamento



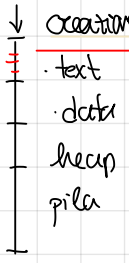
⚠ La CPU utilizza solo indirizzi virtuali

Deve contenere le tabelle di traduzione generate dal sistema

Ha bisogno di creare, modificare e distruggerle

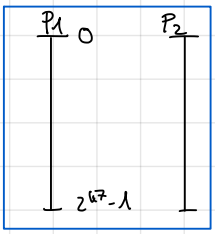


Lista frame liberi gestita con puntatori interni a frame: quando ce ne serve uno estraiamo dalla testa mentre inseriamo in testa quando lo liberiamo



Corrispondenza tra pagine e frame

Per vedere la lista la create_p deve creare tutto l'albero. Le tabelle vengono allocate in RAM perché i frame sono della dimensione giusta ed allineati nel modo giusto. Ricordiamo che tutti gli indirizzi non vengono tradotti dalla MMU e questo comporta che si può accedere solo agli indirizzi fisici appartenenti al codominio della funzione di traduzione ⟶ i frame devono far parte di questo codominio. Fatto che lo siano, il SW deve capire l'indirizzo virtuale delle tabelle. Per puntare alle tabelle di livello inferiore inoltre in ciascuna tabella deve essere presente il suo indirizzo fisico: le tabelle servono alla MMU. ⟶ Risoluzione: di default viene utilizzata la traduzione identità posta nella memoria virtuale di tutti i processi.



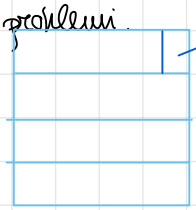
Sistema ⟶ qua mettiamo la traduzione identità di default. Gli indirizzi tradotti sono quelli necessari ad indirizzare lo spazio di indirizzamento fisico dedicato

Risolve il problema dei puntatori nelle tabelle e dell'accesso alla tabella di livello 4. Il sistema non fa quindi distinzione tra indirizzi fisici e virtuali per le sue strutture

Utente

In questa memoria virtuale deve essere presente anche la IDT perché gli indirizzi generati dalle interruzioni sono virtuali ed anche il TSS per indirizzare la pila ed infine la pila utente ed il codice delle system call. Mettiamo quindi tutto lo spazio fisico per risolvere tutti i problemi.

Semplifica attivazione paginazione:



bit 1 = paginazione attivata

↓

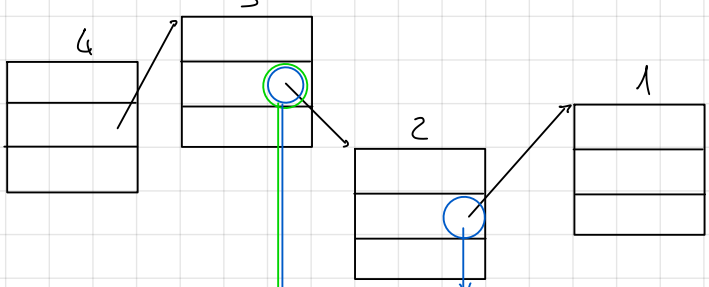
il sistema parte con la MMU disattivata e questa viene attivata successivamente

↓ momento attivazione

MOV $1, %CR0

%RIP → contiene indirizzo fisico: al momento dell'attivazione viene interpretato come virtuale ⟶ il passaggio è banale utilizzando la traduzione identità

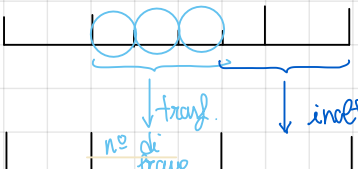
Utilizzo di pagine più grandi



PS: page size ⟶ Dice alla MMU che la traduzione è già lì e non c'è la tabella inferiore

↓

Le pagine sono più grandi:



pagina di 2^{21} = 2 Mega

Se PS=1 qua la pagina è grande 1GB ← La regione di 2Mega viene tradotta tutta nello stesso modo

può avvenire per tutte le pagine. Permette di risparmiare memoria e risparmiare tempo di ricerca. Lo svantaggio è la frammentazione interna che si crea e quindi si fa solo dove possibile.

È utile in particolar modo per la traduzione identità in quanto permette di accorciare alla tabella di livello 3 con pagine di 1GB ciascuna

⚠ PROBLEMA: lentezza

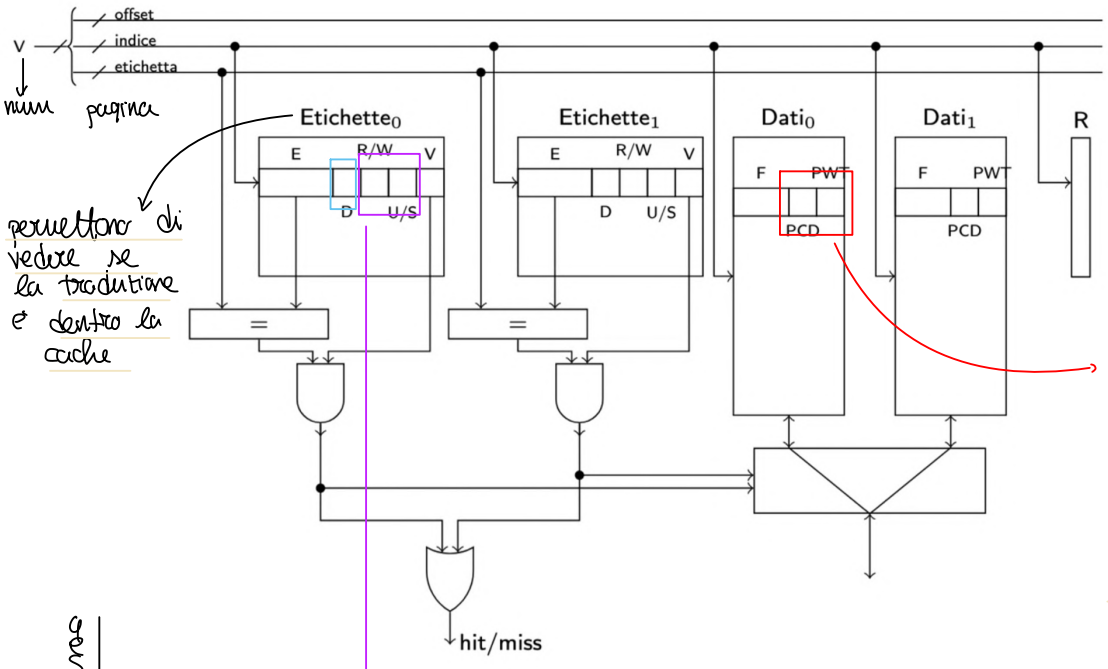
Avendo 4 tabelle, in generale per tradurre l'indirizzo si devono fare 4 accessi in memoria RAM: 500 cicli di clock. Inoltre, la MMU deve aggiornare i BIT e quindi leggerli e scriverli → 8 accessi in memoria

↓ Risoluzione

TLB: translation lookaside buffer: contiene traduzione num. pagina → num. frame

MMU

Prima di fare il percorso verifica se è già presente la traduzione. Se non è presente fa il percorso e porta la traduzione nel TLB



permettono di vedere se la traduzione è dentro la cache

→ Funziona alla stessa maniera della cache della CPU

↓

Valgono anche gli stessi principi di località

informazioni gestite di cache

↓

sui dati perché prima l'accesso deve essere garantito da etichette

Può evitare accessi utilizzando TLB → non serve ricordarsi tutti e 4 i hit del percorso basta fare un AND

BIT A: se la traduzione è già in TLB non lo aggiorna in quanto era già ad 1 per essere portato in TLB. Il hit regola che c'è stato davvero un accesso alla pagina e quindi per garantire il corretto funzionamento questi hit vengono azzerati periodicamente dal sistema. Non aggiornando il hit delle traduzioni nel TLB il servizio fornito da A non funziona più. Il fatto che il sistema azzeri il hit A invalida il funzionamento della cache in quanto le tabelle vengono cambiate da qualcosa che non è la MMU. La cache va in uno stato inconsistente e quindi il software invalida il TLB totalmente ad ogni picco, che viene quindi svuotato e ricostruito

↓ 2 modi per invalidare

modo 1: scrittura in %CR3: invalida tutto TLB perché cambio

allora di tradurre (anche se ci arrivo la stessa cosa)

modo 2: INVlpg indirizzo: invalida la traduzione dell'indirizzo se questa è presente in TLB

BIT D: va gestito correttamente perché è utilizzato nello swap-out dei processi. Il TLB ricorda il suo stato della lettura nell'albero. Se l'accesso è in lettura il hit non viene comunicato. Se l'accesso è in scrittura ed il TLB è 0 allora viene restituita miss in modo che D venga aggiornato e la traduzione ricaricata in TLB mentre se è 1 viene restituita hit e tutto si svolge correttamente. Nel primo caso non si aggiorna solo il hit D in TLB perché non esistono istruzioni per accedere al TLB. Anche se l'istruzione per accedere esistesse questo non si potrebbe fare in quanto l'aggiornamento di D in TLB comporterebbe l'accesso successivo all'albero che può essere fatto solo dalla MMU.

Cosa accade con traduzioni da più di 4K? La traduzione può essere inserita spezzandola in 4K pezzi ed occupando quindi 4K entrate. Oggi c'è un TLB diverso per ogni dimensione di pagina. La MMU deve per questo guardare parallelamente tutti i TLB e nel caso di miss deve caricare la pagina in quello giusto → VANTAGGIO avere pagine grandi

funzioni utili

MAX_LIV = 4 → 4 livelli di tabelle

BIT_SEGNO: serve per normalizzare gli indirizzi → 16 MSB = bit 47 (0 o 1)

MASCHERA: n 1 → 16 bit più sign a 0 e 48 meno sign a 1

↓ normalizzazione 0 seguito da 47 1

```
static inline vaddr norm(vaddr a)
{
        return (a & BIT_SEGNO) ? (a | ~MASCHERA_MODULO) : (a & MASC
HERA_MODULO);
}
```

(a | ~MASCHERA_MODULO): mette a 1 i 16 MSB

(a & MASC...): azzera parte significativa

dimensione regione dato livello

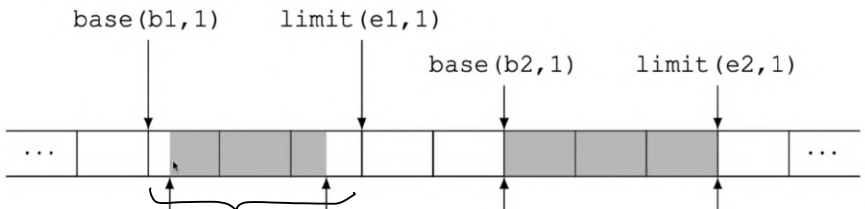
```
static inline constexpr natq dim_region(int liv)
{
        natq v = 1ULL << (liv * 9 + 12);
        return v;
}
```

se passo una costante la f() restituisce sempre lo stesso valore; passetto ottimizzativo

1ULL → parte da 0: pagina

unsigned long long per non avere ov

base/limite



base = prima toccata da intervallo

limit = prima regione non toccata

⚠ intervalli del tipo [b, e)

base

```
static inline vaddr base(vaddr v, int liv)
{
        natq mask = dim_region(liv) - 1;
        return v & ~mask;
}
```

→ n 1 nella parte meno significativa (grand. livello) 0 nella parte meno sign

limite

```
static inline vaddr limit(vaddr e, int liv)
{
        natq dr = dim_region(liv);
        natq mask = dr - 1;
        return (e + dr - 1) & ~mask;
}
```

se sono anche di 1byte dentro vado alla successiva ma entra resto dentro se sono all'inizio

devo vedere prima regione a destra

```
typedef natq tab_entry;
```

→ entrate tabelle:

```
const natq BIT_P    = 1U << 0; // il bit di presenza
const natq BIT_RW   = 1U << 1; // il bit di lettura/scrittura
const natq BIT_US   = 1U << 2; // il bit utente/sistema
const natq BIT_PWT  = 1U << 3; // il bit Page Wright Through
const natq BIT_PCD  = 1U << 4; // il bit Page Cache Disable
const natq BIT_A    = 1U << 5; // il bit di accesso
const natq BIT_D    = 1U << 6; // il bit "dirty"
const natq BIT_PS   = 1U << 7; // il bit "page size"
```

↳ settare: Or con costante
resettare: And con costante negata

Set/estrazione ind. fisico

```
const natq ACCB_MASK  = 0x00000000000000FF; // maschera per il byte
 di accesso
const natq ADDR_MASK  = 0x7FFFFFFFFFFFF000; // maschera per l'indir
izzo
```

```
static inline paddr extr_IND_FISICO(tab_entry descrittore)
{ // (
        return descrittore & ADDR_MASK; // )
}
static inline void  set_IND_FISICO(tab_entry& descrittore, paddr ind
_fisico) //
{ // (
        descrittore &= ~ADDR_MASK;
        descrittore |= ind_fisico & ADDR_MASK; // )
}
```

estrazione indice a partire da livello ed indirizzo

```
static inline int i_tab(vaddr ind_virt, int liv)
{
        int shift = 12 + (liv - 1) * 9;
        natq mask = 0x1ffULL << shift;
        return (ind_virt & mask) >> shift;
}
```

→ 1 solo dove c'è indice che mi serve

→ riporto solo in parte meno sign

estrazione indirizzo fisico

```
static inline int i_tab(vaddr ind_virt, int liv)
{
        int shift = 12 + (liv - 1) * 9;
        natq mask = 0x1ffULL << shift;
        return (ind_virt & mask) >> shift;
}
```

→ bit che mi serve (avrei potuto usare anche dim_region)

→ 1 solo dove serve

↳ ottengo ciò che non mi serve e riporto

riferimento ad entrata

```
static inline tab_entry& get_entry(paddr tab, natl index)
{
        tab_entry *pd = reinterpret_cast<tab_entry*>(tab);
        return  pd[index];
}
```

differenza tra vaddr e paddr nel debugger

v = 000024a49b724b6d S-111-222-333-444 ⟶ otto e percorre in albero

p = 0x00001000 ⟶ solo esadecimale

vogliamo mappare v in p ⟶ riutilizziamo la tabella del bootloader ed implementiamo solo la traduzione (leggiamo cr3 con readCR3();

definiamo le tabelle e l'allineamento

```
extern tab3, tab2, tab1
.data
.align 4096
tab3:
        .space 4096, 0
tab2:
        .space 4096, 0
tab1:
        .space 4096, 0
```

```
int main()
{
        vaddr v = 01112223334445555;
        paddr p = 4096;

        paddr tab4 = readCR3();
        tab_entry& e4 = get_entry(tab4, i_tab(v, 4));
```

⟶ prendo il riferimento alla tabella di livello 4 (dell'entrata) dell'indirizzo specificato.

```
set_IND_FISICO(e4, (paddr)&tab3);
e4 |= BIT_P | BIT_RW;
```

⟶ colleghiamo l'entrata della tabella 4 alla tabella 3

↳ settiamo bit P e RW

```
tab_entry& e3 = get_entry((paddr)&tab3, i_tab(v, 3));
set_IND_FISICO(e3, (paddr)&tab2);
e3 |= BIT_P | BIT_RW;
tab_entry& e2 = get_entry((paddr)&tab2, i_tab(v, 2));
set_IND_FISICO(e2, (paddr)&tab1);
e2 |= BIT_P | BIT_RW;
tab_entry& e1 = get_entry((paddr)&tab1, i_tab(v, 1));
set_IND_FISICO(e1, p);
e1 |= BIT_P | BIT_RW;
```

gestione del TLB

invalida TLB

```
// invalida tutto il TLB
        .global invalida_TLB
invalida_TLB:
        .cfi_startproc
        // e sufficiente riscrivere in cr3 il suo contenuto corrent
e
        movq %cr3, %rax
        movq %rax, %cr3
        retq
        .cfi_endproc
```

invalida entrata TLB

```
// dato un indirizzo virtuale (come parametro) usa l'istruzione inv
lpg per
// eliminare la corrispondente traduzione dal TLB
        .global invalida_entrata_TLB //
invalida_entrata_TLB:
        .cfi_startproc
        invlpg (%rdi)
        ret
        .cfi_endproc
```

iteratore: permette di gestire albero di traduzione come array (visita albero)
↓ utilizzando → start → fine → condizione: falso a fine ⟶ visita ANTICIPATA ALBERO

```
for (tab_iter it(tab4, v); it; it.next())
```

avanzamento

```
for (tab_iter it(tab4, v, v + 4*4096); it; it.next())
```

le pagine possono essere a cavallo di tabelle di livelli diversi: ci pensa l'iteratore a fare una visita anticipata ⟶ si ferma su un elemento prima di uscire e quindi possiamo usare l'iteratore per creare l'albero

↓

visita anticipata in profondità quindi scende fino in fondo prima di cambiare

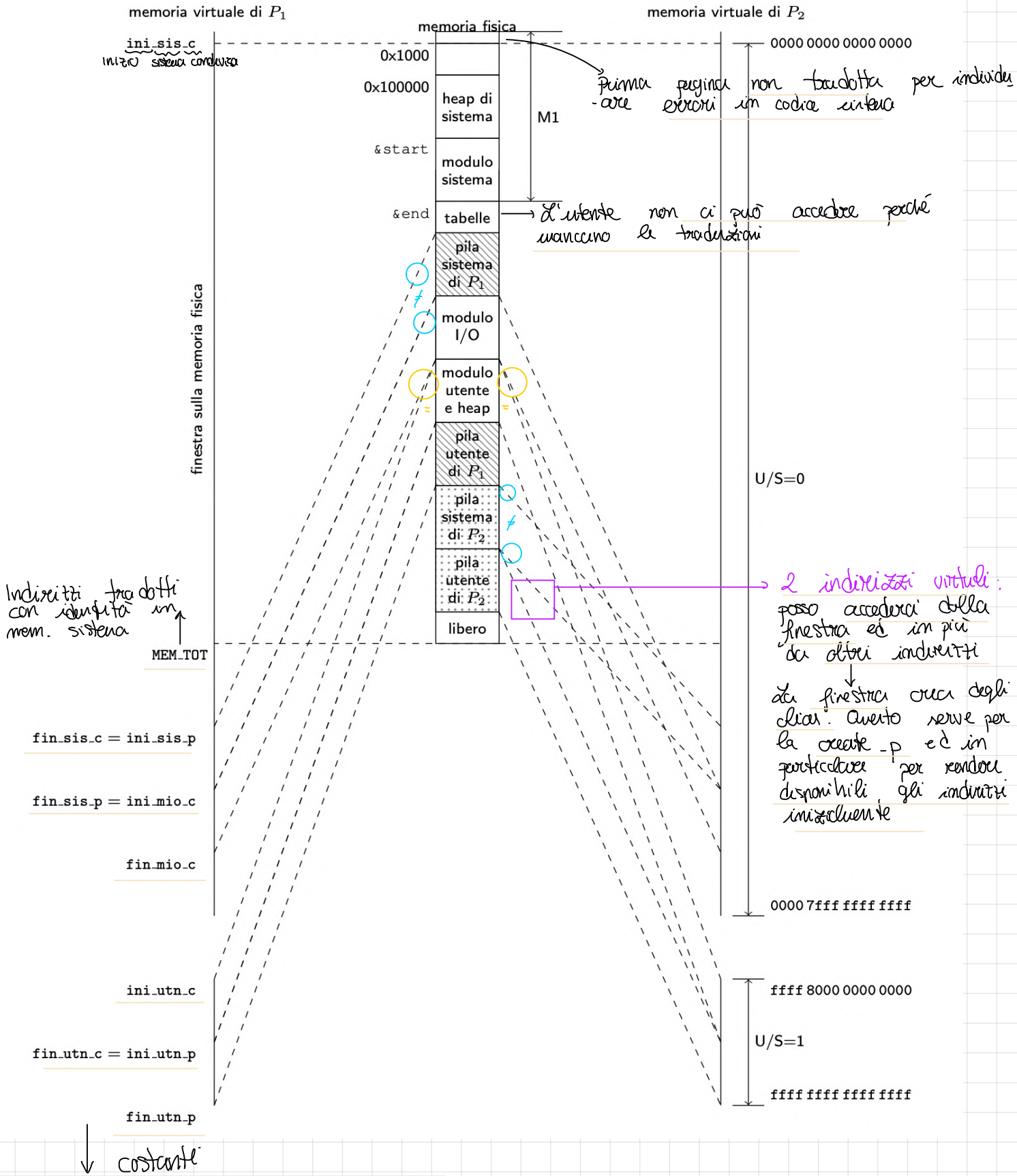
```
for (tab_iter it(tab4, v); it; it.next()) {
        tab_entry& e = it.get_e(); ⟶ rif. entrata
        int l = it.get_l(); ⟶ livello albero
        printf("livello %d: -> %x\n", l,
                        extr_IND_FISICO(e));
}
```

memoria virtuale processi

U/S = 0 (sistema)
- cond (stessi identificativi)
- priv
- I/O cond

Le sezioni text e data sono condivise quindi la loro traduzione, anche se provenienti da processi diversi, porta sempre agli stessi indirizzi di memoria fisica.
La pila utente sarà invece privata e quindi la traduzione degli indirizzi sarà diversa a seconda del processo

U/S = 1 (utente)
- utente / condivisa
- utente / privata



costanti

```
#define MEM_TOT                 (32*MiB)
// dimensione dello heap utente
#define DIM_USR_HEAP            (1*MiB)
// dimensione degli stack utente
#define DIM_USR_STACK           (64*KiB)
// dimensione dello heap del modulo I/O
#define DIM_IO_HEAP             (1*MiB)
// dimensione degli stack sistema
#define DIM_SYS_STACK           (4*KiB)
```

→ sistema cerca di usare meno memoria possibile

```
// ( suddivisione della memoria virtuale
//    N     = Numero di entrate in root_tab
//    I     = Indice della prima entrata in root_tab
//    SIS   = SIStema
//    MIO   = Modulo IO
//    UTN   = modulo UTeNte
//    C     = Condiviso
//    P     = Privato
#define I_SIS_C         0
#define I_SIS_P         1
#define I_MIO_C         2
#define I_UTN_C       256
#define I_UTN_P       384

#define N_SIS_C         1
#define N_SIS_P         1
#define N_MIO_C         1
#define N_UTN_C       128
#define N_UTN_P       128
```

quali entrate

quante entrate

⚠ Traduzioni di dimensioni multiple di 512 GB: occupano righe intere di tabelle di livello 4

Sistema caricato a partire dal 2MB, utente da indirizzi che iniziano con FFFF

gestore dei frame

```
struct des_frame {
        union {
                // numero di entrate valide (se il frame contiene una tabella)
                natw nvalide;
                // lista di frame liberi (se il frame è libero)
                natl prossimo_libero;
        };
};
```

Le informazioni non vanno mai insieme

Abbiamo scelto per deallocare frame

```
// numero totale di frame (M1 + M2)
natq const N_FRAME = MEM_TOT / DIM_PAGINA;
// numero totale di frame in M1 e in M2
natq N_M1;
natq N_M2;

// array dei descrittori di frame
des_frame vdf[N_FRAME];
// testa della lista dei frame liberi
natq primo_frame_libero;
// numero di frame nella lista dei frame liberi
natq num_frame_liberi;
```

Inizializzazione

```
void init_frame()
{
        // primo frame di M2
        paddr fine_M1 = allinea(reinterpret_cast<paddr>(&end), DIM_PAGINA);
        // numero di frame in M1 e indice di f in vdf
        N_M1 = fine_M1 / DIM_PAGINA;
        // numero di frame in M2
        N_M2 = N_FRAME - N_M1;

        if (!N_M2)
                return;
                                // creiamo la lista dei frame liberi, che inizialmente contiene tutti i
                                // frame di M2
                                primo_frame_libero = N_M1;
```

multiplo di pagina

creato da compilatore

Allocazione

```
paddr alloca_frame()
{
        if (!num_frame_liberi) {
                flog(LOG_ERR, "out of memory");
                return 0;
        }
        natq j = primo_frame_libero;
        primo_frame_libero = vdf[primo_frame_libero].prossimo_libero;
        vdf[j].prossimo_libero = 0;
        num_frame_liberi--;
        return j * DIM_PAGINA; ——→ Restituzione indirizzo fisico frame allocato
}
```

Deallocazione

```
void rilascia_frame(paddr f)
{
        natq j = f / DIM_PAGINA; ——→ indice corretto
        if (j < N_M1) {
                fpanic("tentativo di rilasciare il frame %p di M1", f);
        }        ↳ errore di sintassi
        vdf[j].prossimo_libero = primo_frame_libero;
        primo_frame_libero = j;
        num_frame_liberi++;
}
```

gestione indirizzi virtuali

```
const vaddr ini_sis_c = norm(I_SIS_C * dim_region(MAX_LIV - 1)); // sistema condivisa
const vaddr ini_sis_p = norm(I_SIS_P * dim_region(MAX_LIV - 1)); // sistema privata
const vaddr ini_mio_c = norm(I_MIO_C * dim_region(MAX_LIV - 1)); // modulo IO
const vaddr ini_utn_c = norm(I_UTN_C * dim_region(MAX_LIV - 1)); // utente condivisa
const vaddr ini_utn_p = norm(I_UTN_P * dim_region(MAX_LIV - 1)); // utente privata

// indirizzo del primo byte che non appartiene alla zona specificata
const vaddr fin_sis_c = ini_sis_c + dim_region(MAX_LIV - 1) * N_SIS_C;
const vaddr fin_sis_p = ini_sis_p + dim_region(MAX_LIV - 1) * N_SIS_P;
const vaddr fin_mio_c = ini_mio_c + dim_region(MAX_LIV - 1) * N_MIO_C;
const vaddr fin_utn_c = ini_utn_c + dim_region(MAX_LIV - 1) * N_UTN_C;
const vaddr fin_utn_p = ini_utn_p + dim_region(MAX_LIV - 1) * N_UTN_P;
```

prima entrata liv. 1 dim. regioni livello 3 (dim pagina regione con un livello in meno)

allocazione

```
paddr alloca_tab()
{
        paddr f = alloca_frame();
        if (f) {
                memset(reinterpret_cast<void*>(f), 0, DIM_PAGINA);
        }
        vdf[f / DIM_PAGINA].nvalide = 0; ——→ stato consistente
        return f;
}
```

Deallocazione

```
void rilascia_tab(paddr f)
{
        if (int n = get_ref(f)) { ——→ le entrate valide devono essere 0
                fpanic("tentativo di deallocare la tabella %x con %d entrate valide", f, n);
        }
        rilascia_frame(f);
}
```

set delle entrate

```
void set_entry(paddr tab, natl j, tab_entry se)
{
        tab_entry& de = get_entry(tab, j);
        if ((se & BIT_P) && !(de & BIT_P)) {
                inc_ref(tab);
        } else if (!(se & BIT_P) && (de & BIT_P)) {
                dec_ref(tab);
        }
        de = se;
}
```

→ Nuova entrata ha bit P=1

→ nuova entrata ha bit P=0

map

```
vaddr map(paddr tab, vaddr begin, vaddr end, natl flags, T& getpaddr, int ps_lvl)
```

→ template

Radice TRIE

[ )

flag

funzione che, dato un indirizzo virtuale, restituisce l'indirizzo fisico che vogliamo

Crea tutte le tabelle necessarie per creare le traduzioni [begin, end) e quando arriva a quelle di livello l ci mette l'indirizzo dato dalla funzione passata

↓ Algoritmo

```
for (/* tutto il sotto-albero visitato in pre-ordine */) {
     tab_entry& e =  /* riferimento all'entrata corrente */
     vaddr v =       /* indirizzo virtuale corrente */
     if (/* livello non foglia */) {
             if (/* tabella assente */) {
                     new_f = alloca_tab();
             }
     } else {
             new_f = getpaddr(v);
     }
     if (new_f) {
             /* inizializza 'e' in modo che punti a 'new_f' */
     }
}
```

→ Usa iteratore

\+ controlli: [begin, end) non deve contenere traduzioni precedenti
[begin, end) non deve sconfinare il buco → intervallo valido

\+ Ritorno: primo indirizzo per cui non c'è traduzione (end). Se il carattere restituito è diverso vi è verificato un errore → Si distrugge la traduzione

```
void unmap(paddr tab, vaddr begin, vaddr end, T& putpaddr)
```

distrugge tab con valido=0

Si passa l'indirizzo fisico e ci dice cosa fare

```
// otteniamo il corrispondente indirizzo fisico
// chiedendolo all'oggetto getpaddr
new_f = getpaddr(v);
```

→ Possiamo passare funzione o oggetto che si comporta come funzione

↓ Vantaggio

L'oggetto ha uno stato proprio che sopravvive all'invocazione dei suoi metodi e non è condiviso con altri oggetti. Questo è comodo nel caso in cui volessi passare un array di traduzione

Applicazione: creazione finestra di memoria

```
paddr root_tab = alloca_tab();
if (!root_tab)
        goto error;
```

: crea radice

↓ semplificando

```
paddr identity(vaddr v)
{
        return v;
}

map(root_tab, 0, MEM_TOT, flag, identity);
```

: traduzione identità

→ US=0; RW=1 → non controllato di default ma controllato dal nucleo perché ha flag settato

↓ Realtà non mappiamo 1°

```
if (map(root_tab, DIM_PAGINA, first_reg, BIT_RW, identity_map) != first_reg)
        return false;
```

→ non MEM_TOT perché mappo pagine da 4K per fare distinzione mem. video

→ non riuscito a mappare

```
tab_iter it(root_tab, 0xb8000, DIM_PAGINA);
while (it.down())
        ;
tab_entry& e = it.get_e();
e |= BIT_PWT;
```

→ PWT = 1 per mem. video

iteratore scorre solo nel percorso di quell'indirizzo e si ferma sulla foglia

→ iteratore fermo sulla foglia

```
if (MEM_TOT > first_reg) {
        if (map(root_tab, first_reg, MEM_TOT, BIT_RW, identity_map, 2) != MEM_TOT)
                return false;
}
```

→ Pagine da 2MB (livello 2)

```
// Mappiamo gli ultimi 20MiB prima di 4GiB settando sia PWT che PCD.
// Questa zona di indirizzi è utilizzata dall'APIC per mappare i propri registri.
vaddr   beg_pci = 4*GiB - 20*MiB,
        end_pci = 4*GiB;
if (map(root_tab, beg_pci, end_pci, BIT_RW|BIT_PCD|BIT_PWT, identity_map, 2) != end_pci)
        return false;
```

disabilita cache

Creazione spazio condiviso → deve creare corrispondente con sezioni data e text

```
if (map(root_tab,
        virt_beg,
        virt_end,
        flags,
        copy_segment{mod_beg, mod_end, virt_beg}) != virt_end)
        return 0;
```

→ oggetto che legge il file ELF per vedere posizione .data e .text

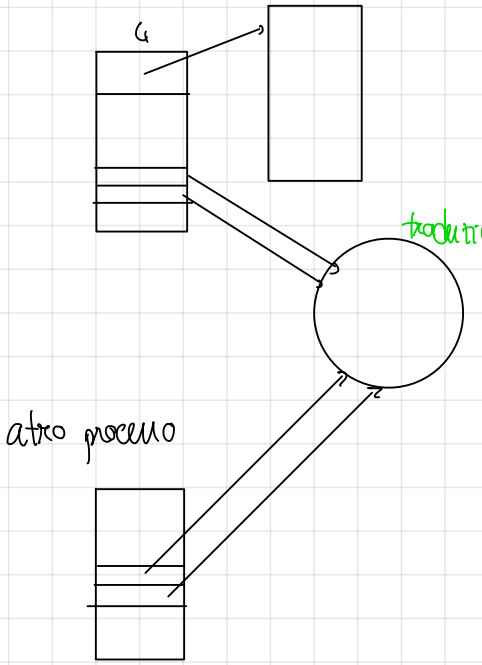
nella creazione dei processi

```
// creazione della tabella radice del processo
p->cr3 = alloca_tab();
if (p->cr3 == 0)
        goto err_rel_id;
init_root_tab(p->cr3);
```

: tabella di livello 4

SEMPLIFICAZIONE: parti condivise tra tutti i processi non cambiano

processo esistente (quello che crea il processo nuovo)



: se condividiamo: puntano agli stessi sottoalberi

Non condividiamo la tabella di livello 0 perché contiene anche le traduzioni per le parti private

```
void init_root_tab(paddr dest)
{
        paddr pdir = readCR3();

        // ci limitiamo a copiare dalla tabella radice corrente i puntatori
        // alle tabelle di livello inferiore per tutte le parti condivise
        // (sistema, utente e I/O). Quindi tutti i sottoalberi di traduzione
        // delle parti condivise saranno anch'essi condivisi. Questo, oltre a
        // semplificare l'inizializzazione di un processo, ci permette di
        // risparmiare un po' di memoria.
        copy_des(pdir, dest, I_SIS_C, N_SIS_C);
        copy_des(pdir, dest, I_MIO_C, N_MIO_C);
        copy_des(pdir, dest, I_UTN_C, N_UTN_C);
}
```

pdir: proc che ha chiamato activate_p

copia del disegno sopra

Creazione pila

Radice

Bottom

```
if (!crea_pila(p->cr3, fin_sis_p, DIM_SYS_STACK, LIV_SISTEMA))
        goto err_rel_tab;
```

che indirizzo passiamo?

```
paddr mia_getpaddr(vaddr v)
{
        return alloca_frame();
}
```

: Mettiamo la pila sistema in un nuovo frame

```
bool crea_pila(paddr root_tab, vaddr bottom, natq size, natl liv)
{
        vaddr v = map(root_tab,
                bottom - size,   begin
                bottom,          end
                BIT_RW | (liv == LIV_UTENTE ? BIT_US : 0),
                [](vaddr) { return alloca_frame(); });
        if (v != bottom) {   → map ha fallito
                unmap(root_tab, bottom - size, v,
                        [](paddr p, int) { rilascia_frame(p); });
                return false;
        }
        return true;
}
```

livello

Inizializzazione

```
if (liv == LIV_UTENTE) {
        // inizializziamo la pila sistema.
        pl[-5] = reinterpret_cast<natq>(f); // RIP (codice utente)
        pl[-4] = SEL_CODICE_UTENTE;         // CS (codice utente)
        pl[-3] = BIT_IF;                    // RFLAGS
        pl[-2] = fin_utn_p - sizeof(natq);  // RSP
        pl[-1] = SEL_DATI_UTENTE;           // SS (pila utente)
```

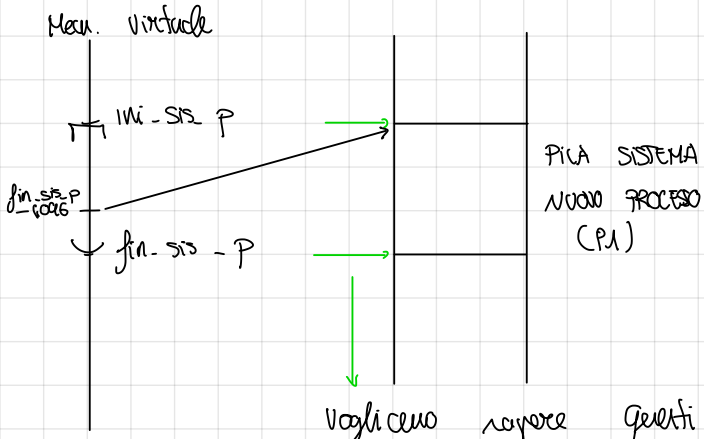
Per accedere alla pila del processo appena creato (e non nel cr3 corrente) utilizziamo l'indirizzo fisico della finestra di memoria

```
pila_sistema = trasforma(p->cr3, fin_sis_p - DIM_PAGINA) + DIM_PAGINA;
// convertiamo a puntatore a natq, per accedervi più comodamente
pl = reinterpret_cast<natq*>(pila_sistema);
```

→ fa la traduzione "a mano"

vedi sotto — scritto sotto frame — ultima parola quadrupla frame

Primo byte della RAM fuori della pagina



Vogliamo sapere questi indirizzi fisici: Dato che la pila è grande 4k è sufficiente una pagina. Ci interessa tradurre fin_sis_p - 4096 fino a fin_sis_p (escluso)!

```
if (!crea_pila(p->cr3, fin_utn_p, DIM_USR_STACK, LIV_UTENTE)) {
        flog(LOG_WARN, "creazione pila utente fallita");
        goto err_del_sstack;
}
```

```
paddr trasforma(paddr root_tab, vaddr v)
{
        // usiamo un tab_iter sul solo indirizzo 'v', fermandoci
        // sul descrittore foglia lungo il percorso di traduzione
        tab_iter it(root_tab, v);
        while (it.down())
                ;

        // si noti che il percorso potrebbe essere incompleto.
        // Ce ne accorgiamo perché il descrittore foglia ha il bit P a
        // zero. In quel caso restituiamo 0, che per noi non è un
        // indirizzo fisico valido.
        tab_entry e = it.get_e();
        if (!(e & BIT_P))
                return 0;
// se il percorso è completo calcoliamo la traduzione corrispondente.
// Si noti che non siamo necessariamente arrivati al livello 1, se
// c'era un bit PS settato lungo il percorso.
int l = it.get_l();
natq mask = dim_region(l - 1) - 1;
return (e & ~mask) | (v & mask);
```

e → numero frame

→ n bit col 1 (offset)

isoliamo nr. di frame | otteniamo nr. di pagina e teniamo offset

Distruzione processo

```
void distruggi_processo(des_proc* p)
{
        paddr root_tab = p->cr3;
        if (p->livello == LIV_UTENTE)
                distruggi_pila(root_tab, fin_utn_p, DIM_USR_STACK);
        ultimo_terminato = root_tab;
        if (p != esecuzione_precedente) {
                distruggi_pila_precedente();
        }
        rilascia_proc_id(p->id);
        delete p;
```

→ Rilascia pila sistema quando non ci siamo

```
void distruggi_pila(paddr root_tab, vaddr bottom, natq size)
{
        unmap(
                root_tab,
                bottom - size,
                bottom,
                [](paddr p, int) { rilascia_frame(p); });
}
```

Carica stato

```
// nuovo valore per cr3
movq CR3(%rbx), %r10
movq %cr3, %rax
cmpq %rax, %r10
je 1f

movq %r10, %rax
movq %rax, %cr3
```

→ carica traduzione

↓

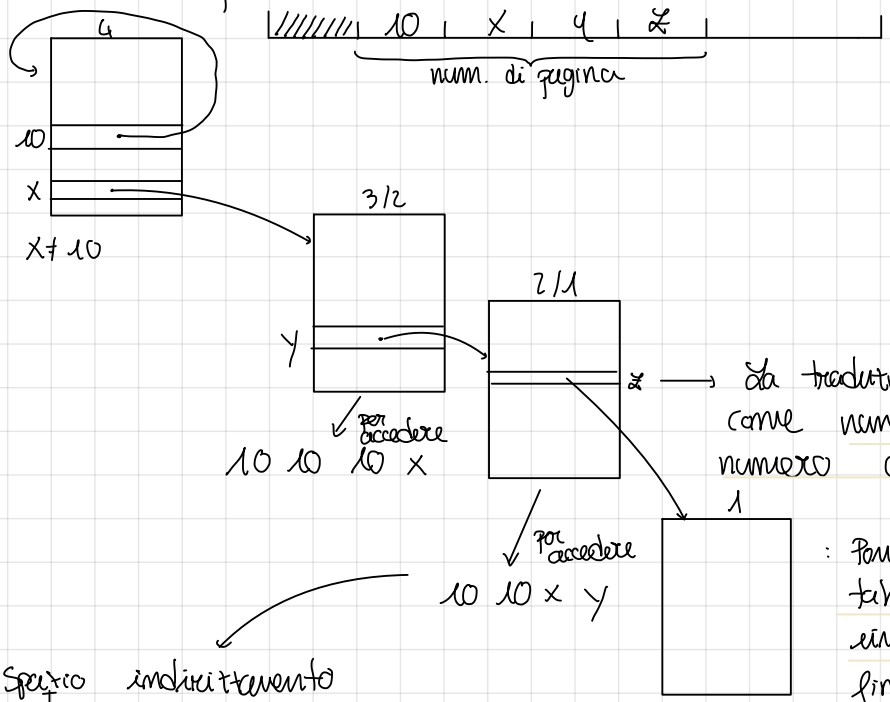
Accessibile memoria nuovo processo

```
// evitiamo di invalidare il TLB
// se cr3 non cambia

                    // il TLB viene invalidato
```

→ Può capitare che il TRIE non venga cambiato e che venga rieseguito lo stesso processo

⚠ La unmap deve fare una visita posticipata, ovvero deve accedere fino in fondo prima di eliminare → funzione bt.post()

Utilizzi: puntatore alla stessa tabella

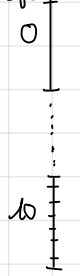


→ La traduzione finisce: l'indirizzo viene usato come numero di frame per sostituire il numero di pagina

↓

: Possiamo leggere/scrivere in questa tabella tramite l'offset: utile per sistema → metodo alternativo alla finestra

Spazio indirizzamento



→ tutte tabelle di livello 1 visibili

↓

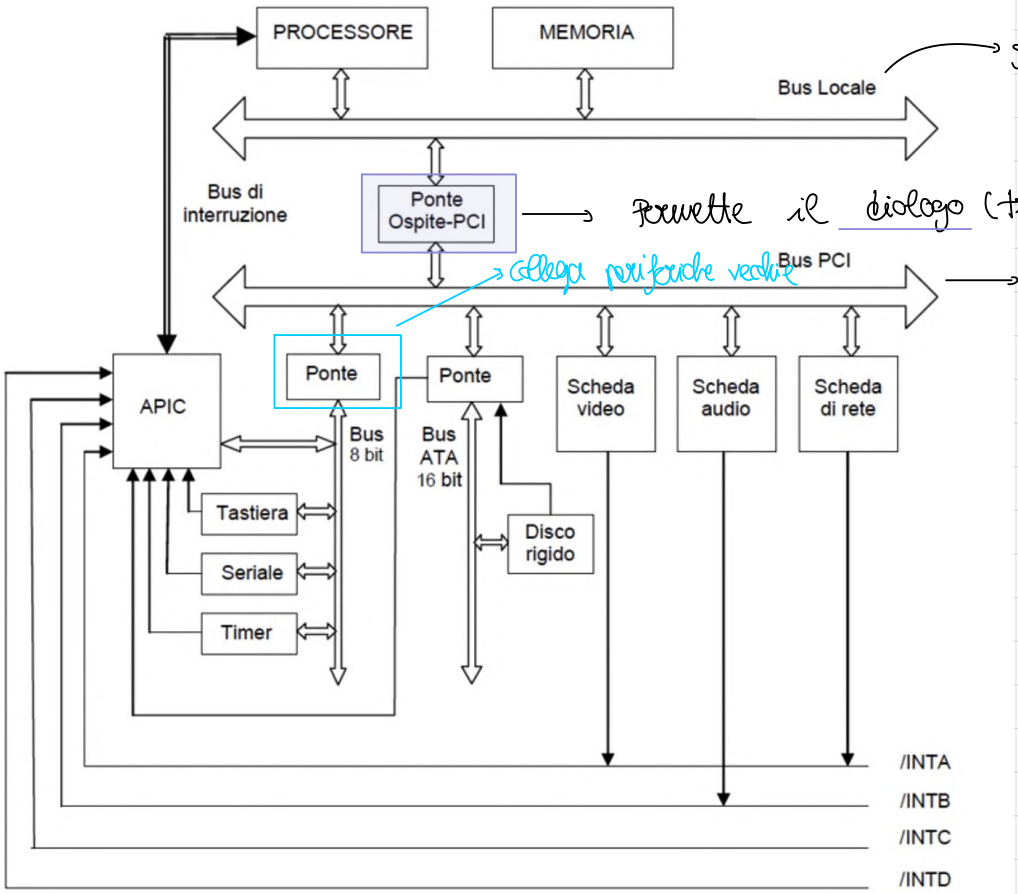
Elimina il problema di avere la finestra di memoria che "ruba" spazio all'utente

Ingresso / uscita

- Bus PCI

Standard: sostituisce il PC-AT perché aveva vari problemi → il PC IBM era espandibile con altri dispositivi. Altre aziende producevano schede per questo. È necessario che queste schede siano compatibili tra di loro e c'è bisogno di un software (driver) che ci permette di usarle e deve capire se è collegata o no. Per scoprirlo il driver deve scrivere all'indirizzo fornito dal produttore e vedere se legge poi il risultato atteso. Ci sono problemi (c'erano) se più schede usano gli stessi indirizzi. Il bus PCI nasce per risolvere questi problemi.

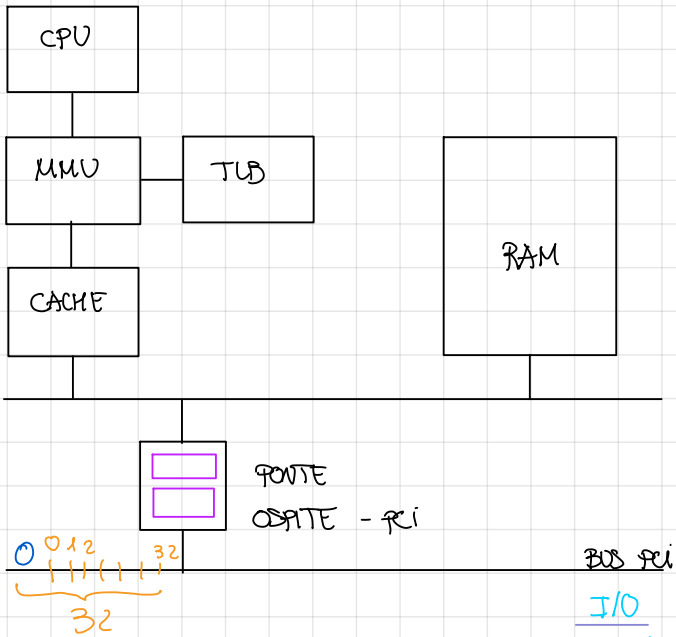
Struttura



Specifico del processore

Permette il dialogo (traduttori)

Possono essere creati albori di massimo 256 bus PCI collegati tra loro da ponti



BUS PCI → Risolve problemi di prima: definisce 3 nuovi spazi di indirizzamento

I/O

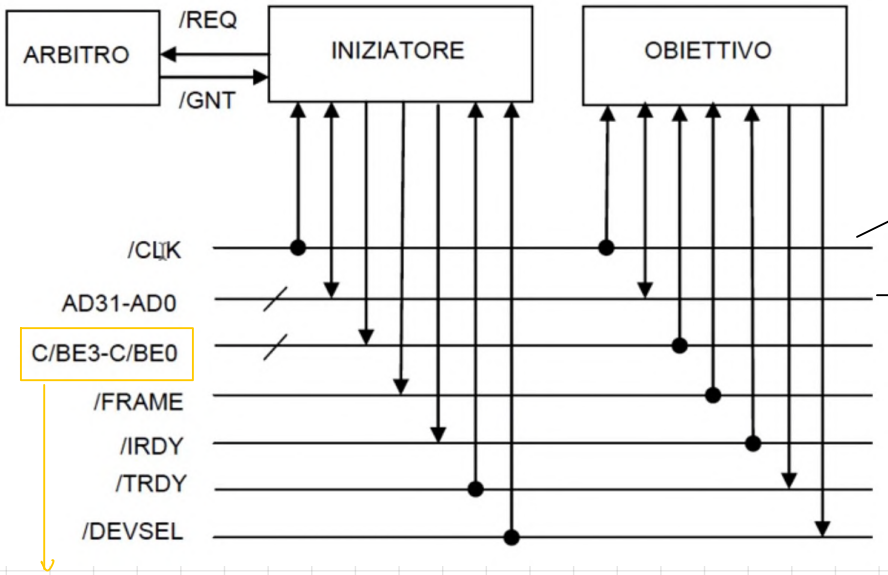
MEMORIA

CONFIGURAZIONE → i dispositivi acquisiscono un indirizzo automaticamente

Identificazione geografica

In ogni BUS (max 256) ci sono vari Spazi ove inserire schede (massimo 32). Ogni slot ha un identificatore dato dall'hardware (numero). Le schede multifunzione possono avere al massimo 8 funzioni, numerate da 0 a 7 dal costruttore. Alla fine, ogni scheda è identificata univocamente. Non sono però disponibili interrupt che permettono alla CPU di accedere allo spazio di configurazione e quindi ci si accede indirettamente in fase di avvio tramite i registri del ponte nello spazio di I/O della CPU → molto costoso: le operazioni devono diventare disponibili nello spazio di I/O per fare IN/OUT. Le schede compatibili PCI devono avere maschere configurabili e quindi è necessario avere sulla scheda un comparatore con un registro. Una volta configurato il comparatore, alla scheda si accede con le normali istruzioni IN/OUT → vantaggio: configurati da SW sistema che si occupa di assegnare indirizzi diversi all'avvio. Inoltre, il BUS stabilisce anche cosa fare se un dispositivo non risponde: serve una configurazione particolare riconosciuta dal suo come errore

collegamenti



→ Linee comunicate secondo il clock: 66 MHz (prima 33)
→ Linee di indirizzo/dati condivise.
↓ Fasi transazioni BUS
1. indirizzamento
2. scambio dati: possono essere più di una

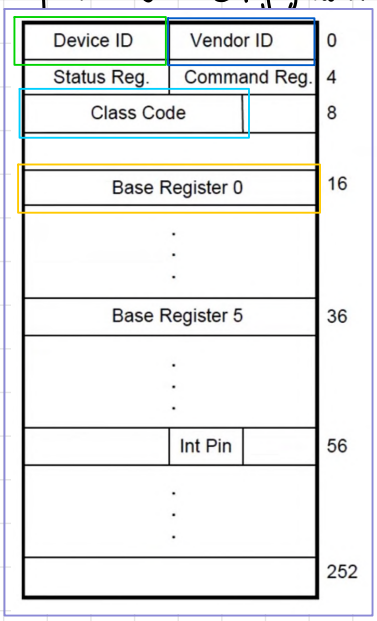
Linee di controllo e byte-enable: nella fase di indirizzamento fungono da linee di controllo e quindi codificano il tipo di operazione (r/w) e lo spazio di indirizzamento. Nelle fasi successive servono come byte enable, ovvero specificano quali byte della parola trasferire. I dati sono trasferiti con un parallelismo di 4 e quindi AD1 e AD0 sono sempre 0. Per stabilire quali byte trasferire tra i 4 indirizzati sempre viene utilizzato il byte enable

↓

Le operazioni sono dette **transazioni**, ciascuna con un proprio iniziatore ed obiettivo. Qualunque dispositivo può essere iniziatore o obiettivo (un ruolo per volta). Tipicamente l'iniziatore è il ponte. L'iniziatore pilota la fase di indirizzamento e per fare questo è necessario che gli iniziatori siano collegati con un arbitro che esamina le richieste e prima o poi dà l'assenso all'iniziatore. Questo poi deve attendere che le altre transazioni terminino e poi può pilotare il bus. Deve attendere in quanto l'architettura può essere fatta neanche è in esecuzione un'altra transazione. Per pilotare il bus deve: asserire /FRAME, pilotare l'indirizzo di controllo ed attendere che i dispositivi attivino la linea /DEVSEL. Se questa non viene attivata entro un certo numero di clock questo implica che il dispositivo non è presente. Fatto questo si passa al trasferimento dati utilizzando /IRDY e /TRDY come handshake. Finché /FRAME viene tenuto a 0 si continuano a trasferire dati in quanto viene messo ad 1 all'ultima fase. Quando anche /IRDY e /TRDY sono disattivati si può dare il via ad un nuovo trasferimento. I byte

enable sono sempre pilotati dall'iniziatore mentre le linee dati sono pilotate dall'iniziatore o dall'obiettivo a seconda che l'operazione sia di lettura o scrittura

Spazio di configurazione



Ogni funzione PCI acquisisce un indirizzo nello spazio di configurazione automaticamente. Ciascuna di esse deve avere al suo interno 256 byte di configurazione. Questi vengono chiamati registri ed alcuni sono obbligatori

→ Venditore

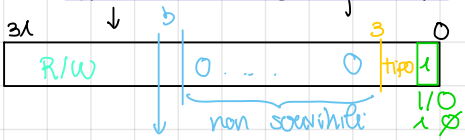
→ Scelto architetturalmente dal venditore

→ Ci permette di sapere cos'è il dispositivo: è possibile usarlo anche se non si ha un driver specifico

Class Code (byte più significativo)	Significato PCI
0x00	Pre revision 2.0
0x01	Mass storage controller
0x02	Network controller
0x03	Display controller
0x04	Multimedia device
0x05	Memory controller
0x06	Bridge device
0x07	Simple communication controller
0x08	Base system peripheral
0x09	Input device
0x0A	Docking station
0x0B	Processor
0x0C	Serial bus controller
0xFF	Other

→ Permettono di programmare i comparatori: sono al massimo 6 e sono decisi dal costruttore

Alcune cose le sa solo il venditore (quanti registri e se devono stare in I/O o memoria). Il software decide invece dove disporli e deve comunicarlo alla scheda. Il base register gestisce un blocco di registri e codifica quanto è grande il blocco. Il base register viene letto dal SW ed assegna l'indirizzo di partenza del blocco scrivendolo nel registro



Il primo bit b scrivibile codifica la dimensione del blocco: grande 2^b Per scoprire quanto è b il SW scrive tutti 1 e vede fin dove è rimasto a scrivere

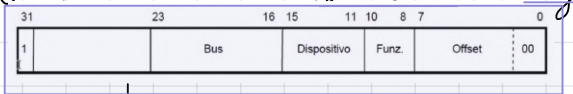
2^b Deve poi decidere dove allocare il blocco: vengono scelte regioni naturali di 2^b. Per comunicare al dispositivo qual è il suo blocco scrive

2^b l'indirizzo: 32 - b | b (offset)

2^b Ciò che va a finire in R/W. Per il resto ci pensa il driver

2^b

Queste operazioni vengono fatte all'avvio: il ponte ha 2 registri nello spazio di I/O (0xCF8 e 0xCFC) che permettono di accedere allo spazio di configurazione indirettamente: nel primo (CAP) si scrive l'indirizzo a cui accedere, dopodiché il secondo (CDP) diventa una finestra all'interno della memoria di configurazione di quella particolare funzione. Per scrivere in CAP:



4 byte enable vengono presi dai byte enable dell'operazione fatta su CDP.

Se vengono rispettate vengono scritti tutti i in CDP. Il software in questo modo può enumerare i dispositivi installati e può configurarli per dargli indirizzi non sovrapposti. Per iniziare quelli collegati, il software prova a leggere qualcuno dello spazio di configurazione di ogni possibile dispositivo e funzione ed in particolare legge il vendor ID. Se legge tutti 1 (inesistente) allora conclude che non c'è niente, altrimenti può leggere anche le altre informazioni e configurare i base register

UTILIZZO LIBCE

```
natb pci_read_confb(natb bus, natb dev, natb fun, natb off)
{
        natl confaddr = make_CAP(bus, dev, fun, off);
        outputl(confaddr, CAP); : scrive configurazione
        return inputb(CDP + (off & 0x03));
}
```

utilizzo byte enable

"compone" CAP come detto precedentemente

```
natl make_CAP(natb bus, natb dev, natb fun, natb off)
{
        return 0x80000000 | (bus << 16) | (dev << 11) | (fun << 8) | (off &
0xFC);
}
```

2 bit meno significativi =0

```
void pci_write_confb(natb bus, natb dev, natb fun, natb off, natb data)
{
        natl confaddr = make_CAP(bus, dev, fun, off);
        outputl(confaddr, CAP);
        outputb(data, CDP + (off & 0x03));
}
```

Scrittura config

Lettura elenco dispositivi attivi

```
#include <libce.h>
int main()
{
        natb bus = 0, c;
        for (natb dev = 0; dev < 32; dev++) {
                for (natb fun = 0; fun < 8; fun++) {
                        natw vendorID, deviceID;

                        vendorID = pci_read_confw(bus, dev, fun, 0);
                        if (vendorID == 0xFFFF) continue;
                        deviceID = pci_read_confw(bus, dev, fun, 2);
                        printf("%2x.%2x.%2x: %4x.%4x\n",
                                        bus, dev, fun,
                                        vendorID, deviceID);
                }
        }
        pause();
        return 0;
}
```

Con CTRL+ALT e info pci

```
QEMU
wavcapture path audiodev [frequency [bits [channels]]] -- capture audio to a wav
e file (default frequency=44100 bits=16 channels=2)
x /fmt addr -- virtual memory dump starting at 'addr'
x_colo_lost_heartbeat  -- Tell COLO that heartbeat is lost,
                          a failover or takeover is needed.
xp /fmt addr -- physical memory dump starting at 'addr'
(qemu) info pci
  Bus  0, device   0, function 0:
    Host bridge: PCI device 8086:1237
      PCI subsystem 1af4:1100
      id ""
  Bus  0, device   1, function 0:
    ISA bridge: PCI device 8086:7000
      PCI subsystem 1af4:1100
      id ""
  Bus  0, device   1, function 1:
    IDE controller: PCI device 8086:7010
      PCI subsystem 1af4:1100
      BAR4: I/O at 0xc000 [0xc00f].
      id ""
  Bus  0, device   1, function 3:
    Bridge: PCI device 8086:7113
      PCI subsystem 1af4:1100
      IRQ 9, pin A
```

tastiera

HDD

```
QEMU
      PCI subsystem 1af4:1100
      id ""
  Bus  0, device   1, function 0:
    ISA bridge: PCI device 8086:7000
      PCI subsystem 1af4:1100
      id ""
  Bus  0, device   1, function 1:
    IDE controller: PCI device 8086:7010
      PCI subsystem 1af4:1100
      BAR4: I/O at 0xc000 [0xc00f].
      id ""
  Bus  0, device   1, function 3:
    Bridge: PCI device 8086:7113
      PCI subsystem 1af4:1100
      IRQ 9, pin A
      id ""
  Bus  0, device   2, function 0:
    VGA controller: PCI device 1234:1111
      PCI subsystem 1af4:1100
      BAR0: 32 bit prefetchable memory at 0xfd000000 [0xfdffffff].
      BAR2: 32 bit memory at 0xfebf0000 [0xfebf0fff].
      BAR6: 32 bit memory at 0xffffffffffffffff [0x0000fffe].
      id ""
(qemu)
```

scheda video

: 16 MB → memoria video mod. grafica

Come facciamo a spostare le operazioni nello spazio di indirizzamento? Ci serve il ponte che deve capire, in base agli indirizzi, quali sono le operazioni dirette ai dispositivi e quali no.

↓ 2 modi

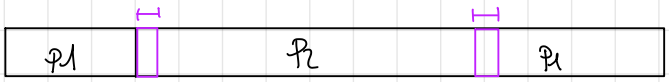
ATTIVO: Riconosce finestra PCI → quando un indirizzo è in tale intervallo capisce che l'operazione è destinata al BUS PCI (tipico della memoria: ultimi 20MB)

PASSIVO: Se il ponte vede che nessuno risponde per un certo tempo prova a trasferire la transazione al BUS PCI (utilizzato per I/O)

interrupt

4 possibili linee per richiesta interruzione: A, B, C, D e deve dire se una funzione è collegata ad una di queste 4 linee (ed eventualmente a quale). Questo è scritto in INTPIN. Non viene detto come sono collegate a linee per rendere tutto indipendente dal processore. In QEMU si utilizzano le linee dell'APIC dopo la 16. Inoltre, il sw di configurazione scrive a che piedino dell'APIC è collegata la funzione

- I/O



Avvia operazione di I/O

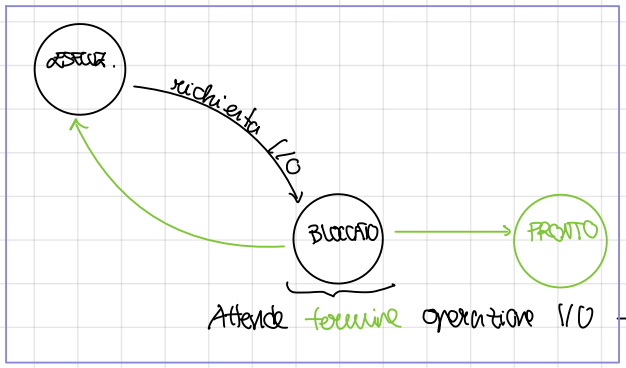
→ I processi non hanno interesse a collaborare ed in particolare a cedere la CPU l'uno all'altro

↓

L'I/O può essere usata solo a livello sistema con IOPL o tradurione ed utilizziamo le interruzioni

Le porte viste sono porzioni di codice di sistema. Di fatto stiamo effettuando un'operazione di sincronizzazione

P1:



→ Possiamo usare i semafori: vogliamo che P1 si blocchi su un semaforo di sincronizzazione finché l'operazione di I/O non termina

↓ Altro problema risolto

Possiamo risolvere anche il problema di mutua esclusione nel caso in cui due processi vogliano usare la stessa periferica

↓ **organizzazione operazione**

a_in:
sem_wait (mutex)
:
sem_signal (mutex)

→ avvia operazione
sem_wait (sync) : blocca processo e ne schedula un altro

↓

Tutte chiamate di sistema chiamate da altre primitive di sistema.

⚠ a_in:
call salva_stato
call c_in
call carica_stato
IRETQ

→ ⚠ anche tutte le altre chiamate fanno salva_stato e carica_stato

↓

Le salva_stato di a e di sem_wait salvano nello stesso descrittore di processo e quindi la seconda sovrascrive la prima

↓

Errore concettuale: a non deve fare salva/carica stato perché deve cambiare processo ed inoltre le operazioni di a possono essere interrotte e riprese. Sono quindi simili ad un processo utente e non una primitiva atomica. Non è un problema perché stiamo utilizzando i semafori

Parte codice sistema che interrompe P2

a_driver:

call salva_stato (P2)

⋮

// sem_signal (sync) → non possiamo usarla e quindi dato che siamo a livello sistema possiamo fare le stesse cose "a mano"

call scelta_stato

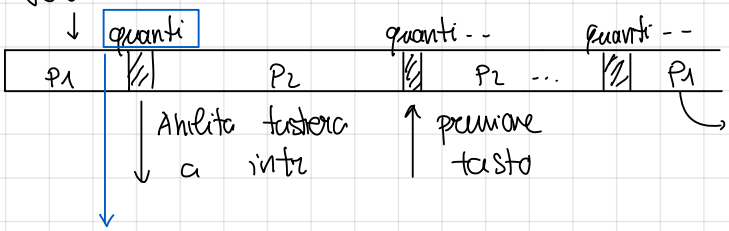
IRETQ

Primitive fornite per dialogare con I/O:

read (int id , char * buffer , natl quanti)
- id: identificatore numerico periferica data da sistema
- buffer: contiene byte letti (ad esempio da tastiera)
- quanti: quanti byte leggere

write (int id, const char * buffer, natl quanti)

Non è detto che una periferica possa svolgere da sola tutte l'operazione: il interrupt fornisce a system call e la primitiva svolge l'operazione con ciò che la periferica sa fare con tutti gli interrupt segnalano la fine dell'operazione (ad esempio, dovevamo leggere più caratteri da tastiera e ci basa un interrupt dopo la lettura di ciascuno). La system call si ricorda quanti byte erano stati richiesti dall'utente e decrementa ogni volta



Come passiamo quanti? Utilizziamo lo spazio di sistema condiviso. Avremo un array di descrittori di periferiche con queste informazioni e gli indici dei semafori associati a tali periferiche. Il driver deve sapere la periferica a cui esso è associato in modo da accedere alla struttura dati corretta dove trova tutte le sue informazioni e le informazioni sulle operazioni di I/O in corso. Per sapere a che periferica è associato il driver è possibile guardare il tipo di interruzione. Il driver inoltre deve prelevare il testo e scriverlo ove richiesto dalla primitiva. La memoria virtuale attiva adesso è quella di P2 e quindi il driver non può scrivere all'indirizzo buf specificato in read perché in questo modo scriverebbe in P2

Importanza limitazioni sul buffer passato a read: deve essere nella memoria condivisa → deve essere dichiarato a livello globale.

nella macchina QEMU

```
extern "C" void cread_n(natl id, char *buf, natl quanti);
```

possono essere di numero 4

```
cread_n:
        int $IO_TIPO_CREAD
        ret
```

: livello utente

A livello sistema

```
// i tipi da 0x50 a 0xFE verrano usati per gli handler
// (si veda load_handler() più avanti)
carica_gate     0xa0            a_driver_ce_0   LIV_SISTEMA
carica_gate     0xa1            a_driver_ce_1   LIV_SISTEMA
carica_gate     0xa2            a_driver_ce_2   LIV_SISTEMA
carica_gate     0xa3            a_driver_ce_3   LIV_SISTEMA
```

: 4 driver per differenziare le 4 periferiche possibili

```
a_driver_ce_0:
        .cfi_startproc
        .cfi_def_cfa_offset 40
        .cfi_offset rip, -40
        .cfi_offset rsp, -16
        call salva_stato
        mov $0, %rdi
        call c_driver_ce
        call carica_stato
        iretq
        .cfi_endproc
```

→ Comunica che è stata la periferica 0 a fare la richiesta

Primitiva:

```
a_ceread_n:
        .cfi_startproc
        .cfi_def_cfa_offset 40
        .cfi_offset rip, -40
        .cfi_offset rsp, -16
        call c_ceread_n
        iretq
        .cfi_endproc
```

⚠ Non ci sono salva / carica stato: il processo prosegue dopo aver innalzato il livello di privilegio facendo altre cose

Descrittore

```
static const int MAX_CE = 4;
struct des_ce {
        ioaddr iCTL, iSTS, iRBR;
        char *buf;
        natl quanti;
        natl sync;
        natl mutex;
};
des_ce array_ce[MAX_CE];
natl next_ce;
```

Periferica PCI: I registri possono essere ovunque nello spazio di I/O → l'indirizzo viene scoperto all'avvio e poi non cambiano

} quante ne abbiamo tante

Inizializzazione

```
bool ce_init()
{

        for (natb bus = 0, dev = 0, fun = 0;
             pci_find_dev(bus, dev, fun, 0xedce, 0x1234);
             pci_next(bus, dev, fun))
        {
                if (next_ce >= MAX_CE) {
                        flog(LOG_WARN, "troppi dispositivi ce");
                        break;
                }
                des_ce *ce = &array_ce[next_ce];
                flog(LOG_INFO, "ce%d %2x:%1x:%1x base=%4x IRQ=%d", next_ce, bus, dev, fun, ce->iSTS, irq)
;
                next_ce++;
        }
        return true;
}
```

→ RIFERIMENTO: bus, dev, fun

vendor ID device ID

→ percorriamo all'inizio

```
natl base = pci_read_confl(bus, dev, fun, 0x10);
base &= ~1;
ce->iCTL = base;
ce->iSTS = base + 4;
ce->iRBR = base + 8;
ce->sync = sem_ini(0);
ce->mutex = sem_ini(1);
natb irq = pci_read_confb(bus, dev, fun, 0x3c);
apic_set_VECT(irq, 0xa0 + next_ce);
apic_set_MIRQ(irq, false);
apic_set_TRGM(irq, false); // riconscimento sul fronte
```

→ La periferica usa il nuovo Base Address Register (offset 16)

→ LSB ad 1 per spazio di I/O: il resto è indirizzo

→ Primo = base

→ Registro stato

→ RBR

} Semafori utilizzati per gestire la periferica

→ Associazione a richieste di interruzione

↓

0x3C è il punto dove il BIOS scrive a che piedino dell'APIC è collegata la periferica

→ Tipo

primitiva:

```
extern "C" void c_cread_n(natl id, char *buf, natl quanti)
{
        if (id >= MAX_CE) {
                flog(LOG_WARN, "id non valido");
                abort_p();
        }

        if (id >= next_ce) {
                flog(LOG_WARN, "periferica non presente");
                return;
        }

        des_ce *ce = &array_des[id];
        sem_wait(ce->mutex);
        ce->buf = buf;
        ce->quanti = quanti;
        outputb(1, ce->iCTL);
        sem_signal(ce->sync);
        sem_signal(ce->mutex);
}
```

dobbiamo controllare che sia nella parte condivisa, mappata e scrivibile

deve essere in spazio utente + traduzione

⚠ da buf a quanti-1

```
extern "C" bool c_access(vaddr begin, natq dim, bool writeable, bool shared = true)
{
    if (!tab_iter::valid_interval(begin, dim))
        return false;
    if (shared && (!in_utn_c(begin) || (dim > 0 && !in_utn_c(begin + dim - 1))))
        return false;

    // usiamo un tab_iter per percorrere tutto il sottoalbero relativo
    // alla traduzione degli indirizzi nell'intervallo [begin, begin+dim).
    for (tab_iter it(esecuzione->cr3, begin, dim); it; it.next()) {
        tab_entry e = it.get_e();

        // interrompiamo il ciclo non appena troviamo qualcosa che non va
        if (!(e & BIT_P) || !(e & BIT_US) || (writeable && !(e & BIT_RW)))
            return false;
    }
    return true;
}
```

controllo se stia in spazio utente cond: int. valido + indirizzi

↓ modifica cread_n

```
if (!c_access((vaddr)buf, quanti, true, true)) {
        flog(LOG_WARN, "indirizzo non valido");
        abort_p();
}
```

tutti gli indirizzi passati dall'utente devono essere controllati attraverso ⚠ cavallo di Troia

Modulo sistema sua non atomica: ci servono INTR

driver:

```
extern "C" void c_driver_ce(int id)
{
        des_ce *ce = &array_des[id];

        ce->quanti--;

        if (ce->quanti == 0) {
                outputb(0, ce->iCTL);

                des_sem *s = &array_dess[sem];
                s->counter++;

                if (s->counter <= 0) {
                        des_proc* lavoro = rimozione_lista(s->pointer);
                        inspronti();    // preemption
                        inserimento_lista(pronti, lavoro);
                        schedulatore(); // preemption
                }
        }

        char b = inputb(ce->iRBR);
        *ce->buf = b;
        ce->buf++;
}
```

l'ID proviene dal driver chiamante della richiesta di interruzione giusto

e quindi si suppone

⚠ ATT: non possiamo mettere sem_wait perché verrebbe usato il descrittore di P2 da parte del driver. Non sono usare neanche le sem_signal in C perché il controllo sul livello di sempre fallirebbe in quanto usando visto quello di P2 (utente)

disattiva interruzioni

preleviamo byte

e mettiamo un carattere dietro l'altro

Lo mettiamo qua perché non vogliamo che avvenga più richiesta di interruzione nel caso di più byte pronti

↓

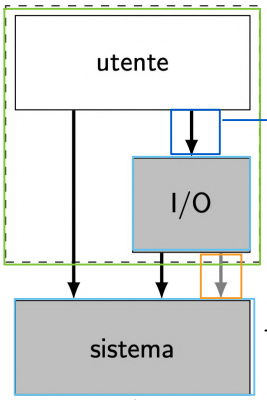
Le interruzioni in questo caso non possono essere annidate

↓

Nuovo modulo che elimina le incomprensioni: permette di avere INTR e invocare chiamate sistema → Permette di trovare più facilmente i BUG di I/O (più frequenti). Possiamo eseguire il driver a livello utente? Dato che lo trasformiamo in un processo, dobbiamo fare in modo che IOPL per questo tipo di processi sia utente. Questo però permette di fare CLI e STI e non ci fidiamo perché stiamo di fatto dando questo privilegio all'utente. → Il modulo verrà eseguito a livello sistema con interruzioni abilitate

↓

Rischio: errori involontari: nuova struttura sistema non disponibili



INTR abilitate

Primitive non atomiche: INTR abilitate: gate di tipo TRAP → no salva_stato e carica_stato

Livello sistema

primitive usabili solo da modulo I/O: DPL a livello sistema

→ Moduli offrono system call a livelli più in alto

Il modulo di I/O parte ed utilizza delle primitive rinascite per installare system call nella IDT di tipo TRAP (analoghe a funzioni già viste in precedenza). Le interruzioni vengono intercettate dall'handler nel modello sistema che mette in esecuzione il processo che permette di eseguire tali funzioni. Il processo è analogo al driver ma non ha accesso alla struttura dati.

```
extern "C" void c_ceread_n(natl id, char *buf, natl quanti)
{
        if (id >= next_ce) {
                flog(LOG_WARN, "ce non riconosciuto: %d", id);
                abort_p();
        }

        if (!access(buf, quanti, true)) {
                flog(LOG_WARN, "buf non valido\n");
                abort_p();
        }

        des_ce *ce = &array_ce[id];
        sem_wait(ce->mutex);
        ce->buf = buf;
        ce->quanti = quanti;
        outputb(1, ce->iCTL);
        sem_wait(ce->sync);
        sem_signal(ce->mutex);
}
```

```
access:
        .cfi_startproc
        int $TIPO_ACC
        ret
        .cfi_endproc
```

```
handler_0:
        .cfi_startproc
        .cfi_def_cfa_offset 40
        .cfi_offset rip, -40
        .cfi_offset rsp, -16
        call salva_stato
        call inspronti

        movq $0, %rcx
        movq a_p(, %rcx, 8), %rax
        movq %rax, esecuzione

        call carica_stato
        iretq
        .cfi_endproc
```

→ Mette processo in testa pronti perché era in esecuzione

→ Forzatamente in esecuzione: priorità I/O > utente
↳ accept costruito prima con processo corrispondente al device

```
extern "C" void estern_ce(int id)
{
        des_ce *ce = &array_ce[id];

        for (;;) {
                ce->quanti--;
                if (ce->quanti == 0) {
                        outputb(0, ce->iCTL);
                }
                char b = inputb(ce->iRBR);
                *ce->buf = b;
                ce->buf++;
                if (ce->quanti == 0) {
                        sem_signal(ce->sync);
                }
                wfi();
        }
}
```

Processo deve avere stessa priorità del driver: non deve mai interrompersi per rispondere a richiesta di INT

SPIEGATO: sem_signal notifica trasferimento tutti dati. Se viene prima richiamata ancora c'è ultimo dato

PRIMITIVA: attende interruzione

```
a_wfi:
        .cfi_startproc
        .cfi_def_cfa_offset 40
        .cfi_offset rip, -40
        .cfi_offset rsp, -16
        call salva_stato
        call apic_send_EOI
        call schedulatore
        call carica_stato
        iretq
        .cfi_endproc
```

: Processo esterno si blocca e ne viene eseguito un altro. Non si salva quello che era in esecuzione perché viene preso dall'array.

modulo I/O (unione video/tastiera)

```
extern "C" void iniconsole(natb cc);
extern "C" natq readconsole(char* buff, natq quanti);
extern "C" void writeconsole(const char* buff, natq quanti);

extern "C" void readhd_n(void* vetti, natl primo, natb quanti);
extern "C" void writehd_n(const void* vetto, natl primo, natb quanti);
extern "C" void dmareadhd_n(void* vetti, natl primo, natb quanti);
extern "C" void dmawritehd_n(const void* vetto, natl primo, natb quanti);

extern "C" natq getiomeminfo();
```

primitive fornite da sistema ad I/O

```
extern "C" natl activate_pe(void f(int), int a, natl prio, natl liv, natb irq);
extern "C" void wfi();
extern "C" void abort_p();
extern "C" void io_panic();
extern "C" paddr trasforma(void* ff);
extern "C" bool access(const void* start, natq dim, bool writeable, bool shared = true);
extern "C" void fill_gate(natl tipo, vaddr f);
```

(crea processo esterno (sostituisce driver)) (piedino apic a cui è connesso)

Parte Amewhile: funzioni del modello utente + specifiche + primitive fornite ad utente non comuni e senza salva stato e carica stato

```
fill_io_gates:
        pushq %rbp
        movq %rsp, %rbp

        fill_io_gate    IO_TIPO_RCON    a_readconsole
        fill_io_gate    IO_TIPO_WCON    a_writeconsole
        fill_io_gate    IO_TIPO_INIC    a_iniconsole
        fill_io_gate    IO_TIPO_HDR     a_readhd_n
        fill_io_gate    IO_TIPO_HDW     a_writehd_n
        fill_io_gate    IO_TIPO_DMAHDR  a_dmareadhd_n
        fill_io_gate    IO_TIPO_DMAHDW  a_dmawritehd_n
        fill_io_gate    IO_TIPO_GMI     a_getiomeminfo

        leave
        ret
```

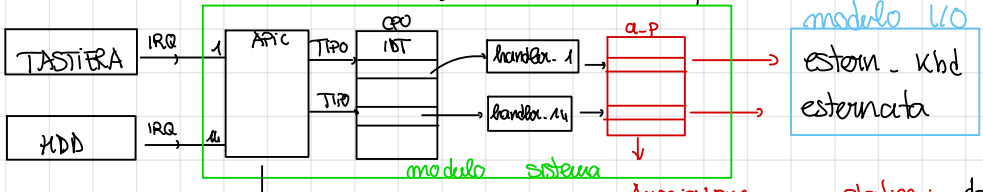
carica gate della IDT per primitive fornite da I/O : primitive TRAP

activate_pe

```
p = crea_processo(f, a, prio, liv);
if (p == 0)
        goto err_out;
```

: stessa di activate_p ma solo liv. sistema

↓ creiamo il collegamento tra processo e interrupt



Associazione non statica: da modulo sistema a modulo I/O

Codifica la precedenza che compare anche nel processo: calcoliamo il tipo in base ad esso → assumiamo che tutti i processi esterni abbiano priorità minima. La priorità totale sarà data da MIN_PRIO_IO + tipo.

Assegnato ad APIC. Nella IDT viene poi inserito l'indirizzo dell'handler corrispondente

```
bool aggiungi_pe(des_proc *p, natw tipo, natb irq)
{
        if (irq >= MAX_IRQ) {
                flog(LOG_WARN, "irq %d non valido (max %d)", irq, MAX_IRQ);
                return false;
        }
        if (a_p[irq]) {
                flog(LOG_WARN, "irq %d occupato", irq);
                return false;
        }
        if (!load_handler(tipo, irq)) {
                flog(LOG_WARN, "tipo %x occupato", tipo);
                return false;
        }
a_p[irq] = p;
apic_set_VECT(irq, tipo);
apic_set_MIRQ(irq, false);
apic_set_TRGM(irq, false);
return true;
```

new e delete: usano la mutua esclusione in quanto le INTR sono abilitate

```
void* operator new(size_t s)
{
        void *p;

        sem_wait(ioheap_mutex);
        p = alloca(s);
        sem_signal(ioheap_mutex);

        return p;
}
```

```
void operator delete(void* p)
{
        sem_wait(ioheap_mutex);
        dealloca(p);
        sem_signal(ioheap_mutex);
}
```

Gestione della console

write console

```
extern "C" void c_writeconsole(const char* buff, natq quanti) //
{
        des_console *p_des = &console;  non importa spazio condiviso: stesso processo di writeconsole

        if (!access(buff, quanti, false, false)) {  → controllo se posso leggere i parametri
                flog(LOG_WARN, "writeconsole: parametri non validi: %p, %d:", b
uff, quanti);
                abort_p();
        }
```

```
        sem_wait(p_des->mutex);
#ifndef AUTOCORR
        for (natq i = 0; i < quanti; i++)
                char_write(buff[i]);
```

```
#else /* AUTOCORR */
        if (buff[quanti - 2] == '\n')
                quanti -= 2;
        flog(LOG_USR, "%.*s", quanti, buff);
#endif /* AUTOCORR */
        sem_signal(p_des->mutex);
}
```

read console

```
extern "C" natq c_readconsole(char* buff, natq quanti) //
{
        des_console *d = &console;
        natq rv;

        if (!access(buff, quanti, true)) {
                flog(LOG_WARN, "readconsole: parametri non validi: %p, %d:", bu
ff, quanti);
                abort_p();
        }
#ifdef AUTOCORR
        return 0;
#endif

        if (!quanti)
                return 0;

        sem_wait(d->mutex);
        startkbd_in(d, buff, quanti);
        sem_wait(d->sincr);
        rv = d->dim - d->cont;
        sem_signal(d->mutex);
        return rv;
}
```

buffer in mem. condivisa perché scritto da processo esterno

```
void startkbd_in(des_console* d, char* buff, natq dim)
{
        d->punt = buff;
        d->cont = dim;
        d->dim  = dim;
        enable_intr_kbd();
}
```

Numero caratteri letti

extern - kbd

```
for(;;) {
        disable_intr_kbd();

        a = char_read_int();
```

Prima di leggere RBR: Non voglio nuovo carattere se buffer è pieno o carattere è INVIO

```
default: → carattere standard
        *d->punt = a;
        d->punt++;
        d->cont--;
        char_write(a);
        if (d->cont == 0) {
                fine = true;
        }
        break;
if (fine)
        sem_signal(d->sincr);
else
        enable_intr_kbd();
wfi();
```

```
case '\n':   a capo
        fine = true;
        *d->punt = '\0';
        str_write("\r\n");
        break;
```

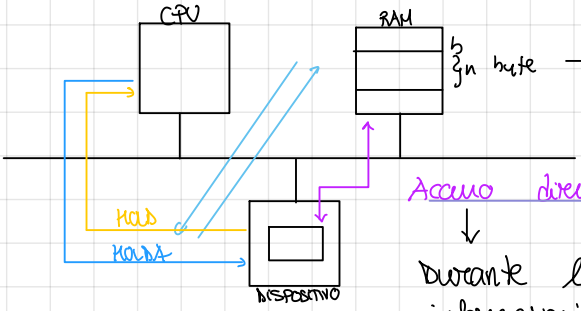
```
case '\b':  cancellazione carattere
        if (d->cont < d->dim) {
                d->punt--;
                d->cont++;
                str_write("\b \b");
        }
        break;
```

```
handler_1:
        .cfi_startproc
        .cfi_def_cfa_offset 40
        .cfi_offset rip, -40
        .cfi_offset rsp, -16
        call salva_stato
        call inspronti

        movq $1, %rcx
        movq a_p(, %rcx, 8), %rax
        movq %rax, esecuzione

        call carica_stato
        iretq
        .cfi_endproc
```

Passo dove da che punto riparterà: la prima volta parte dall'inizio, la seconda da dopo call. Questo accade perché se è partito l'handler, nel frattempo l'APIC ha mandato la richiesta di INTR. Se questo ha mandato l'interrupt, non c'era nessuna richiesta di INTR con la stessa priorità pendente. Una richiesta di INTR è in gestione fin quando il processo esterno è ancora in esecuzione

⚠ Mutua esclusione: L'accesso alla coda viene fatto senza semafori in varie parti. Questo si può fare perché le interruzioni sono di norma disabilitate e vengono abilitate solo dopo l'inizializzazione di coda. In questo momento in poi possono essere eseguiti i processi esterni.

- DMA



$\longrightarrow$ Buffer: [b, b+n[

Accesso diretto RAM - dispositivo

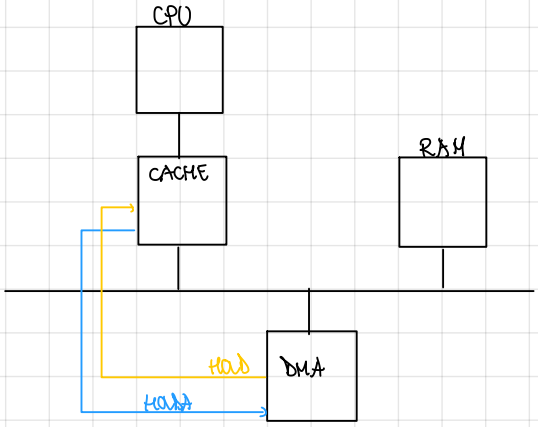
↓

Durante l'accesso la CPU è libera in quanto le informazioni necessarie all'accesso sono comunicate prima

↓

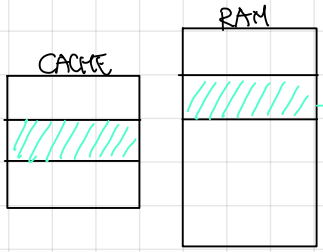
Essendo il bus unico, il dispositivo vi accede con le stesse modalità già usate dalla CPU e deve avere registri per contenere dove informazioni: b, un commutatore per incrementarlo e un contatore per sapere quando uscire. Non è possibile avere due dispositivi che pilotano le stesse linee e quindi introducono i fili HOLD e HOLDA con i quali viene fatto un hs. Utilizzando HOLD richiede l'accesso al BUS alla CPU. Per concedere il controllo deve mettere le uscite in Hiz e attivare HOLDA. Alla fine, il dispositivo disattiva HOLD e mette le linee in Hiz. La CPU dà precedenza al dispositivo e quindi si fa "rubare" cicli anche in mezzo ad istruzioni. La CPU può continuare a fare delle operazioni nel mentre. Vantaggio nel caso in cui l'operazione di ingresso debba essere eseguita il prima possibile (scheda di rete).

Aggiungiamo la cache



⟶ Cade l'ipotesi fondamentale in cui si basa la cache: la RAM può comunicare anche se la CPU non ci vuole sopra
↓ Caso diverso a seconda di:
a. Cache write through
b. Cache write back
I. Trasferimenti completi di cacheline: il dispositivo fa operazioni di r/w di una cella cacheline intera
II. Trasferimenti parziali di cacheline

Caso a.



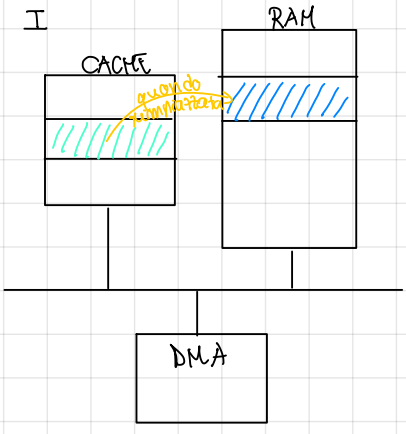
⟶ Se la cacheline è in cache lo stesso contenuto in RAM

output: Non c'è nessun problema perché il contenuto che il DMA legge è già in RAM

input: Problemi ⟶ il SW vede la vecchia versione dei dati.
Soluzione hw: Mentre il DMA scrive in RAM il controllore cache è in ascolto sulle linee del bus. Le linee di controllo gli comunicano l'operazione di scrittura e legge le linee di indirizzo accedendo alla RAM come se questo fosse fatto dalla CPU e se c'è hit invalida l'entrata
⚠ se SW non accede al buffer durante questa operazione
Soluzione SW: ↗ programmatori hanno a disposizione istruzioni per invalidare la cache ⟶ le usano per invalidare gli indirizzi $[b, b+n[$ prima o dopo l'operazione (tipicamente dopo).

Caso b.

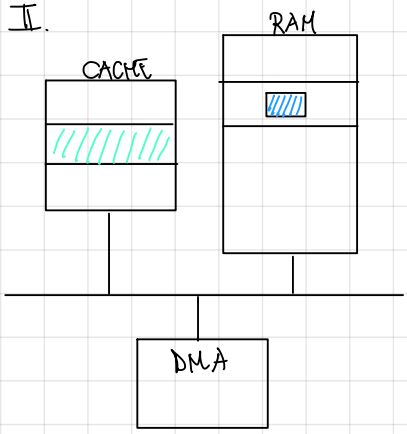
I



output: Problema. Se parti del buffer si trovano in cache il DMA legge dati vecchi. Come nell'input del caso a il controllore cache può fare snooping e accorgersi dell'operazione. Quando le istruzioni software vi possono scrivere i contenuti del buffer della CACHE della RAM e questo deve essere fatto prima. Dal punto di vista hw deve essere il dispositivo a fare snooping: deve chiedere al controllore cache se la cacheline interessata è dirty. Se sì, o il controllore cache scrive in RAM, o passa direttamente i dati al dispositivo se li ha

input: Può essere fatto uno snooping passivo come prima al controllore cache che però non deve scrivere in RAM durante l'operazione. Nel caso del software, la cacheline deve essere invalidata prima di eseguire l'operazione.

II.

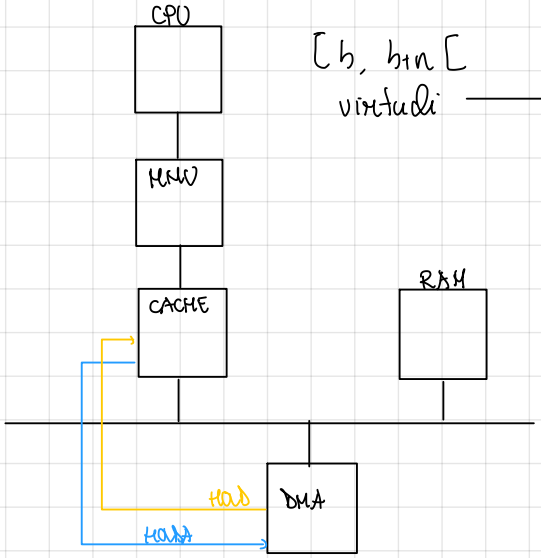


output: snooping da parte del dispositivo come I.
input: lo snooping del controllore è valido solo se è in grado di prelevare i dati ed aggiornare la cacheline. Non può invalidarla perché dovrebbe essere cambiata tutta. Se non è in grado di farlo è il dispositivo che deve fare snooping. Se deve fare trasferimento parziale e chiede al controllore cache se è dirty. Se sì, il controllore fa il write back e poi il dispositivo copierà solo la parte desiderata oppure fino la cacheline, il DMA scrive la parte intercettata e poi scrive in RAM.

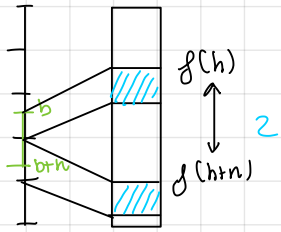
perché facciamo trasf. parziali? ↓

+ modo in cui dati escono dal dispositivo (in genere non riempiono cacheline)

Aggiungiamo la MMU



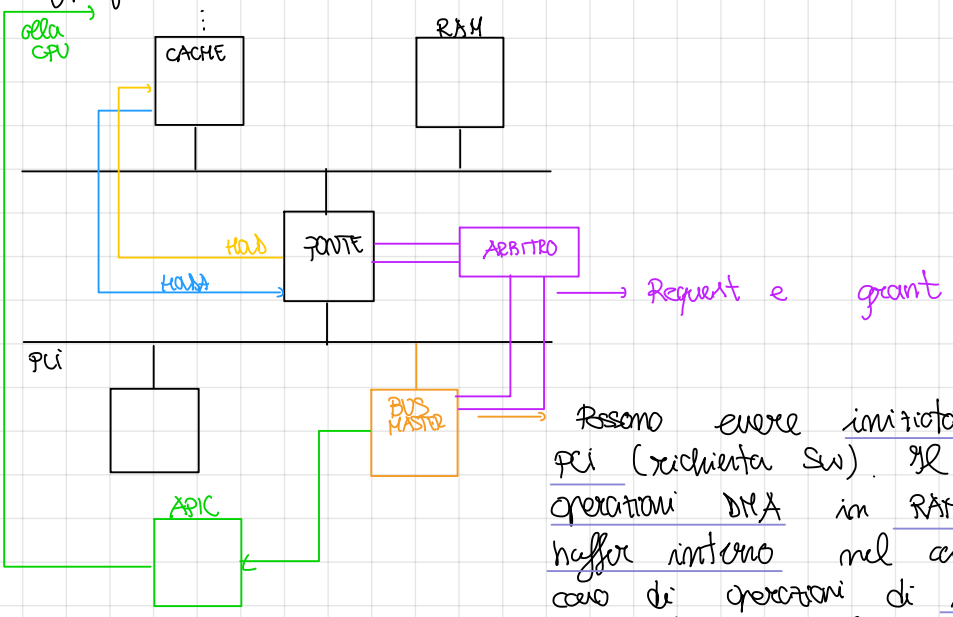
[b, b+n[
virtuali → DMA conosce solo indirizzi fisici
↓ Conseguente
1. Quando si comunicano dati a dispositivo per avviare operazione gli va comunicato f(b) [traduzione]
2. Dobbiamo comunicare sempre tutto n



2 trasferimenti: da inizio a fine di ciascun frame

Alcuni dispositivi possono ricevere una lista di trasferimenti da fare. Anche se ciò non è supportato, comunque il problema deve essere risolto via sw
3. La traduzione degli indirizzi virtuali non deve cambiare mentre è in corso il trasferimento: possibile in caso di swap-in e swap-out dei processi.

Aggiungiamo il bus PCI



→ Request e grant

Possono essere iniziatori di transazioni sul bus PCI (richiesta sw). Il ponte è obiettivo e fa da operazioni DMA in RAM mettendo prima i dati in un buffer interno nel caso di operazioni di ingresso. Nel caso di operazioni di uscita, il dispositivo PCI fa operazioni di lettura nello spazio PCI ed il ponte, obiettivo,

legge i dati della RAM e risponde al dispositivo con tali dati

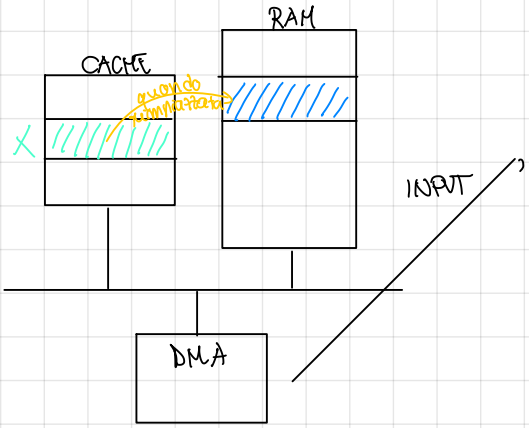
Come interagiscono con le interruzioni? Il dispositivo, al termine dell'operazione, inoltra una richiesta di interruzione all'APIC che a sua volta la inoltra alla CPU. Ciò però comunica solo che i dati sono arrivati al ponte e non alla RAM ⟶ è possibile che l'INTR venga accettata prima che i dati arrivino in RAM

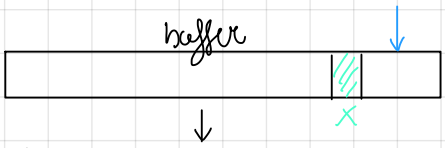
↓ Soluzioni

SW: proprietà ponte: gestisce in ordine FIFO il trasferimento dati dal bus PCI. Quando arriva l'INTR il SW può leggere un registro del dispositivo creando una transazione che si accoda alle altre. Quando questa operazione termina notifica la fine del trasferimento dei dati precedenti. Non è un costo in più perché il driver avrebbe già dovuto leggere un registro del dispositivo a seguito dell'INTR in quanto il dispositivo si aspettava una risposta.

hw: Handshake tra apic e ponte ⟶ Prima di inoltrare richiesta chiede OK al ponte che glielo concede solo dopo aver trasferito i dati che aveva fino a quel momento. ⟶ Rallenta le interruzioni anche se non serve.

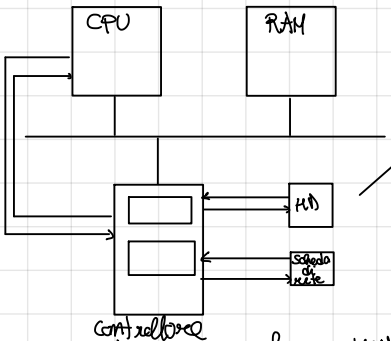
⚠





È possibile che il controllore cache faccia il write-back prima di scrivere a X durante il DMA? Sì perché può avvenire rimpiazzamento. Può accadere che arrivi a freccia prima di sub appare nel secondo che viene tutto sovrascritto e quindi va bene

Situazione pre-PCI



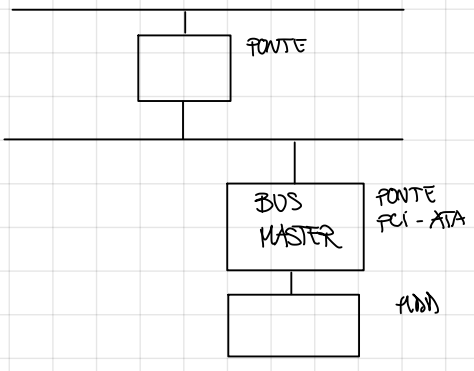
Viene comunicato al controllore dove trasferire e poi c'è un handshake, ad esempio con l'hard disk, che comunica quando è pronto a trasferire. A questo punto il controllore si impadronisce del bus passando indirizzo e controllo e l'hard disk passa i dati

↓

⟶ fa HOLD/HOLDA con la cache

Per compatibilità anche oggi l'hard disk pensa di parlare con il controllore DMA che però in realtà è il ponte pci-ata

↓



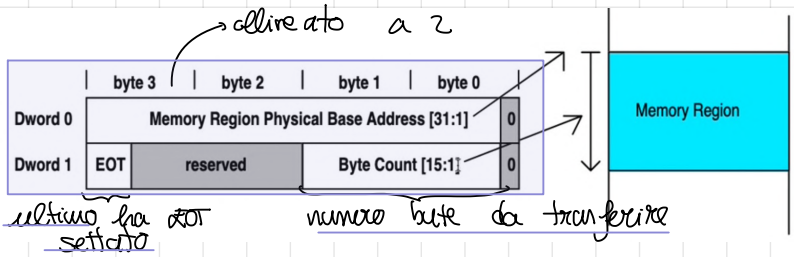
Le informazioni su dove leggere e quanti byte trasferire vanno dette al ponte pci-ATA ed il resto delle informazioni (nr settori e quali settori) vanno dette all'HDD. Per gestire le interruzioni l'APIC fa un bis con ponte e ponte pci-ATA

Come programmare il bus-master

Il bus master può eseguire più operazioni una dietro l'altra e manda un'interruzione alla fine di tutte

↓

Servono dei descrittori in memoria (array) e poi si deve comunicare l'indirizzo dell'array



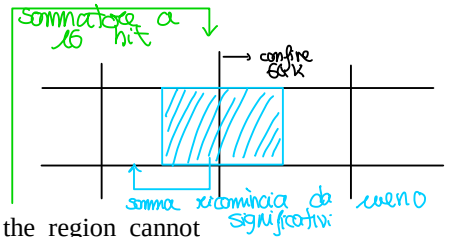
Registri del dispositivo:

The bus master IDE function uses 16 bytes of IO space. All bus master IDE IO space registers can be accessed as byte, word, or Dword quantities. The description of the 16 bytes of IO registers follows:

Registri Primary e Secondary per HDD parallel ATA ←

Offset from Base Address	Register	Register Access
00h	Bus Master IDE Command register Primary	R/W
01h	Device Specific	
02h	Bus Master IDE Status register Primary	RWC
03h	Device Specific	
04h-07h	Bus Master IDE PRD Table Address Primary	R/W
08h	Bus Master IDE Command register Secondary	R/W
09h	Device Specific	
0Ah	Bus Master IDE Status register Secondary	RWC
0Bh	Device Specific	
0Ch-0Fh	Bus Master IDE PRD Table Address Secondary	R/W

Vedi sotto per la descrizione dei registri, operazioni ed errori e specifiche PCI



La regione non può attraversare un confine di 64K

Note

The memory region specified by the descriptor is *further* restricted such that the region cannot straddle a 64K boundary. This means that the byte count can be limited to 64K, and the incrementer for the current address register need only extend from bit [1] to bit [15]. Also, the total sum of the descriptor byte counts must be equal to, or greater than the size of the disk transfer request. If greater than, then the driver must terminate the Bus Master transaction (by resetting bit zero of the command register to zero) when the drive issues an interrupt to signal transfer completion.

2.0. Bus Master IDE Register Description

The bus master IDE function uses 16 bytes of IO space. All bus master IDE IO space registers can be accessed as byte, word, or Dword quantities. The description of the 16 bytes of IO registers follows:

Offset from Base Address	Register	Register Access
00h	Bus Master IDE Command register Primary	R/W
01h	Device Specific	
02h	Bus Master IDE Status register Primary	RWC
03h	Device Specific	
04h-07h	Bus Master IDE PRD Table Address Primary	R/W
08h	Bus Master IDE Command register Secondary	R/W
09h	Device Specific	
0Ah	Bus Master IDE Status register Secondary	RWC
0Bh	Device Specific	
0Ch-0Fh	Bus Master IDE PRD Table Address Secondary	R/W

2.1. Bus Master IDE Command Register

Register Name: Bus Master IDE Command Register
Address Offset: Primary Channel: Base + 00h
Secondary Channel: Base + 08h
Default Value: 00h
Attribute: Read / Write
Size: 8 bits

Bit	Description
7:4	**Reserved.** Must return 0 on reads.
3	**Read or Write Control:** This bit sets the direction of the bus master transfer: when set to zero, PCI bus master reads are performed. When set to one, PCI bus master writes are performed. This bit must NOT be changed when the bus master function is active.
2:1	**Reserved.** Must return 0 on reads.
0	**Start/Stop Bus Master:** Writing a '1' to this bit enables bus master operation of the controller. Bus master operation begins when this bit is detected changing from a zero to a one. The controller will transfer data between the IDE device and memory only when this bit is set. Master operation can be halted by writing a '0' to this bit. All state information is lost when a '0' is written; Master mode operation cannot be stopped and then resumed. If this bit is reset while bus master operation is still active (i.e., the Bus Master IDE Active bit of the Bus Master IDE Status register for that IDE channel is set) and the drive has not yet finished its data transfer (The Interupt bit in the Bus Master IDE Status register for that IDE channel is not set), the bus master command is said to be aborted and data transfered from the drive may be discarded before being written to system memory. This bit is intended to be reset after the data transfer is completed, as indicated by either the Bus Master IDE Active bit or the Interrupt bit of the Bus Master IDE Status register for that IDE channel being set, or both.

2.2. Bus Master IDE Status Register

Register Name: Bus Master IDE Status Register
Address Offset: Primary Channel: Base + 02h
Secondary Channel: Base + 0Ah
Default Value: 00h
Attribute: Read/Write Clear
Size: 8 bits

Bus Master IDE Status Register

Bit	Description
7	**Simplex only:** This read-only bit indicates whether or not both bus master channels (primary and secondary) can be operated at the same time. If the bit is a '0', then the channels operate independently and can be used at the same time. If the bit is a '1', then only one channel may be used at a time.
6	**Drive 1 DMA Capable:** This read/write bit is set by device dependent code (BIOS or device driver) to indicate that drive 1 for this channel is capable of DMA transfers, and that the controller has been initialized for optimum performance.
5	**Drive 0 DMA Capable:** This read/write bit is set by device dependent code (BIOS or device driver) to indicate that drive 0 for this channel is capable of DMA transfers, and that the controller has been initialized for optimum performance.
4:3	**Reserved.** Must return 0 on reads.
2	**Interrupt:** This bit is set by the rising edge of the IDE interrupt line. This bit is cleared when a '1' is written to it by software. Software can use this bit to determine if an IDE device has asserted its interrupt line. When this bit is read as a one, all data transfered from the drive is visible in system memory.
1	**Error:** This bit is set when the controller encounters an error in transferring data to/from memory. The exact error condition is bus specific and can be determined in a bus specific manner. This bit is cleared when a '1' is written to it by software.
0	**Bus Master IDE Active:** This bit is set when the Start bit is written to the Command register. This bit is cleared when the last transfer for a region is performed, where EOT for that region is set in the region descriptor. It is also cleared when the Start bit is cleared in the Command register. When this bit is read as a zero, all data transfered from the drive during the previous bus master command is visible in system memory, unless the bus master command was aborted.

2.3. Descriptor Table Pointer Register

Register Name: Descriptor Table Pointer Register
Address Offset: Primary Channel: Base + 04h
Secondary Channel: Base + 0Ch
Default Value: 00000000h
Attribute: Read / Write
Size: 32 bits

Descriptor Table Pointer Register

Bit	Description
31:2	Base address of Descriptor table. Corresponds to A[31:2]
1:0	reserved

The Descriptor Table must be Dword aligned. The Descriptor Table must not cross a 64K boundary in memory.

3.0. Operation

3.1. Standard Programming Sequence

To initiate a bus master transfer between memory and an IDE DMA slave device, the following steps are required:

1) Software prepares a PRD Table in system memory. Each PRD is 8 bytes long and consists of an address pointer to the starting address and the transfer count of the memory buffer to be transferred. In any given PRD Table, two consecutive PRDs are offset by 8-bytes and are aligned on a 4-byte boundary.
2) Software provides the starting address of the PRD Table by loading the PRD Table Pointer Register . The direction of the data transfer is specified by setting the Read/Write Control bit. Clear the Interrupt bit and Error bit in the Status register.
3) Software issues the appropriate DMA transfer command to the disk device.
4) Engage the bus master function by writing a '1' to the Start bit in the Bus Master IDE Command Register for the appropriate channel.
5) The controller transfers data to/from memory responding to DMA requests from the IDE device.
6) At the end of the transfer the IDE device signals an interrupt.
7) In response to the interrupt, software resets the Start/Stop bit in the command register. It then reads the controller status and then the drive status to determine if the transfer completed successfully.

3.1. Data Synchronization

When reading data from an IDE device, that data may be buffered by the IDE controller before using a master operation to move the data to memory. The IDE device driver in conjunction with the IDE controller is responsible for guaranteeing that any buffered data is moved into memory before the data is used.

The IDE device driver is required to do a read of the controller Status register after receiving the IDE interrupt. If the Status register returns with the Interrupt bit set then the driver knows that the IDE device generated the interrupt (important for shared interrupts) and that any buffered data has been flushed to memory. If the Interrupt bit is not set then the IDE device did not generate the interrupt and the state of the data buffers is unknown.

When the IDE controller detects a rising edge on the IDE device interrupt line (INTRQ) it is required to:

- Flush all buffered data
- Set the Interrupt bit in the controller Status register
- Guarantee that a read to the controller Status register does not complete until all buffered data has been written to memory.

Another way to view this requirement is that the first read to the controller Status register in response to the IDE device interrupt must return with the Interrupt bit set and with the guarantee that all buffered data has been written to memory.

3.1. Status Bit Interpretation

The table below gives a description of how to interpret the Interrupt and Active bits in the Controller status register after a DMA transfer has been started.

Interrupt	Active	Description:
0	1	DMA transfer is in progress. No interrupt has been generated by the IDE device.
1	0	The IDE device generated an interrupt. The controller exhausted the Physical Region Descriptors. This is the normal completion case where the size of the physical memory regions was equal to the IDE device transfer size.
1	1	The IDE device generated an interrupt. The controller has not reached the end of the physical memory regions. This is a valid completion case where the size of the physical memory regions was larger than the IDE device transfer size.
0	0	This bit combination signals an error condition. If the Error bit in the status register is set, then the controller has some problem transferring data to/from memory. Specifics of the error have to be determined using bus-specific information. If the Error bit is not set, then the PRD's specified a smaller size than the IDE transfer size.

4.0. Error Conditions

IDE devices are sector based mass storage devices. The drivers handle errors on a sector by sector basis; either a sector is transferred successfully or it is not.

If the IDE DMA slave device never completes the transfer due to a hardware or software error, the Bus Master IDE command will eventually be stopped (by setting Command Start bit to zero) when the driver times out the disk transaction. Information in the IDE device registers will help isolate the cause of the problem.

If the controller encounters an error while doing the bus master transfers it will stop the transfer (ie. reset the Active bit in the Command register) and set the ERROR bit in the Status register. The controller does not generate an interrupt when this happens. The device driver can use device specific information (e.g.; PCI Configuration Space Status register) to determine what caused the error.

Whenever a requested transfer does not complete properly, information in the IDE device registers (Sector Count) can be used to determine how much of the transfer was completed and to construct a new PRD table to complete the requested operation. In most cases the existing PRD table can be used to complete the operation.

5.0. PCI Specifics

Bus master IDE controllers built to attach to a PCI bus must have the following characteristics:

1) The Class Code in PCI configuration space indicates IDE device (top two bytes have the value 0x0101) and bit 7 of the Programming Interface register (offset 0x09) in PCI configuration space must be set to 1 to indicate that the device supports the Master IDE capability.
2) The control registers for the controller are allocated via the devices Base Address register at offset 0x20 in PCI configuration space.
3) In the controller Status register the Error bit will be set and the Active bit reset if any of the following conditions occur on the PCI bus while the controller is doing a master operation on the bus. The exact cause can be determined by examining the Configuration Space Status register.

Error Condition	Configuration Space Status bits
Target Abort	Anytime bit 12 of the Config Space Status register is set.
Master Abort	Anytime bit 13 of the Config Space Status register is set.
Data Parity Error Detected	Anytime bit 8 of the Config Space Status register is set.

ESEMPIO: trasferimento in ingresso

```
volatile bool done = false;
extern char vv[]; buffer
const natl BUFSIZE = 65536;
extern natl prd[];
descrittori
extern "C" void a_bmide();
extern "C" void c_bmide()
```

```
prd:
        .fill 16384, 4
.balign 65536
```
→ allineato a 64K → Se lo togliamo e proviamo a scrivere la somma fa overflow e ritorna all'inizio della regione di 64K (che non c'entra nulla con il buffer voluto)

↓

Per risolvere il problema il trasferimento viene fatto in 2 pezzi

```
int main()
{
        natb nn = BUFSIZE / 512; → numero settori
        natb lba = 0; → logical block address partenza
        natb bus = 0, dev = 0, fun = 0;
        numero di bus, device, funzione bus master
        clear_screen(0x0f);
                → cerchiamo bus, dev, find
        if (!bm_find(bus, dev, fun))
                printf("bm non trovato!\n");
        printf("PCI-ATA at %2x:%2x:%2x\n", bus, dev, fun);
        bm_init(bus, dev, fun); → inizializzazione

        apic_set_VECT(14, HD_VECT);
        gate_init(HD_VECT, a_bmide);
        apic_set_MIRQ(14, false);
                } Per intercettare HDD (14)

        for (int i = 0; i < BUFSIZE; i++)
                vv[i] = '-';

        printf("primi 80 caratteri di vv:\n");
        for (int i = 0; i < 80; i++)
                char_write(vv[i]);
        printf("ultimi 80 caratteri di vv:\n");
        for (int i = BUFSIZE - 80; i < BUFSIZE; i++)
                char_write(vv[i]);
```

```
extern "C" void a_bmide();
extern "C" void c_bmide()
{
        done = true; → flag globale
        bm_ack(); bus master
 hdd    hd_ack_intr();
 apic   apic_send_EOI();
}
```

```
prd[0] = reinterpret_cast<natq>(vv);
prd[1] = 0x80000000 | ((nn * 512) & 0xFFFF);
bm_prepare(reinterpret_cast<natq>(prd), false);
```

Prepariamo prd (vv indirizzo fisico)

Prendiamo nr byte e scegliamo solo i primi 16 settando bit più significativo

↓

```
void bm_prepare(natq prd, bool write)
{
        outputl(prd, iBMDTPR);
        natb work = inputb(iBMCMD);
        if (write) {
                work &= ~0x8;
        } else {
                work |= 0x8;
        }
        outputb(work, iBMCMD);
        work = inputb(iBMSTR);
        work |= 0x6;
        outputb(work, iBMSTR);
}
```

→ fa le cose richieste dalla documentazione per l'inizializzazione

```
hd_enable_intr();
hd_start_cmd(lba, nn, READ_DMA);
bm_start();
```

↓ Punto 4 documentazione

```
void bm_start()
{
        natb work = inputb(iBMCMD);
        work |= 1;
        outputb(work, iBMCMD);
}
```

```
printf("aspetto l'interrupt...\n");
while (!done)
        ;

printf("primi 80 caratteri di vv:\n");
for (int i = 0; i < 80; i++)
        char_write(vv[i]);

printf("ultimi 80 caratteri di vv:\n");
for (int i = BUFSIZE - 80; i < BUFSIZE; i++)
        char_write(vv[i]);
```

Per fare il trasferimento in più passaggi detto prima 2 casi di regione non allineata:

```
//      primo_tratto
//       |------|
// +-----------+------------+
// |    XXXXXXX|XXXX        |
// +-----------+------------+
//       |<----vv-->|
//
// +-----------+------------+
// |    XXXX---|            |
// +-----------+------------+
//       |vv|
```

Caso 1

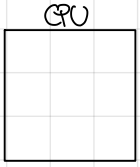
```
prd[0] = reinterpret_cast<natq>(vv);
prd[1] = 0x00000000 | (primo_tratto & 0xFFFF);
prd[2] = tmp2 + 65536;
prd[3] = 0x80000000 | (BUFSIZE - primo_tratto);
```

```
natq tmp1 = reinterpret_cast<natq>(vv);
natq tmp2 = tmp1 & ~((1U<<16)-1);
```

inizio regione che contiene tmp2

La CPU moderna

- VELOCIZZAZIONE delle OPERAZIONI



Legge Dennard Scaling: aumentando la frequenza aumenta la potenza (e quadrato) e riducendo la dimensione dei transistor l'area diminuisce (sempre al quadrato) Oggi non è più possibile aumentare la frequenza perché non è possibile dissipare la potenza → si fanno fare più cose per clock: esecuzione in pipelining o speculativa

pipelining
↓ fasi esecuzione istruzione

preleva	decodifica	preleva operandi	esecuzione	scrittura risultato

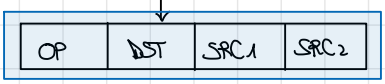
I circuiti che fanno preleva e decodifica non fanno niente qua: li uso per la prossima istruzione

↓

Posso dividere ulteriormente questa cosa allargandola a tutte le singole fasi: creo una catena di istruzioni → è più veloce non dal pdv della singola istruzione ma globalmente in quanto i tempi delle istruzioni si sovrappongono e migliora anche il numero di istruzioni completate al secondo

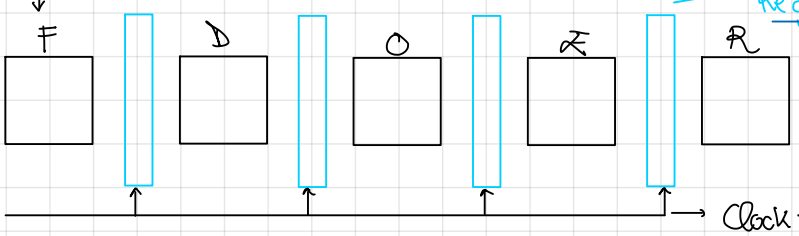
↓ Come fare

È possibile prelevare l'istruzione i+1 mentre si decodifica la i? No perché le istruzioni hanno dimensione diversa e possiamo saperla solo dopo la decodifica. Inoltre, il meccanismo funziona solo se le istruzioni sono regolari, ovvero fanno cose soltanto in determinati punti della loro esecuzione. Questo non accade nei processori intel in quanto le istruzioni non sono pensate per l'esecuzione in pipeline → scelti architetture RISC contrapposte ad (Intel (Sisc). Nelle RISC si fanno solo LOAD e STORE in memoria e poi si lavora sui registri: OP DST, SRC1, SRC2 (DST, SRC1, SRC2 = Registri). I salti inoltre sono del tipo OP REG, offset e non vi hanno sui flag. Le istruzioni inoltre sono sempre lunghe 4 byte e strutturate come segue



I bit che nominano i registri sono inoltre sempre nella stessa posizione. Queste cose risolvono il problema di fare il prelievo di i+1 durante la decodifica di i. La regolarità e la semplicità aiuta inoltre la pipeline

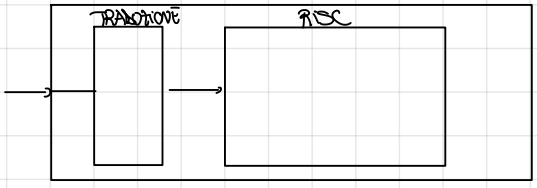
↓



→ Registri di pipeline: memorizzano risultati intermedi

→ Clock: tempo maggiore tra registro e successivo → Affinché il pipelining sia efficace le istruzioni devono fare poco

Nell'Intel



: la traduzione utilizza registri di appoggio

↓ Esempio

ADD % RAX, 1000 (% RCX, % RBX, 4)

diventa

```
SHL  R3, %RCX, $4
ADD  R2, %RBX, R3
LD   R1, 1000(R2)
ADD  %RAX, %RAX, R1
```
} Istruzioni elementari di tipo RISC

Naturalmente conviene se ogni istruzione intel corrisponde ad una sola istruzione RISC

Ma come lavora il traduttore? Fa una prefetch in cui preleva 16 byte della memoria a regioni naturali e su questi 16 byte decodifica le istruzioni una alla volta tradducendole e passandole alla pipeline. Inoltre ha anche una cache delle traduzioni ed il compilatore la utilizza per ottimizzare. Nel fare una pipeline bisogna stare attenti alle dipendenze: avvicinando troppo l'esecuzione delle istruzioni possono crearsi inconvenienti rispetto a come il processore le avrebbe eseguite una dietro l'altra oppure situazioni in cui non è possibile prelevare una nuova istruzione ad ogni clock. Esistono 3 tipologie di dipendenze:

- **strutturali**: 2 istruzioni nella pipeline vogliono usare la stessa risorsa
- **dati**: un'istruzione ha bisogno di dati di un'altra istruzione non ancora disponibili
- **controllo**: per sapere quale è l'istruzione successiva ad una di salto si deve essere aspettati all'esecuzione

Risoluzione: La correttezza del programma è assicurata mettendo in stallo qualche unità della pipeline cioè fermando un'istruzione e le precedenti e facendo andare avanti le altre. Questo risolve le dipendenze strutturali, quelle dati e di controllo. Ogni stallo ci fa perdere un clock ed il massimo tentiamo della situazione sequenziale. Le dipendenze dati possono essere risolte anche tramite circuiti di bypass che permettono ad esempio di riportare indietro dati prodotti utili per istruzioni successive. Per le dipendenze di controllo si fa branch prediction, cercando quindi di indovinare il risultato → dal 15 al 25% delle istruzioni in un programma sono salti. Si utilizza una **predizione statica**: se un salto è all'indietro probabilmente è un ciclo probabilmente eseguito molte volte. Se il salto è in avanti probabilmente è un errore. Quello che è importante è che con la predizione statica le istruzioni vengono sempre predette allo stesso modo. Per fare meglio si fa una predizione dinamica utilizzando una sorta di cache indirizzata facendo hash dell'indirizzo dell'istruzione. Per ogni istruzione di salto ci si ricorda una serie di bit che sono il ricordo del comportamento dell'istruzione in passato. Si utilizzano più bit per memorizzare più volte precedenti e sbagliare di meno le predizioni. A differenza della cache dei dati non utilizziamo i tag perché la previsione può essere sbagliata in ogni caso, sia nel caso in cui è informazione sia relativa ad un'altra istruzione, sia nel caso in cui la predizione sia sbagliata. Quando la predizione è sbagliata nella pipeline ci sono istruzioni prelevate a causa della predizione sbagliata. Queste vanno cancellate e prelevate quelle giuste. L'annullamento consiste nel marcarle e basta in quanto non hanno ancora salvato il risultato in modo tale che al momento del salvataggio del risultato questo venga scartato. Il problema della pipeline sono gli stalli in quanto coinvolgono tutte le istruzioni e non solo quelle che li provocano. Gli stalli inoltre sono spesso provocati dalle istruzioni che interagiscono con la memoria che non trovano i dati interessanti in cache. Questo provoca stalli di centinaia di clock. La cache

di livello 1 è divisa in istruzioni e dati altrimenti ci sarebbero sempre aree strutturali perché le istruzioni di load ci troverebbero sempre in loro strutturale con le altre perché devono leggere dati che vorrebbero sempre nella stessa cache

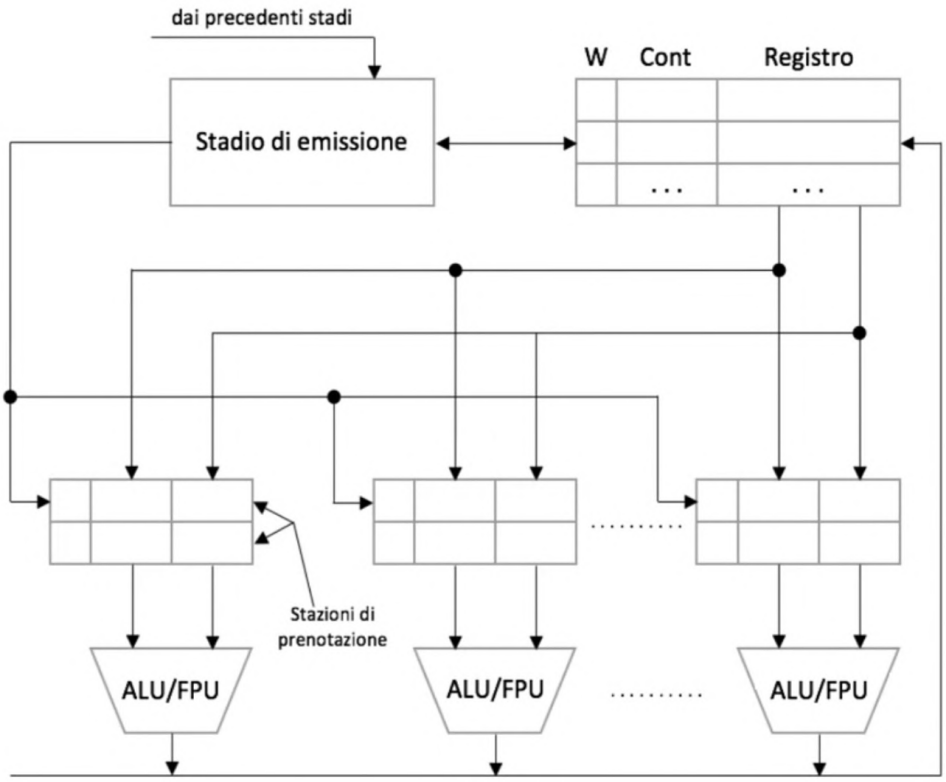
↓ come velocizziamo? È sempre necessario mandare tutto in stallo?

```
for (int i = 0; i < 10000; i++)
        v1[i] = v2[i] + v3[i];
```

È necessario mandare tutto in stallo se un V2[i] non è in cache? NO. È necessario scoprire quali istruzioni non lo necessitano in maniera automatica ⟶ è compito del processore che utilizza la branch prediction per ipotizzare i risultati dei cicli precedenti non eseguiti. La tecnica viene usata inoltre per eseguire alcune istruzioni prima di sapere se ce ne fosse necessità ⟶ esecuzione **speculativa** (ad esempio ipotizzando il risultato di una condizione in un salto)

↓

Fuori ordine: pensiamo al primo momento utile in cui possiamo eseguire un'istruzione ed eseguiamola senza considerare l'ordine in cui si trova nel programma. Organizziamo quindi la pipeline in un modo diverso



⟶ Affianchiamo tante ALU con una stazione di prenotazione ciascuna davanti: memorizzano le istruzioni elementari con gli operandi. Sostituiscono prelievo operandi, esecuzione e scrittura risultato. Aggiungiamo però lo stato di emissione che riceve le istruzioni da prelievo e decodifica: capisce quando un'istruzione può essere inserita in una stazione di prenotazione perché da questo momento in poi verrà prima o poi eseguita quando gli operandi e la sua ALU sono pronti.

↓

Per capire quando gli operandi sono pronti associamo ad ogni registro un flag W che ci dice se c'è un'istruzione che deve scrivere nel registro. Se è 0 indica che gli operandi sono pronti. Tutte le abbiamo più ALU ed abbiamo garantito di leggere gli stessi dati del caso sequenziale, non importa più l'ordine

↓ vanno gestite le

Dipendenze tra le istruzioni (se gestite male generano due)

→ 1. **DATI**: istruzioni i e j con i che viene prima di j nel flusso di esecuzione sequenziale. C'è dipendenza sui dati se i produce un risultato che serve a j (ADD %RAX, %RBX e SUB %RCX, %RAX)

→ 2. **NOMI**: a. **ANTIDIPENDENZE** ⟶ j scrive su un generico registro Rx e i legge Rx

b. **DIPENDENZE di USCITA** ⟶ j scrive sul Rx e i scrive anch'esso sul Rx

j \ i	R	W
R	–	·
W	·	·

- Dipendenza vera dati
- Anti dipendenza
- Dipendenza di uscita

↓

le dipendenze vere nomi possono creare dlee: ADD R1, R2, R3 } anti dip.
SUB R2, R4, R5

↓

Se eseguo prima la SUB e poi la ADD ottengo un risultato errato

3. CONTROLLO: Je dopo

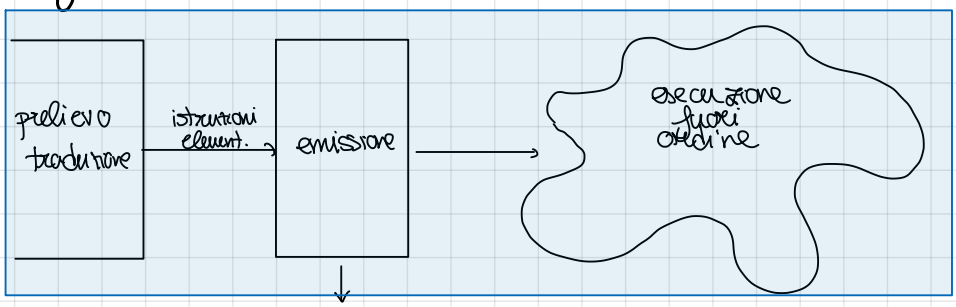
altre istruzioni → Eseguite in base alla decisione della Jump: dipendono da cosa succede in tutto il programma (dipendono per controllo)

dopo: ...

↓

Il processore non può scoprirlo perchè dovrebbe analizzare tutto il programma → le considera dipendenti per controllo da dopo la jump

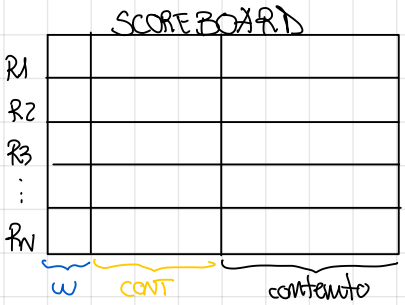
organizzazione intera



↓

Riconosce dip. e risolve dlee. Se non sono possibili altri modi si utilizzano gli stalli

↓ riconoscimento dipendenze



→ Gestita dallo stato di emissione.

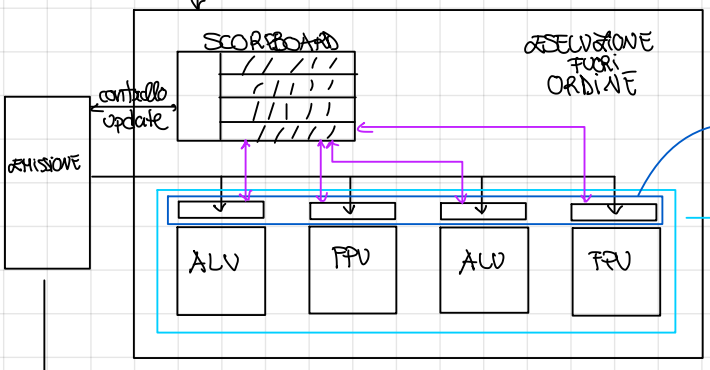
↓ → quante istruzioni vogliono leggere dal registro

I se nell'esecuzione c'è un'istruzione che vuole scrivere nel registro

↓ Come vengono riconosciute (ricevo J)

a. Dipendenze vere dati: J: OP R1, R2, R3 → dipendenza se W di R2, R3 = 1
b. Dipendenze vere nomi: Si guarda il registro di uscita e si vede se W/cont ≠ 0
c. Dipendente sul controllo: se c'è istruzione di controllo nello stato di emissione

↓



Stazioni di prenotazione: se le code si riempiono si ha un'ulteriore strutturale

→ più ALU / FPU: ulteriori già risolte

↓

Metodo più semplice rinominare: Stallo → programma si ferma ogni volta che c'è una dipendenza. Questo accade raramente e quindi ci accorgiamo di rinominare dovuto le dipendenze sui dati. Inoltre quindi istruzioni anche se dipendono sui dati da altre istruzioni emesse non ancora terminate. Perciò colleghiamo le stazioni di prenotazione alla scoreboard in quanto aggiornano il risultato nel registro ed aggiornano anche gli altri campi. L'istruzione appena emessa va nella stazione di prenotazione e viene eseguita solo quando i sorgenti sono pronti (monitorando costantemente la scoreboard). È possibile inoltre scrivere quale stazione produrrà il risultato in modo da poter bypassare i registri. Per quanto riguarda le dipendenze sui nomi, queste possono essere eliminate senza alterare la semantica del programma → L'hardware deve cambiare registro: Algoritmo di **Tomasulo** → Ho la scoreboard e l'istruzione con 2 sorgenti. I sorgenti già pronti li leggo subito e li metto nella stazione di prenotazione. Quelli non pronti ricordo la stazione di prenotazione che produrrà il risultato → Risolve tutte le dipendenze sui nomi perché prende subito il valore già pronto o no da dove scriverà il valore. È possibile, in alternativa, utilizzare la tecnica di **rinominazione dei registri** → Si distingue in logici, usati dalle istruzioni, e registri fisici, usati dalla scoreboard. La corrispondenza tra questi è codificata la corrispondenza viene cambiata ogni volta che un'istruzione vuole scrivere su quel registro e gli operandi sorgenti vengono introdotti con tale corrispondenza. I registri fisici possono essere molti senza codificare istruzioni ed il loro aumento non richiede la modifica dei programmi. Andiamo infine le dipendenze sul controllo. Finché non termina, tutte le altre dipendono da questa. Utilizziamo sempre il branch predictor per predire il risultato. Se viene predetto che non si salterà, le istruzioni successive al salto vengono messe in esecuzione ed eseguite appena possibile producendo dei risultati. Se la predizione è sbagliata vanno buttati via i risultati (esecuzione speculativa) ed è quindi importante che il branch predictor sbagli il meno possibile. L'algoritmo di Tomasulo può essere esteso introducendo un **reorder buffer (ROB)** → coda: i sorgenti sono già pronti o riferiti tramite la stazione che produrrà il risultato, la destinazione va a finire nel ROB dove le istruzioni inseriscono il risultato in ordine di esecuzione → Memorizzo i dati finché non so se l'istruzione doveva essere eseguite o no → Si ha anche un flag che mi dice se l'istruzione è terminata e le istruzioni vengono ritirate solo dalla testa: **stadio di ritiro** → Monitora il ROB e vede quando è terminata l'istruzione in testa: in questo caso rende effettive le modifiche → Se l'istruzione in cima è operativa ci salva il risultato, se è un salto si sposta invece se avevamo indovinato o no. Se aveva indovinato il ROB sotto è valida, altrimenti si svuota tutto il ROB. Con la rinominazione dei registri bisogna ricordarsi 2 registri fisici per ogni registro logico: quello in corso di calcolo (speculativo) e quello che contiene l'ultimo valore che andava sicuramente calcolato (non speculativo), si ha una coda e se l'istruzione operativa arriva alla cima ci salva il risultato trasferendo i dati dal registro speculativo a quello non speculativo.

Le istruzioni di LOAD e STORE hanno gli stessi problemi di dipendenze ma sono più complicate dei registri: LOAD dst, base (reg)
STORE src, base (reg)

Più complicato perché l'indirizzo va sempre calcolato

↓

Eseguire in 2 fasi: calcolo indirizzo + esecuzione op.

Dipendenze viste dopo creare e viene in LOAD/STORE buffer

Inoltre, in caso di speculazione, devo tenere tutto in LOAD/STORE buffer finché non sono sicuro, altrimenti occuperei troppo spazio

Che succede, infine, in caso di speculazioni su interruzioni? Si salva tutto nel ROB senza però poterle eseguire e si eseguono solo se in cima al ROB.